

Embedded Operating System

LIAO Yong

Chapter 2 Kernel of ERTOS

1 Overview

University of Electronic Science & Technology of China (UESTC)

- Task management Scheduling Policy
- Synchronization, Communication, Mutex Coordinate mechanism
- Memory management Inventory management mechanism
- Interruption & Time management Event dealing mechanisms:
Event-triggered mechanism
Time-triggered mechanism

2 Task management

University of Electronic Science & Technology of China (UESTC)

- What does the kernel do ?

Scheduling 😊

- What does the kernel scheduling ?

Tasks

➤ **What is task'** 😊

(P141)

➤ **Characteristics of tasks**

- **Concurrence (Simultaneousness)** Scheduler
- **Independence & Asynchrony**
(dependence & Synchrony)
- **Dynamic**
-

2 Task management

University of Electronic Science & Technology of China (UESTC)

- **How to describe a task**   
 - **How to create a task in uc/OS**  **?**
 - **How to schedule tasks in the ready queue**  **?**
 - **How to switch tasks on (uc/OS II+ ARM 24)**   
 - **Which events will trigger the context switch?** 
- Scheduling Point**
- **Critical section accessing during scheduling**  

2 Task management

University of Electronic Science & Technology of China (UESTC)

➤ **C l a s s i f i c a t i o n o f t a s k s** (P143)

- **Periodic tasks**



- **Sporadic tasks**

- **Aperiodic tasks**

- ❖ **Critical tasks**

- ❖ **Non-critical tasks**

- ✓ **Hard real-time tasks**

- ✓ **Soft real-time tasks**

2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that **their deadlines** are met ?

➤ Tasks and their **special requirements** 😊

- Task number: n
- Task: T_i ($i=1,2,\dots,n$)
- Period: P_i
- Executing time: e_i
- Relative deadline: D_i
- Absolute deadline: d_i
- Release time: r_i
- Phasing: I_i

What's the difference between real-time task and process?

Time related parameters

2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that their deadlines are met ?
 - **Special measures for Guaranteeing task deadlines**
 - **Preemptive** 
 - **Periodic task**
 - **Relative deadline** (D_i) Period (P_i)
 - **Priority** is inversely related to the period (P_i)
 - **Other assumptions:**
 - 1) The cost of Preemption is negligible
 - 2) No shared resource-access during execution 
 - 3) Tasks are independent, no precedence constraints

2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that their deadlines are met ?
 - What does the scheduler do for *Guaranteeing* task deadlines ?
 - 1) **Which** task should be executed by the processor?
 - 2) **When** does a task begin to execute?
 - 3) **How long** is a task executing ?

2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that their deadlines are met ?
 - How to know a given scheduling policy can ***Guarantee*** task deadlines ?
 - ✦ **Schedulability analysis** on the above policy

Priority is inversely related to the period (P_i)



2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that their deadlines are met ?
 - **Necessary and sufficient conditions for schedulability**
 - ✓ **If $\min_{t \in \tau_i} \frac{W_i}{t} \leq 1$, task T_i is schedulable.**
 - ✓ **If $\max_{i \in \{1, \dots, n\}} \left\{ \min_{t \in \tau_i} \frac{W_i}{t} \right\} \leq 1$, for $i \in \{1, \dots, n\}$, then the entire set T is schedulable.**

$$\tau_i = \{ \lfloor P_i \rfloor \mid j=1,2,\dots,i; \quad l=1,2,\dots, \lfloor P_i / P_j \rfloor \}$$

$$i = \{1, 2, \dots, n\}$$

2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that their deadlines are met ?
 - Properties of **the scheduling policy**
 - A set of tasks is said to **fully utilize the processor** 
 - Theorem: Any set of n -periodic tasks that fully utilizes the processor under **the scheduling policy** must have a processor utilization  at least n
 - **($2^{(1/n)} - 1$)** **This scheduling policy** is called **RM (Rate-Monotonic)** scheduling algorithm 

C. Liu, Layland, "Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment", Journal of ACM, 1973.

2 Task management

University of Electronic Science & Technology of China (UESTC)

- How to schedule tasks on processor to ensure that their deadlines are met ?
 - How to use the **Theorem of RM scheduling algorithm** ?
 - What can we learn from RM scheduling algorithm ?
 - How to handling critical section 
 - Priority inversion  Unpredictability of RM
 - Priority inheritance  
 - Priority ceiling  (Reading...)
 - Theorem: Any set of n periodic tasks that fully utilizes the processor under **the RM scheduling policy** must have, for each $i \in \{1, 2, \dots, n\}$

$$e_1/P_1 + e_2/P_2 + \dots + e_i/P_i + b_i/P_i \leq i(2^{1/i} - 1)$$

2 Task management

University of Electronic Science & Technology of China (UESTC)

- Other problems to be considered
 - **Handling critical sections**
 - **If the tasks are not periodic**
 - Reserve a fictitious highest priority task for sporadic tasks 😊
 - **DS (Deferred Server) Policy** 😊
 - **EDF (Earliest Deadline First) Policy** (p193)
 - 1) Schedule the task whose absolute deadline is the earliest
 - 2) No periodic assumption
 - 3) Necessary and sufficient conditions for schedulability
- It also works for periodic task

2 Task management

University of Electronic Science & Technology of China (UESTC)

- Other problems to be considered
 - **EDF (Earliest Deadline First) Policy**
 - 4) **Drawbacks of EDF** (p193)
 - 5) **Dynamic scheduling & Static scheduling** (p189)
 - **DM (Deadline Monotonic)**
 - **Real-time queue theory & other methods**  (p194)
 - **Allowing for precedence and exclusion conditions**
 - **WCET (Worst Case Execution Time) (In retrospect to.....)**
 - **How to determine the estimated execution time ?**
 - **Actual execution time**

e_e
 e_a

2 Task management

University of Electronic Science & Technology of China (UESTC)

- Is real-time **scheduling policy** enough to ensure the task timeliness (especial for hard real-time tasks)
 - **The kernel must respond to external stimuli timely, but how ... ?**

Response time is the time  kernel takes to recognize and respond to an external event.

- **Some mechanisms to improve timeliness (Reading...)**
 - **Preemptive scheduling and Preemptive kernel**
 - **Shielding tim**   **erruption**
 - **Data structure**
 - **Memory management**
 - **Restrictive waiting time for resource**
 - **Priority ceiling**

2 Task management

University of Electronic Science & Technology of China (UESTC)

- Is **real-time scheduling policy** enough to ensure the task timeliness (especial for hard real-time tasks)
 - **Other mechanisms to improve timeliness**
 - **Interruption handling**
 - **Floating-point operation**
 - **Scheduling policy & mechanism**
 - **Factors that trigger scheduling**
 - **Performance metrics of ERTOS**
- **Multi-processor/Multi-core scheduling**

3 Exclusiveness , Synchronization, Communication

University of Electronic Science & Technology of China (UESTC)

- Exclusiveness (mutual exclusion) 
 - Binary semaphore 
 - Mutex 
 - Counting semapho 
 - Synchronization 
 - Semaphore for Synchronizat 
 - Event mechanism for Synchroniza 
 - Asynchronization by signal 
 - Communication
 - Message 
 - Mail
- Configurable ("OS_CFG.H" in *uC/OS II*)**

4 Interruption (Event triggered mechanism)

University of Electronic Science & Technology of China (UESTC)

■ Classification of interruption

(p185)

➤ **Interruption**

1) Outside interruption

2) Hard interruption

➤ **Trap**

1) Inside interruption (INT 21H, SWI ?)

2) Soft interruption

➤ **Exception**

```
#define OS_TASK_SW() asm INT 080H
```

4 Interruption (Event triggered mechanism)

University of Electronic Science & Technology of China (UESTC)

- Case1: interruption on (ARM 2440+uc/OS II) 😊
- Case2: interruption of ARM11MPCORE
 - **Distributed Interrupt Controller**
 - 1) Interrupt Distributor
 - 2) CPU Interfaces
 - **Generic Interrupt controller** 😊
 - **Respond the interruption** 😊
 - **Initialize the interrupt** 😊

5 Time management (Time triggered mechanism)

University of Electronic Science & Technology of China (UESTC)

- Time management is important (p219)
- Real-time Clock (RTC)
- System Clock (Timer)
- Size of system clock 
 - **Tick (the period of output impulse generated by the timer)**
- Time management (p222)
 - **Task Delay (uC/OS II: *OSTimeDly()*** 
 - **Difference time chain** 
 - **Watchdog timer** 
 - **System clock IS** 

(p222)



6 Memory & I/O management

University of Electronic Science & Technology of China (UESTC)

- What is the memory management is special ? (p266)
- Mechanisms of memory management (Reading...)
 - **Fixed size**
 - **Alterable size**
 - **MMU (Memory Management Unit) & Memory protection**

6 Memory & I/O management

University of Electronic Science & Technology of China (UESTC)

■ I/O management

- Architecture (p277)
 - Difference between General OS & ERTOS?
 - Advantages of architecture (p278)
 - Calling modes 😊
- Implementation of I/O
 - API
 - Device Management
 - ✓ Diver Address Table 😊
 - ✓ Device Name Table 😊
 - ✓ File Table 😊
 - Driver logic 😊
 - H/W Abstract (p284)

6 Memory & I/O management

University of Electronic Science & Technology of China (UESTC)

■ I/O management

- **How to use I/O call provided by the ERTOS ? (p285)**
 - **Register the driver**
 - **Register the device**
 - **Open**
 - **Write**
 - **.....**
- **What's the difference between ERTOS-based driver development and none-ERTOS-based driver development ?**
- **Brief introduction to SPI driver**

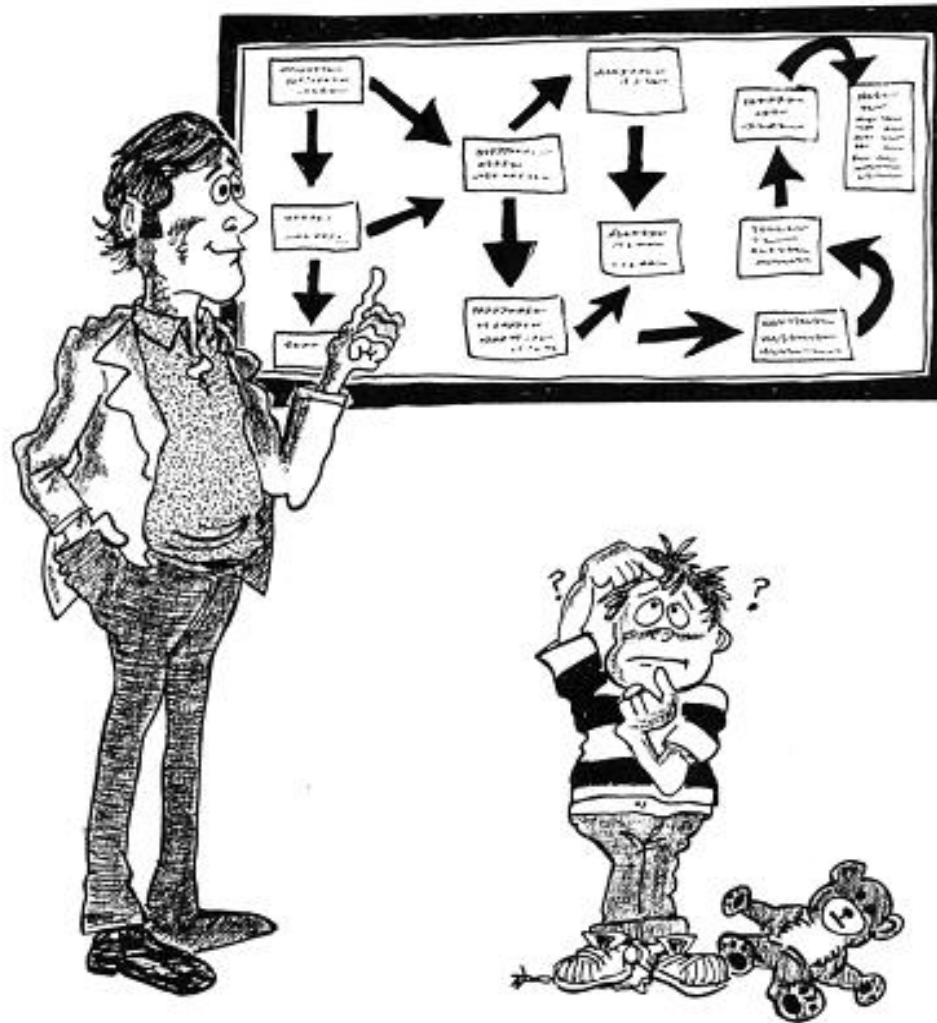
7 Others

University of Electronic Science & Technology of China (UESTC)

■ Questions ?



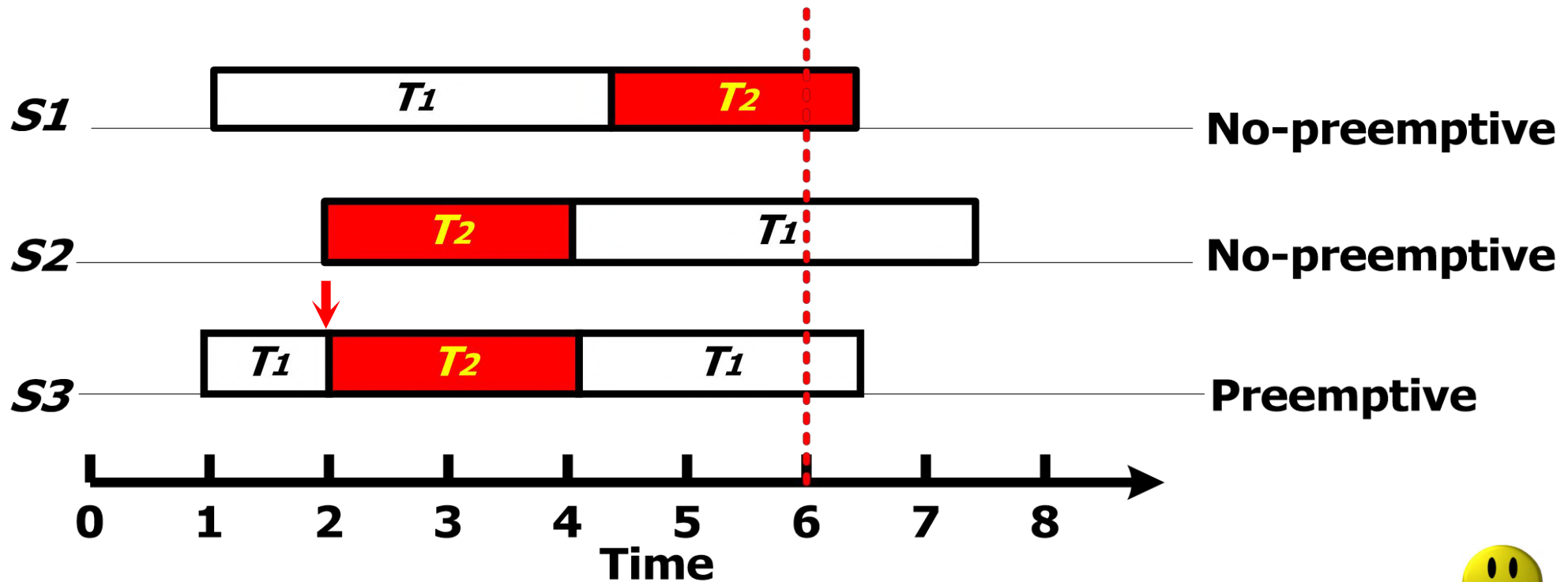
"I'm looking at my schedule.
Before, during, and after lunch is no good."



Priority(T_1) < Priority(T_2)

$A_1=1, A_2=2$

Task 2 deadline



Which one is the highest priority task?

University of Electronic Science & Technology of China (UESTC)



$$A_1 = 0$$

$$A_2 = 1$$

$$A_3 = 3$$

$$E_1 = 0.5$$

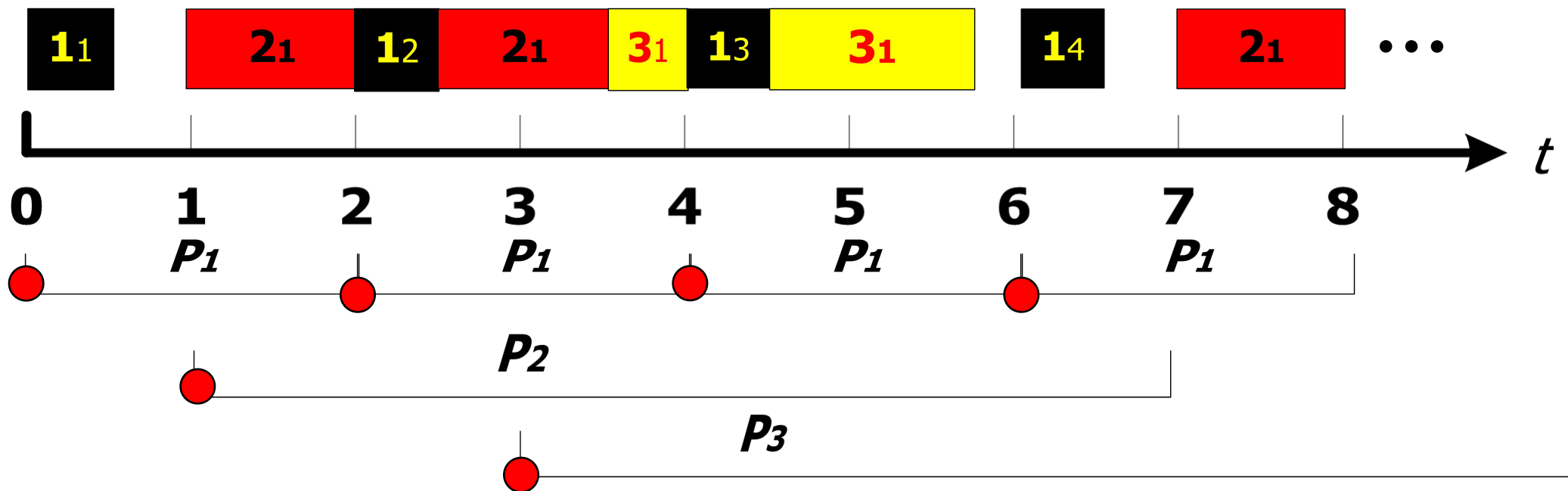
$$E_2 = 2$$

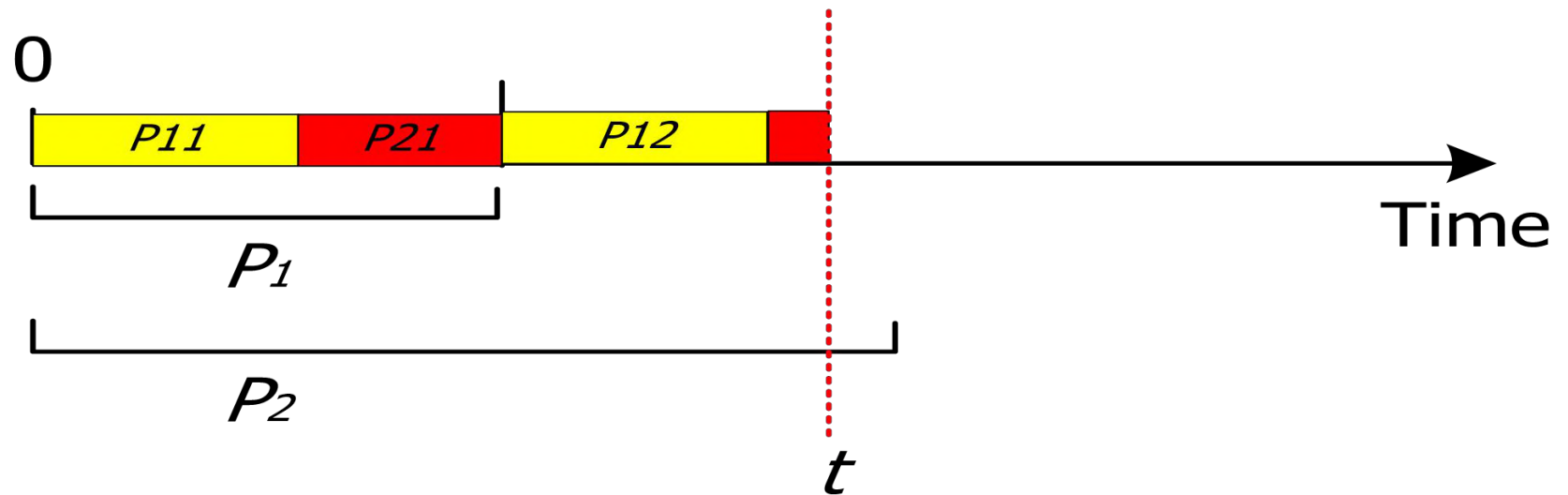
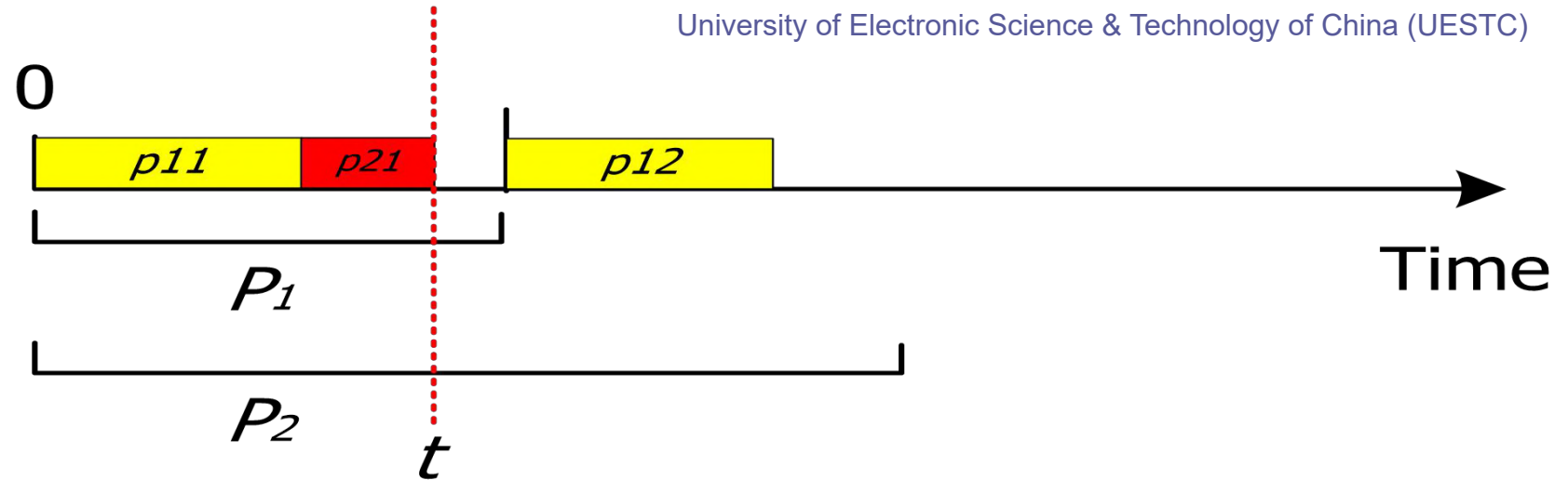
$$E_3 = 1.75$$

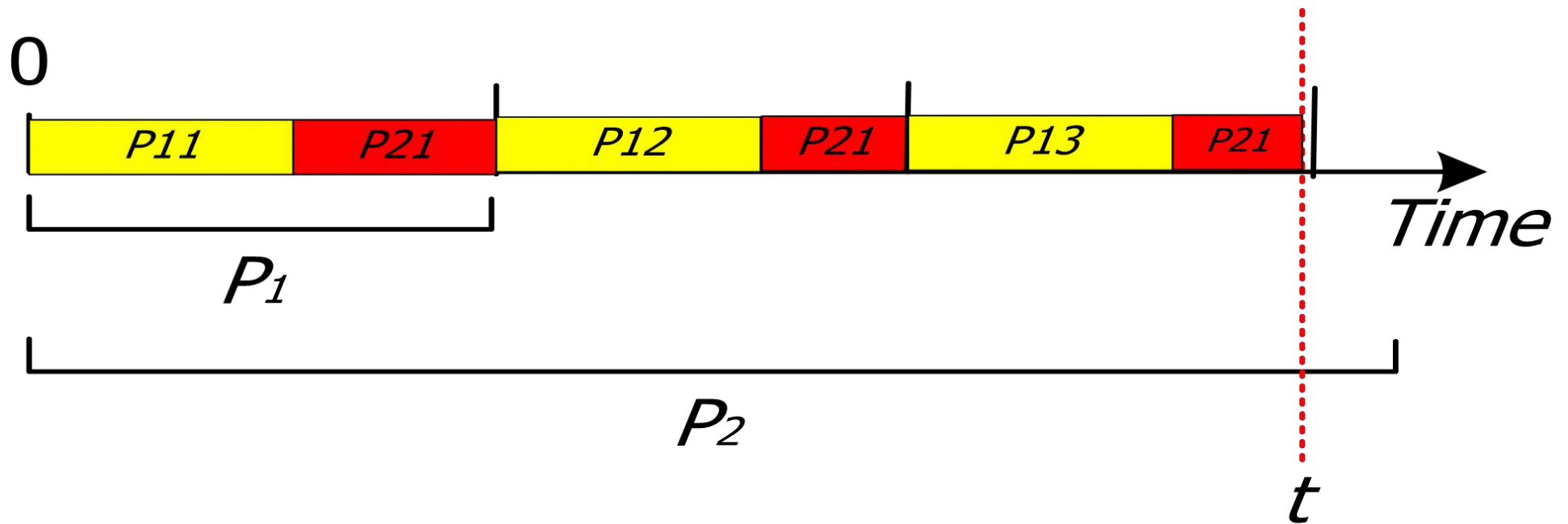
$$P_1 = 2$$

$$P_2 = 6$$

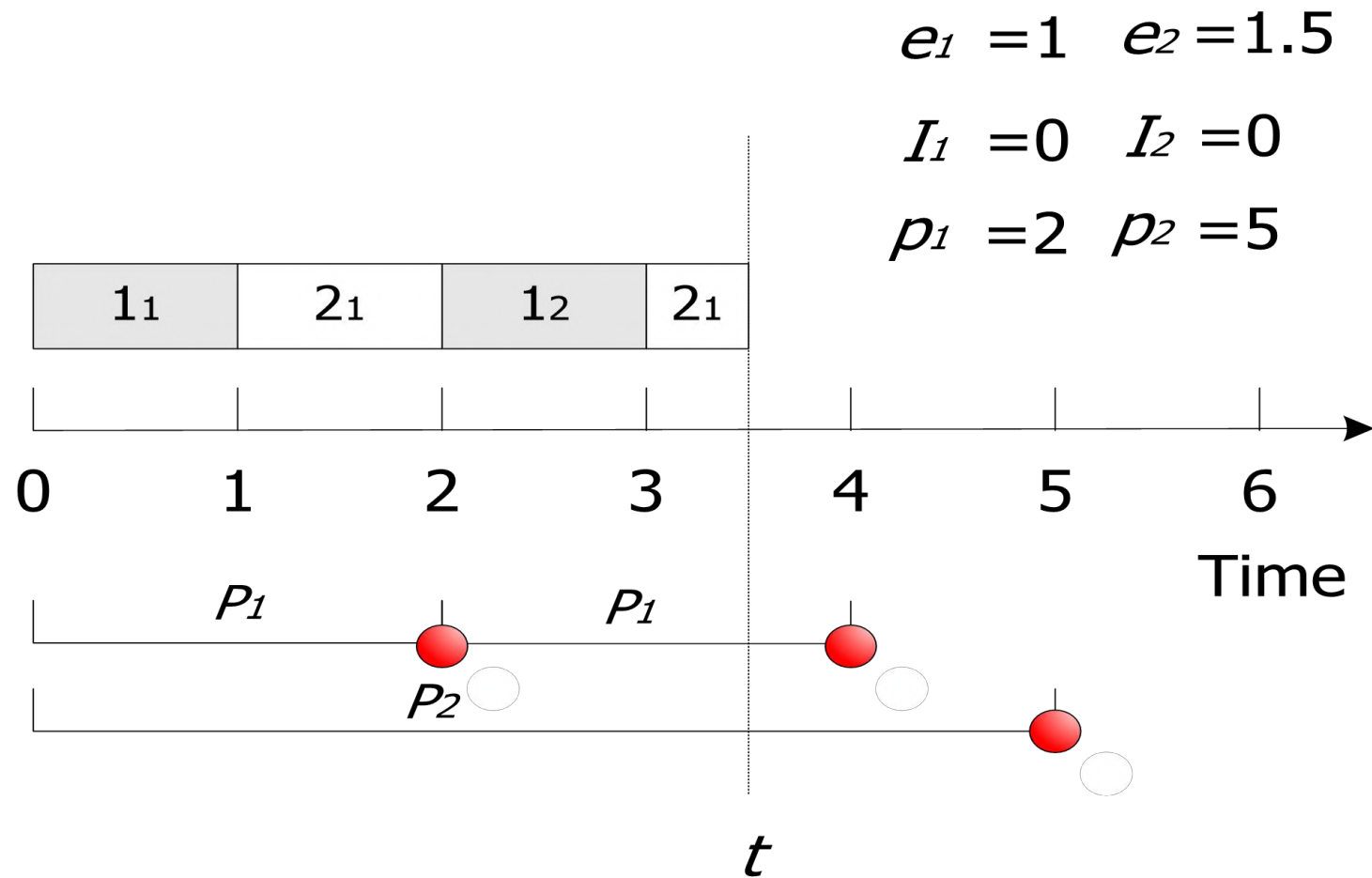
$$P_3 = 8$$

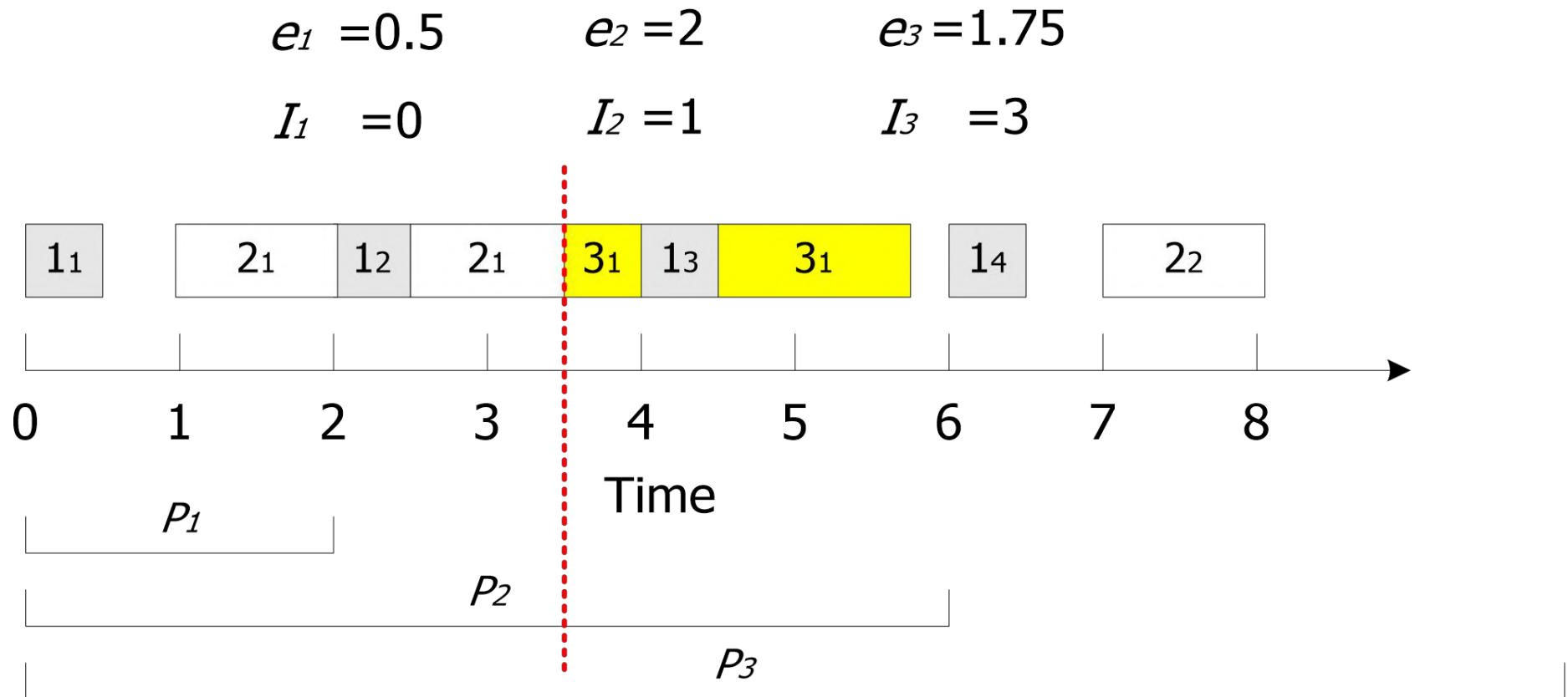




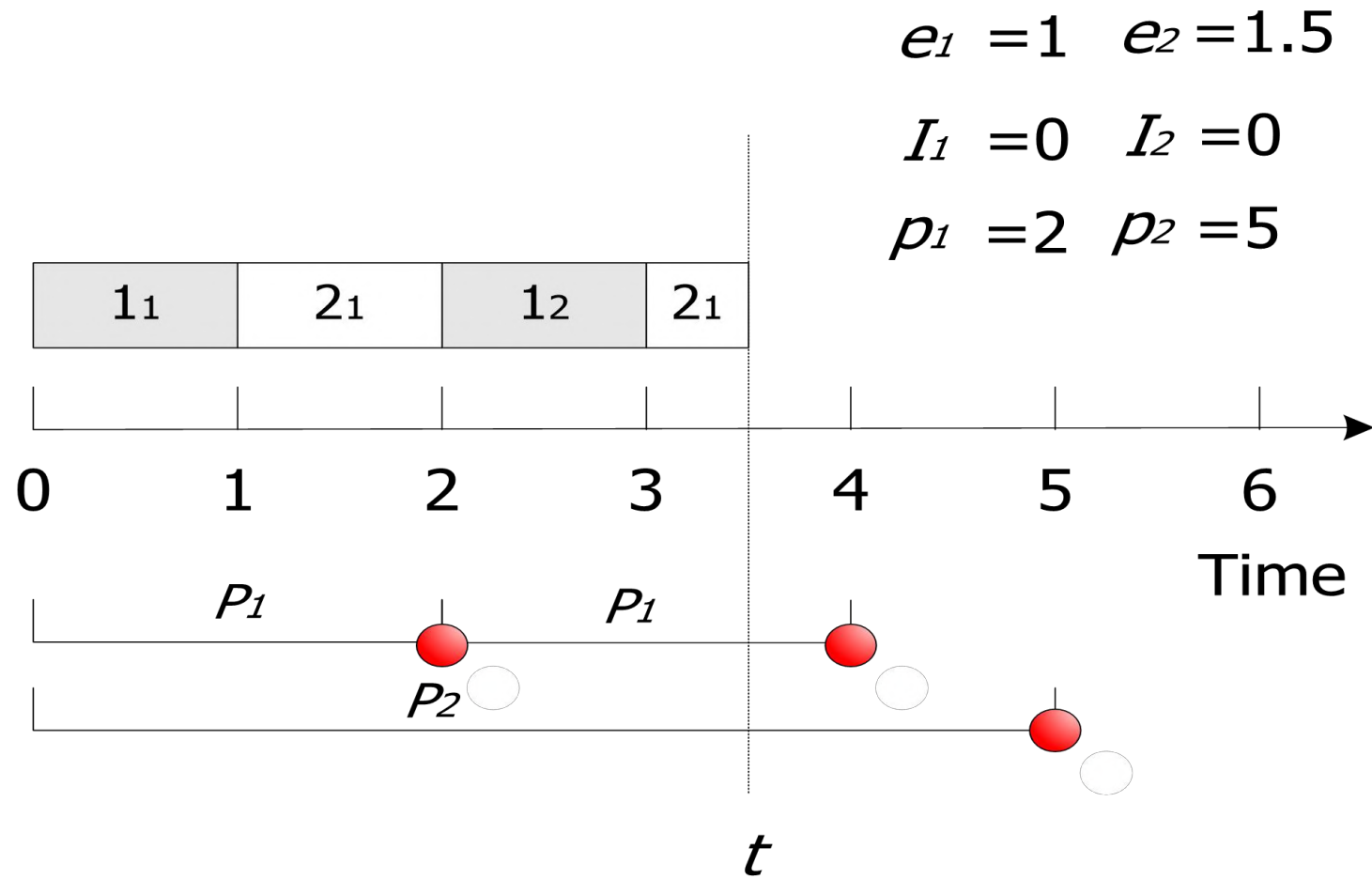


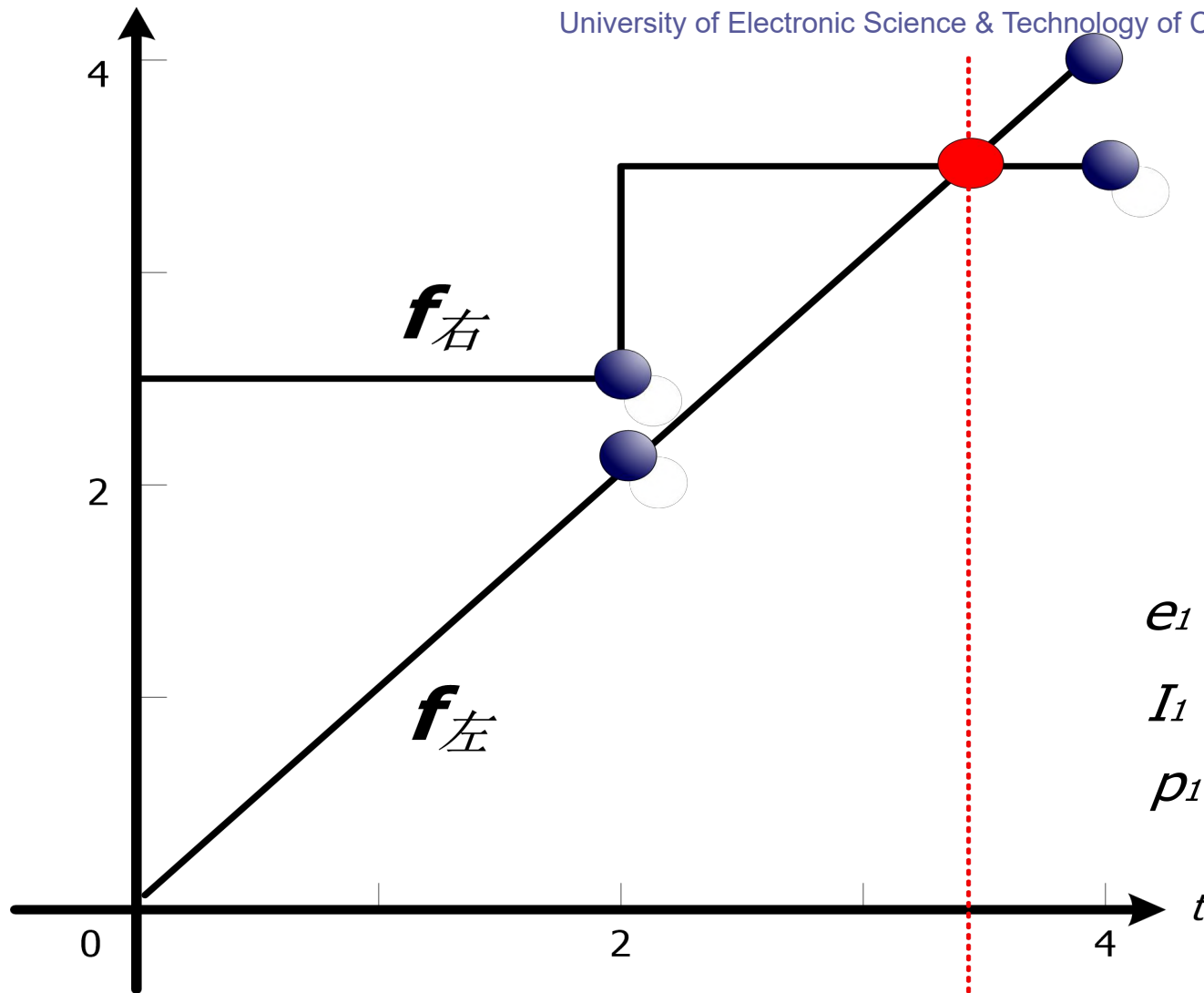
University of Electronic Science & Technology of China (UESTC)





University of Electronic Science & Technology of China (UESTC)

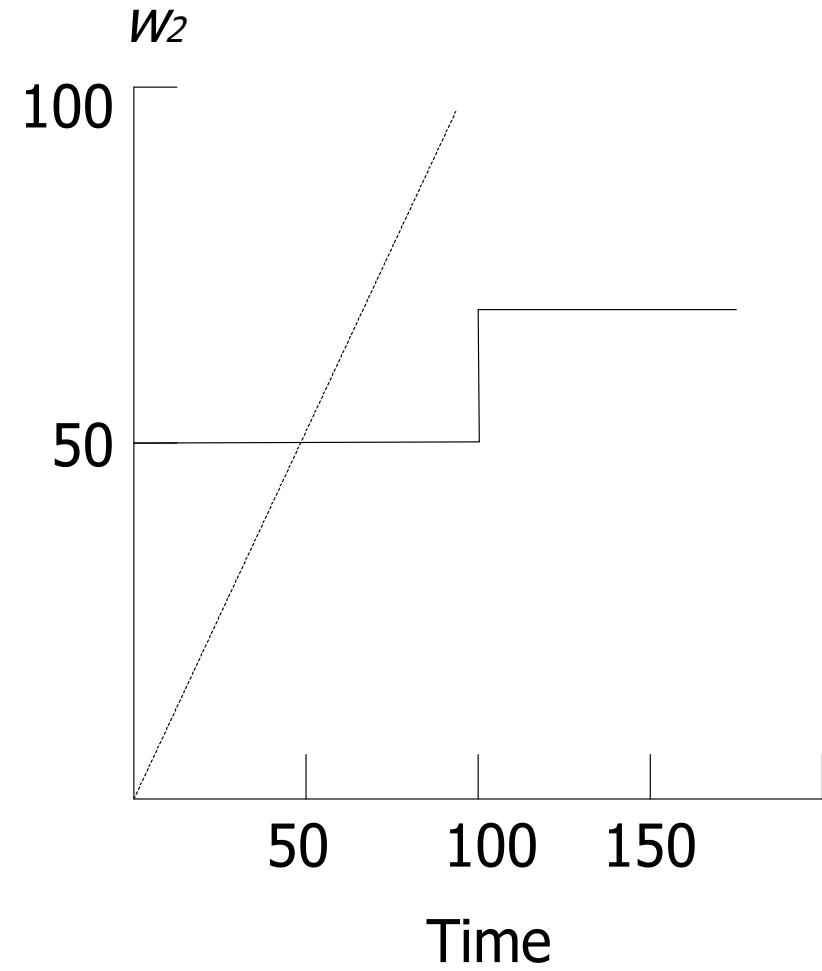
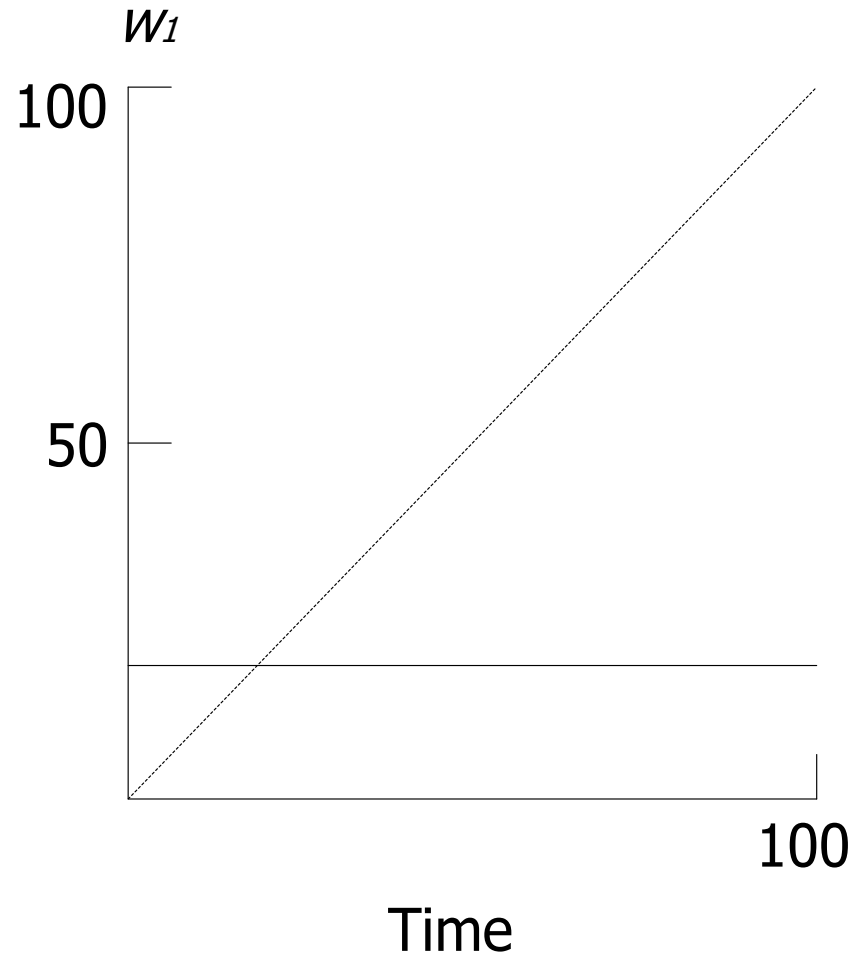


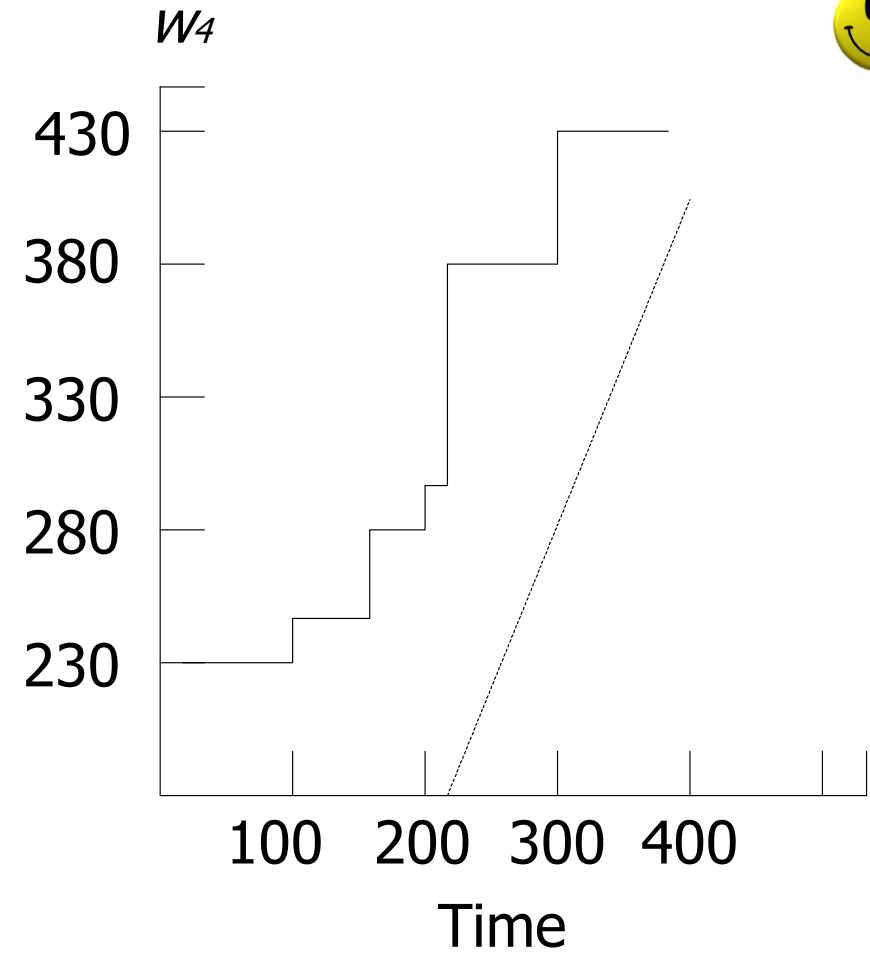
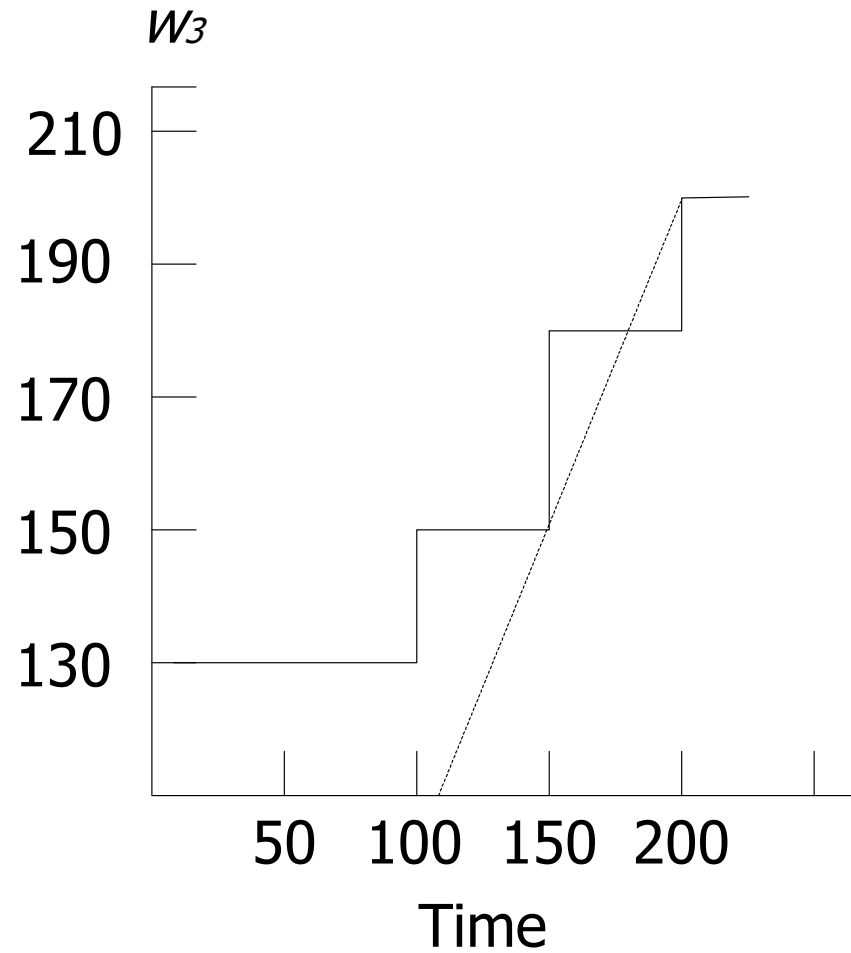


$$e_1 = 1 \quad e_2 = 1.5$$

$$I_1 = 0 \quad I_2 = 0$$

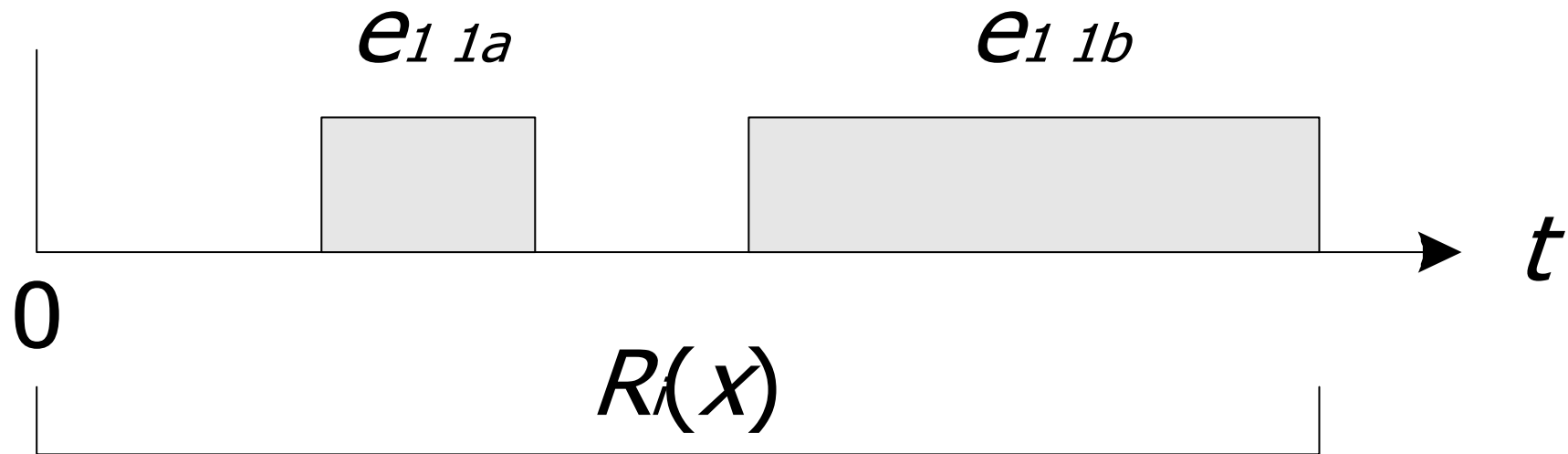
$$p_1 = 2 \quad p_2 = 5$$

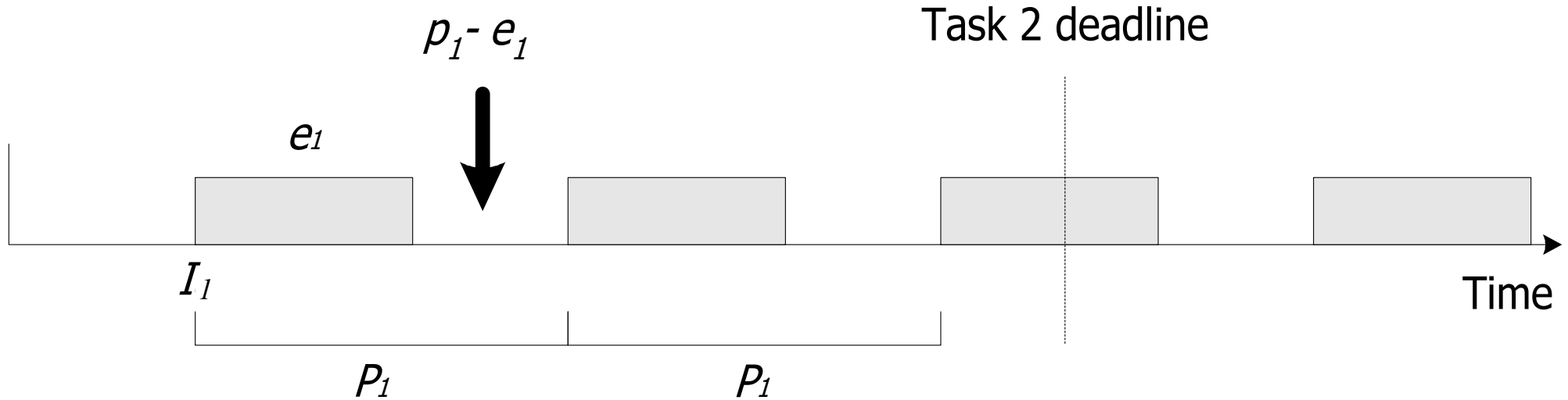






$$e_{11a} + e_{11b} = e_{11}$$

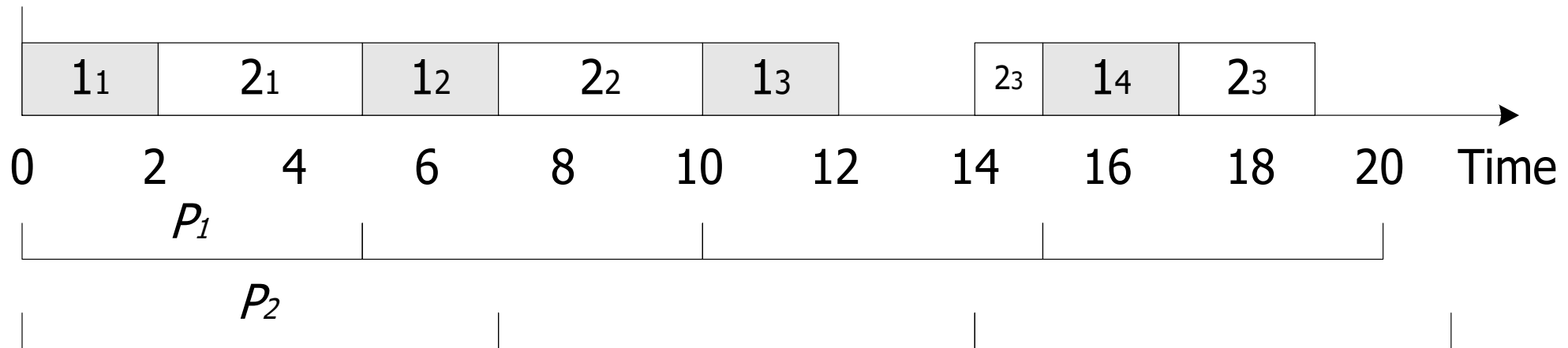




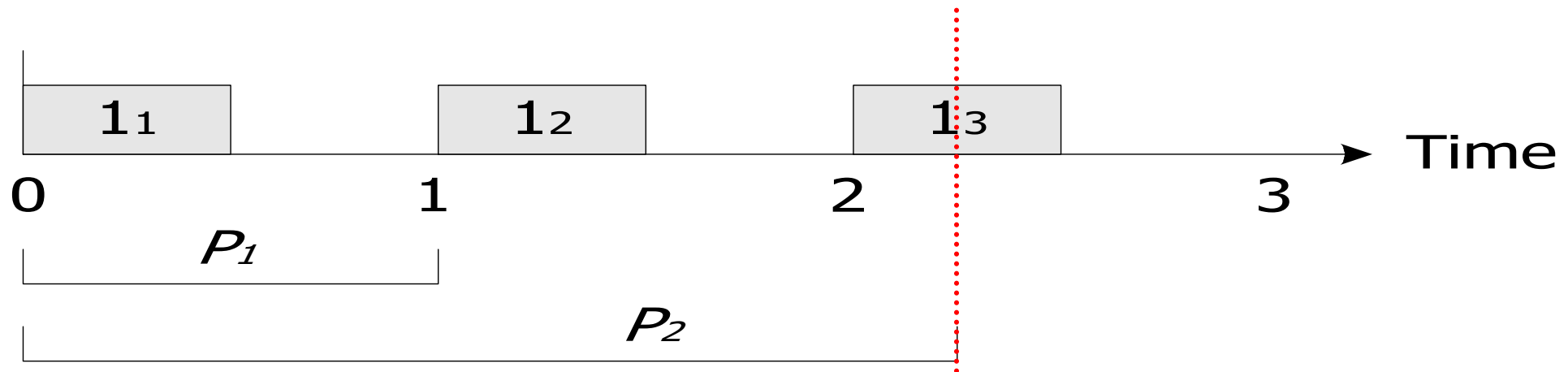
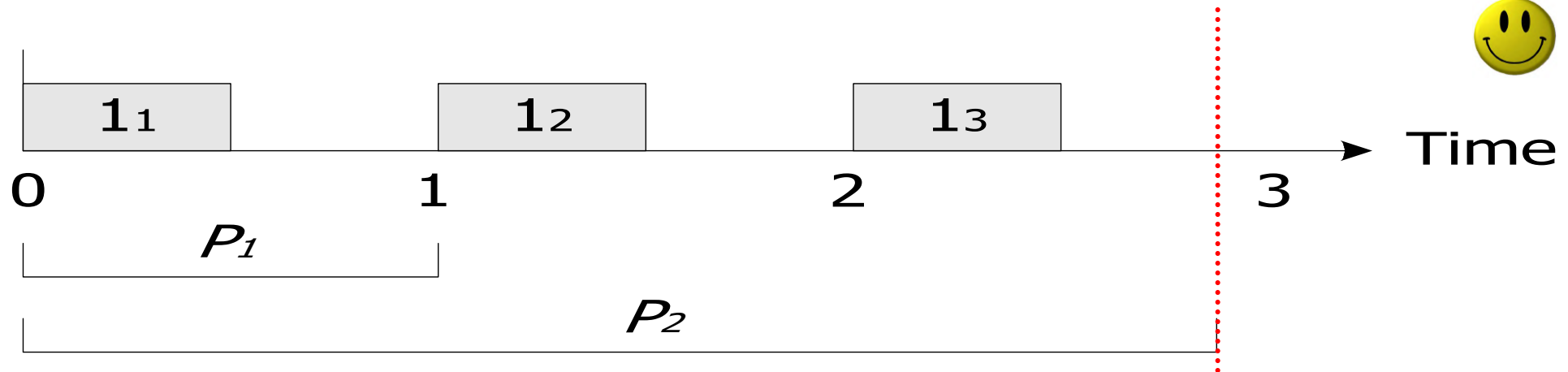


$$e_1 = 2 \quad e_2 = 3$$

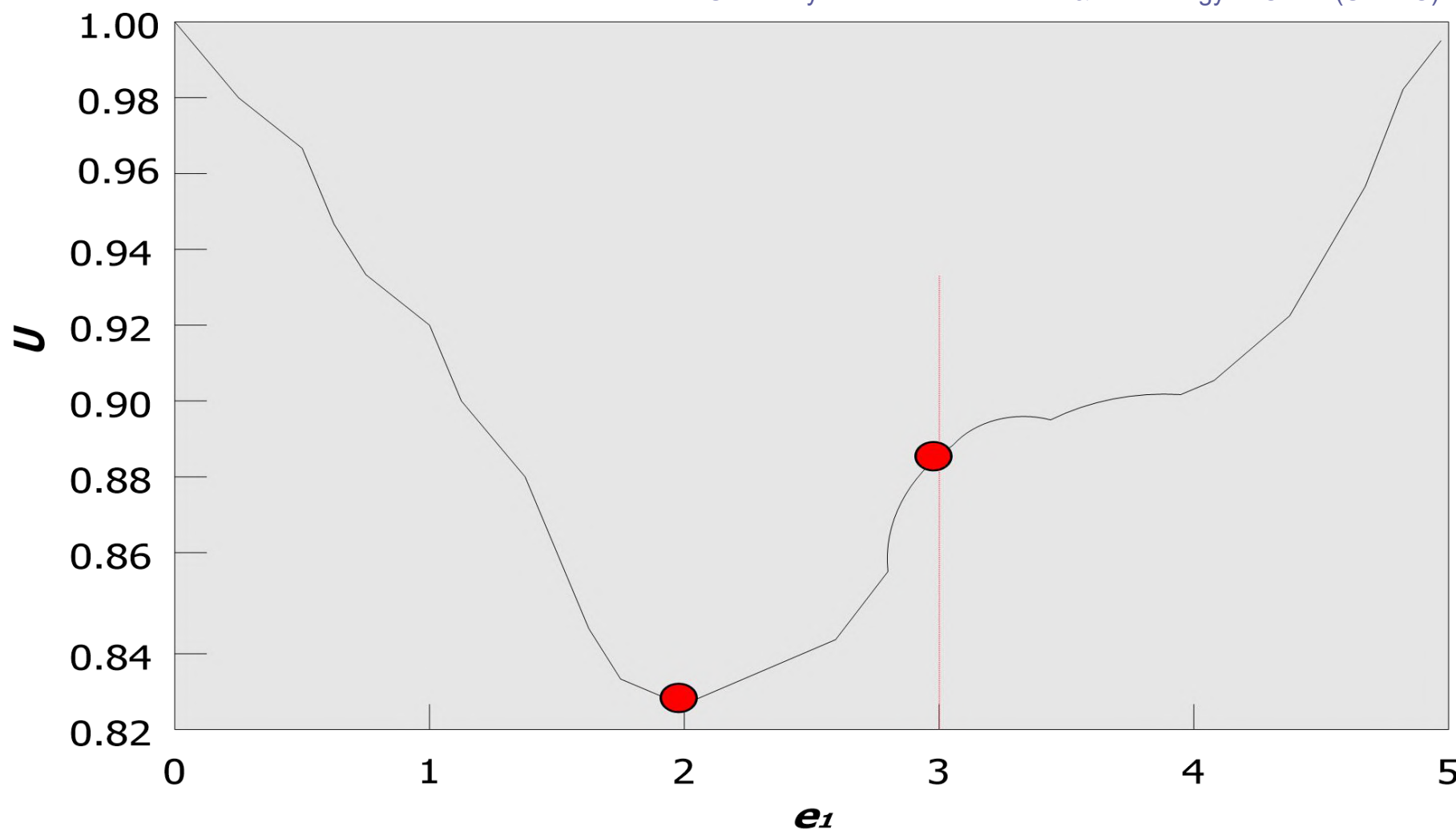
$$I_1 = 0 \quad I_2 = 0$$



$$U = 2/5 + 3/7 = 0.83$$

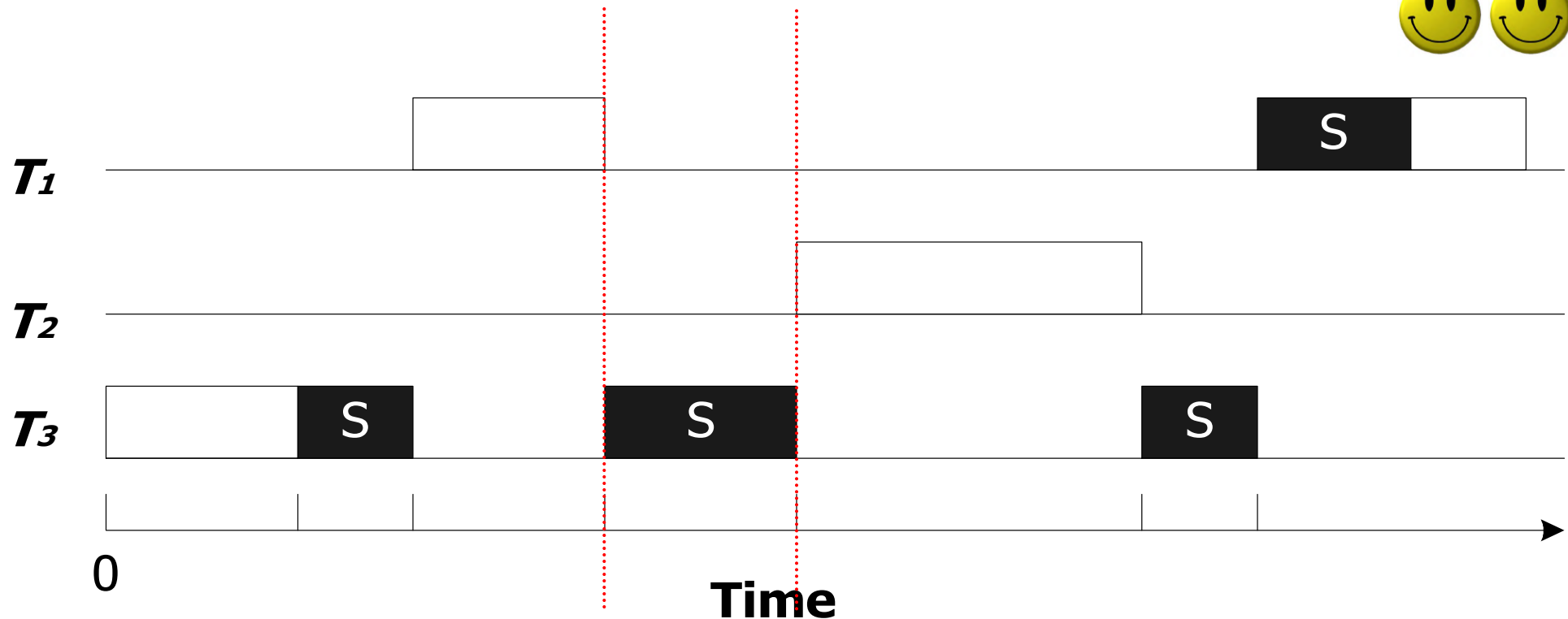


University of Electronic Science & Technology of China (UESTC)

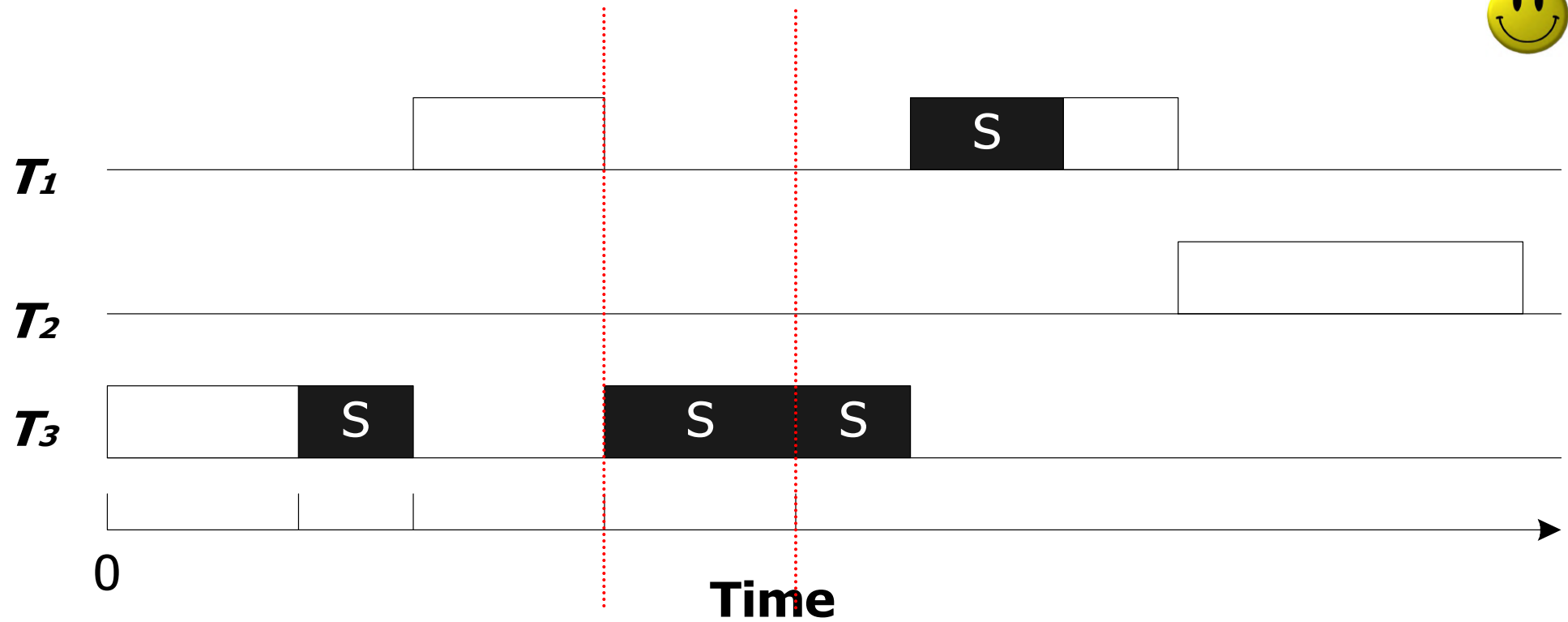




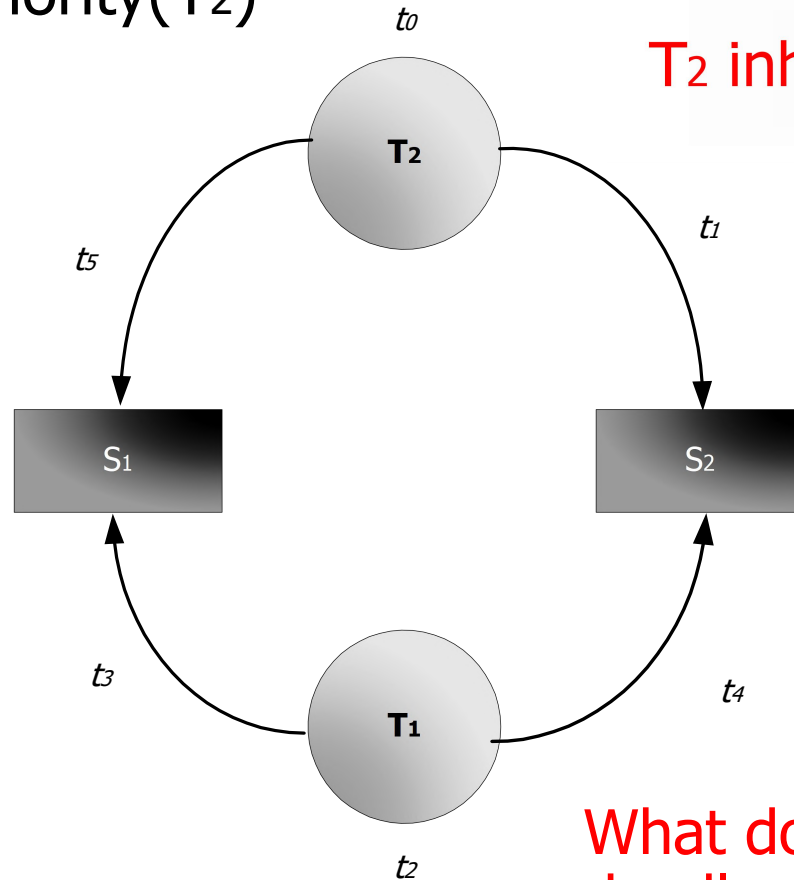
n	Upper bound for schedulability
1	1
2	0.828
3	0.780
4	0.757
5	0.743
6	0.735
...	...
∞	$\ln 2$



Note that although T_2 is of lower priority than T_1 , it was able to delay T_1 indirectly, this phenomenon is known as **priority inversion**



Priority(T_1) > Priority(T_2)



T_2 inherits the priority of T_1

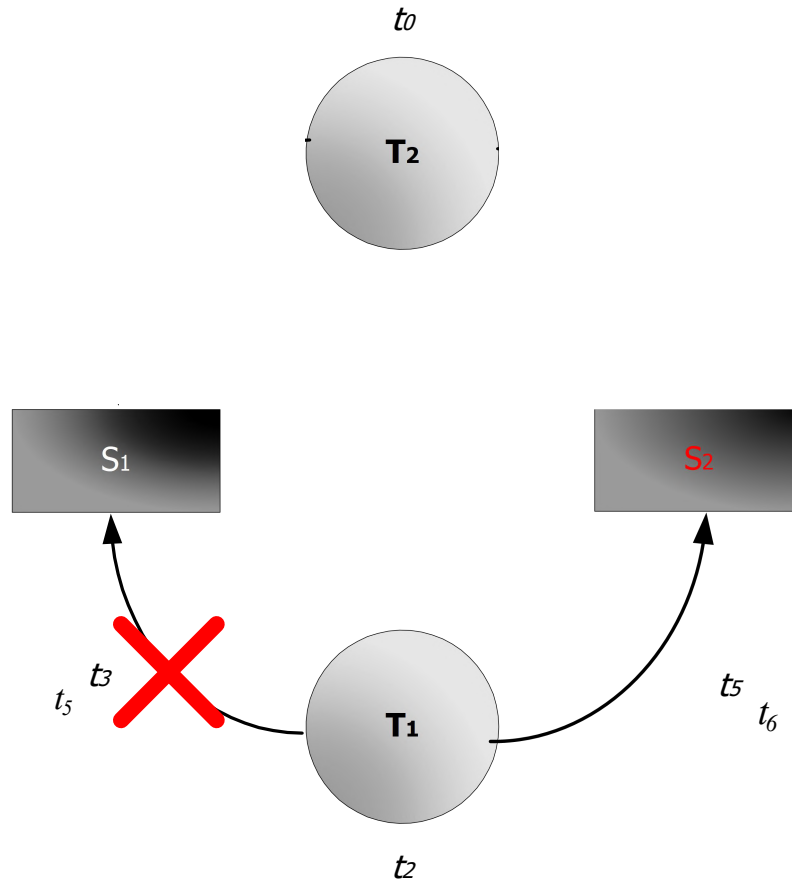
Deadlock !!

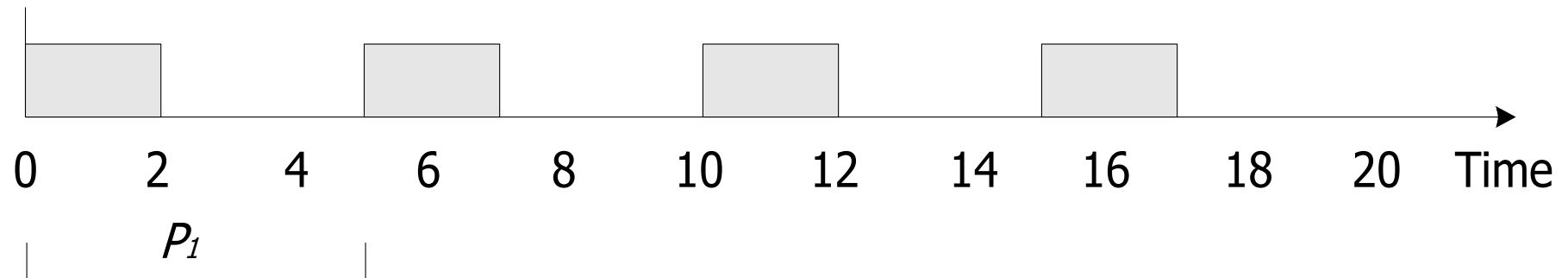
What does this differ from the deadlock happened in Windows ?



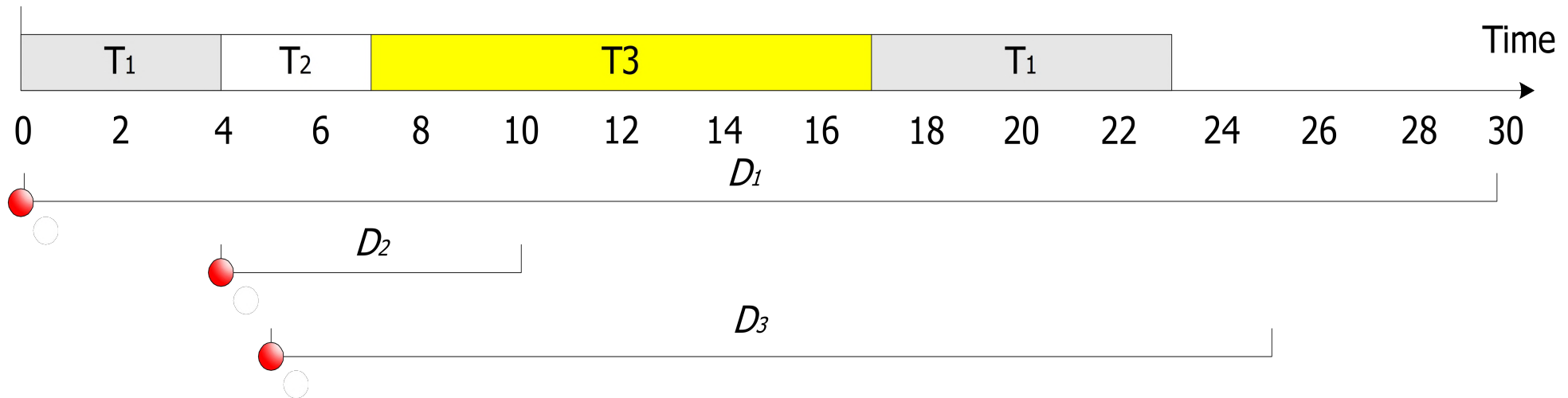
Critical section	Accessed by	Priority ceiling
S_1	T_1, T_2	$P(T_1)$
S_2	T_1, T_2, T_3	$P(T_1)$
S_3	T_3	$P(T_3)$
S_4	T_2, T_3	$P(T_2)$

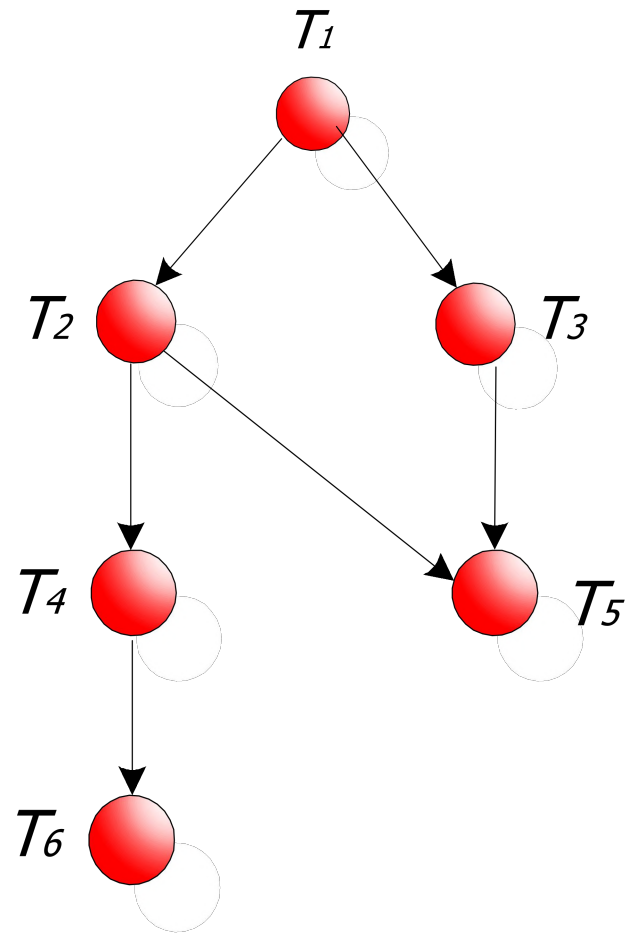
Both S1 and S2 will be accessed by T1 and T2



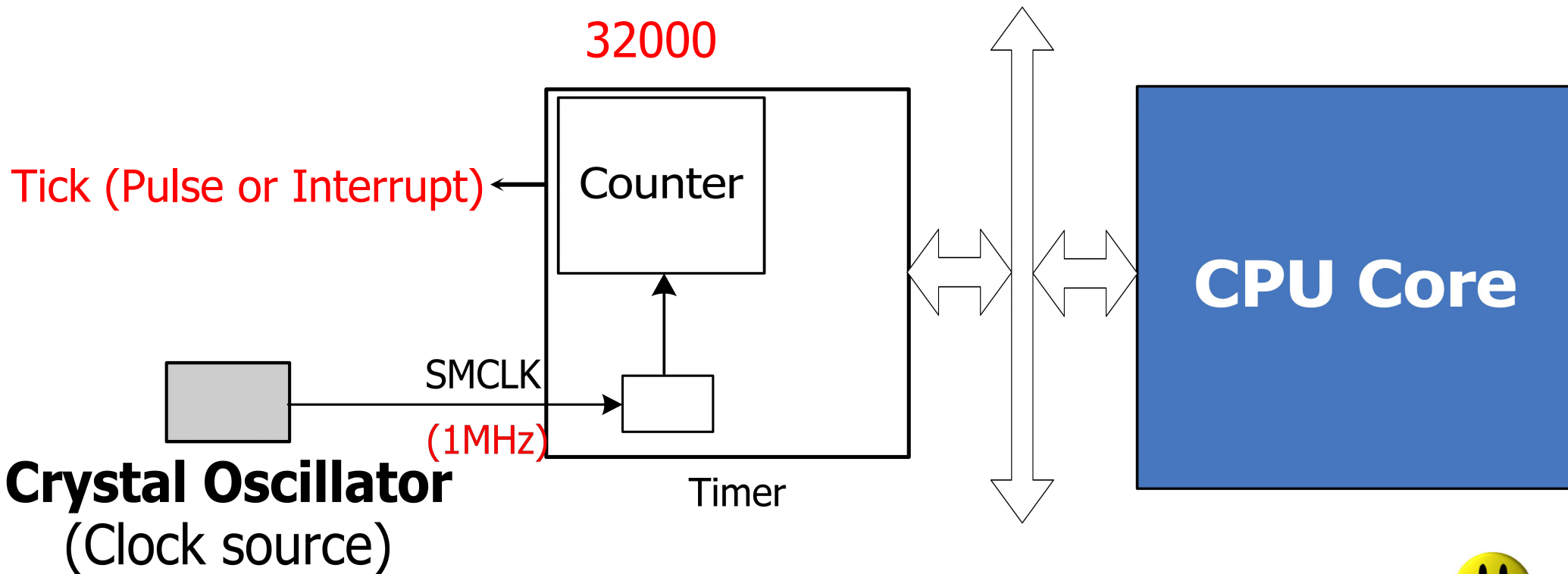


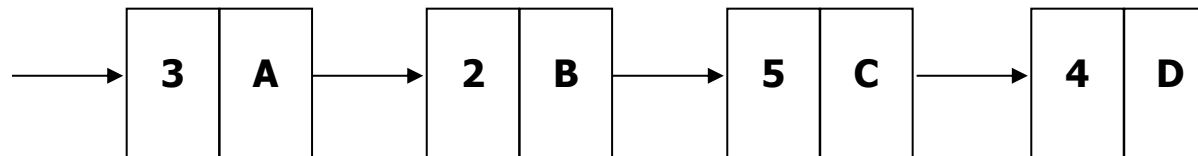
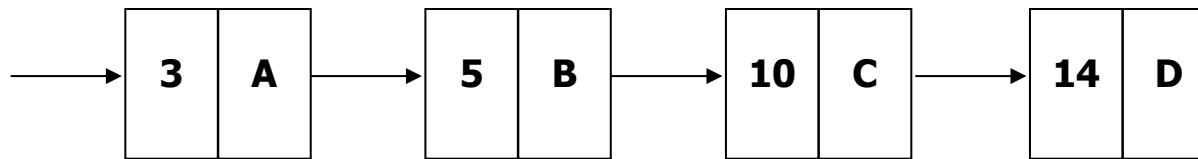
Task	Arrival time	Execution time	Absolute deadline
T_1	0	10	30
T_2	4	3	10
T_3	5	10	25



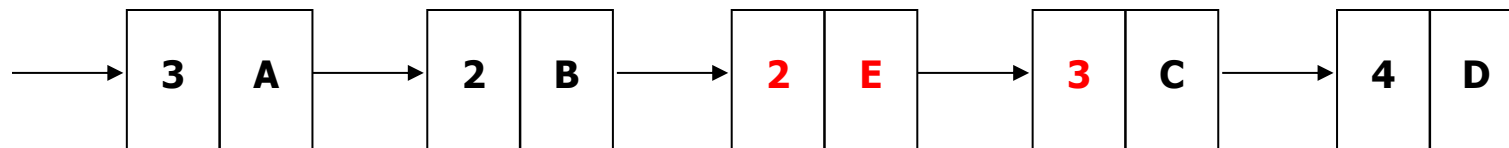


What's the initial value of the Counter if we config Tick = 32ms ?





How to insert an event into the chain ?





```
wdId=wdCreate()
```

```
wdStar(wdId,sysClkRateGet(), DEADLINE_TIME, (FUNCTION)deadlineHandler)
```

Watchdog timer



$$\tau_1 = \{100\}$$

$$\tau_2 = \{100, 150\}$$

$$\tau_3 = \{100, 150, 200, 210\}$$

$$\tau_4 = \{100, 150, 200, 210, 300, 400\}$$



$$\boxed{20} \quad e_1 \leq 100$$

$$\boxed{50} \quad e_1 + e_2 \leq 100 \quad \text{or} \quad \boxed{70} \quad 2e_1 + e_2 \leq 150$$

$$\boxed{130} \quad e_1 + e_2 + e_3 \leq 100 \quad \text{or} \quad \boxed{150} \quad 2e_1 + e_2 + e_3 \leq 150$$

$$\boxed{180} \quad 2e_1 + 2e_2 + e_3 \leq 200 \quad \text{or} \quad \boxed{200} \quad 3e_1 + 2e_2 + e_3 \leq 210$$

$$\boxed{230} \quad e_1 + e_2 + e_3 + e_4 \leq 100 \quad \text{or} \quad \boxed{250} \quad 2e_1 + e_2 + e_3 + e_4 \leq 150$$

$$\boxed{280} \quad 2e_1 + 2e_2 + e_3 + e_4 \leq 200 \quad \text{or} \quad \boxed{300} \quad 3e_1 + 2e_2 + e_3 + e_4 \leq 210$$

$$\boxed{380} \quad 3e_1 + 2e_2 + 2e_3 + e_4 \leq 300 \quad \text{or} \quad \boxed{430} \quad 4e_1 + 3e_2 + 2e_3 + e_4 \leq 400$$

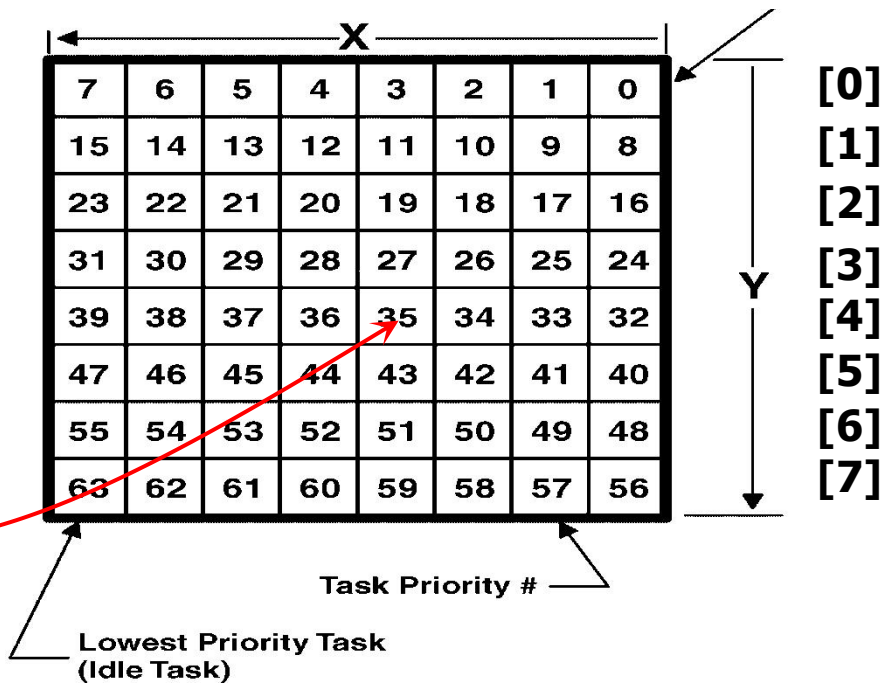
INT8U PriorityReadyGroup

00000000 University of Electronic Science & Technology of China (UESTC)

INT8U PriorityReadyTable[1] = 01010000

INT8U PriorityReadyTable[X] 00000000

35: 00100011



P100

subscript	Value
0	00000001
1	00000010
2	00000100
3	00001000
4	00010000
5	00100000
6	01000000
7	10000000

PriorityReadyTable[4]

35: 00100011

PriorityReadyGroup

PriorityMapTable []

(for PriorityReadyGroup & PriorityReadyTable[])

3: 00000011



When a task switch to the "ready" state, what happens?

Switch to the ready state
Quit the ready state? 😊

① 00010000
② 00001000
35: 00100 011 PriorityReadyGroup

7 6 5 4 3 2 1 0

PriorityReadyTable[4]

7	6	5	4	3	2	1	0	
15	14	13	12	11	10	9	8	[0]
23	22	21	20	19	18	17	16	[1]
31	30	29	28	27	26	25	24	[2]
39	38	37	36	35	34	33	32	[3]
47	46	45	54	43	42	41	40	[4]
55	54	53	52	51	50	49	48	[5]
63	62	61	60	59	58	57	56	[6]
								[7]

PriorityReadyGroup
= 01000001

PriorityReadyTable[4]
= 01000001



Quit the ready state

① ~~00001000~~ → 11110111

② 00010000 → 11101111

35: 00100 011

PriorityReadyGroup

7 6 5 4 3 2 1 0

PriorityReadyTable[4]

7	6	5	4	3	2	1	0	
7	6	5	4	3	2	1	0	[0]
15	14	13	12	11	10	9	8	[1]
23	22	21	20	19	18	17	16	[2]
31	30	29	28	27	26	25	24	[3]
39	38	37	36	35	34	33	32	[4]
47	46	45	54	43	42	41	40	[5]
55	54	53	52	51	50	49	48	[6]
63	62	61	60	59	58	57	56	[7]

PriorityReadyGroup

= 01010001

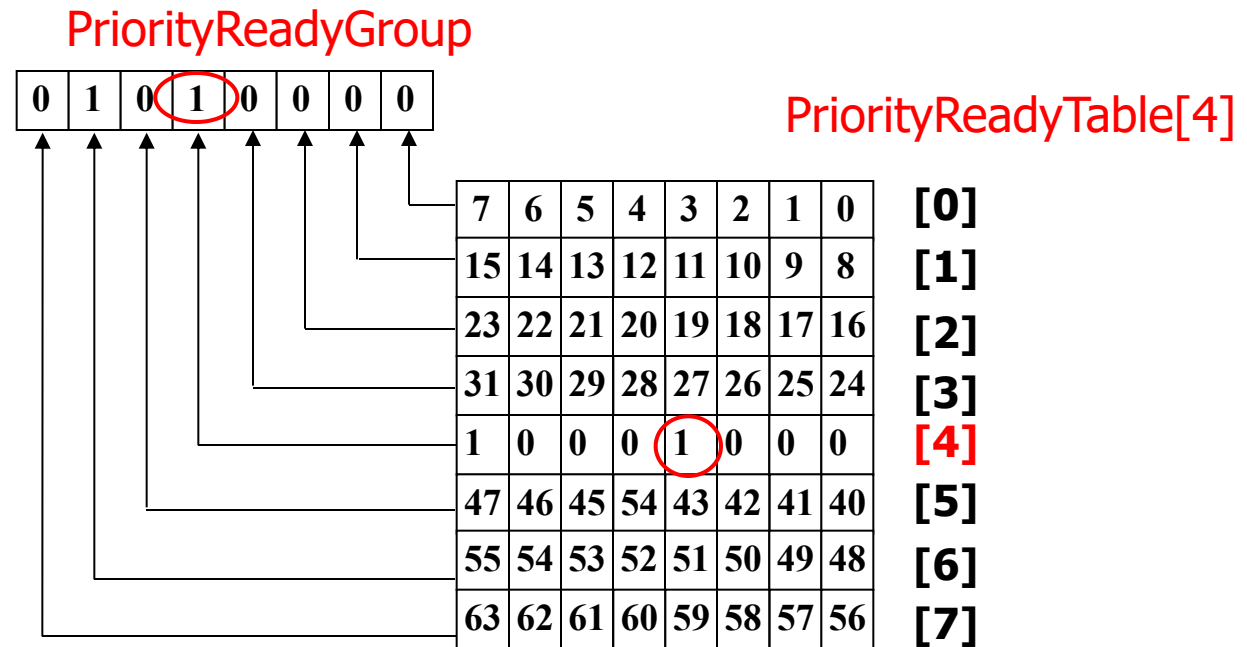
PriorityReadyTable[4]

= 01001001

How to get the highest priority from the "ready" tasks?



35: 00100 011



PriorityDecisionTable[].....

PriorityDecisionTable [256] = {

0, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x00 to 0x0F */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x10 to 0x1F */
5, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x20 to 0x2F */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x30 to 0x3F */
6, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x40 to 0x4F */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x50 to 0x5F */
5, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x60 to 0x6F */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x70 to 0x7F */
7, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x80 to 0x8F */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0x90 to 0x9F */
5, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0xA0 to 0xAF */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0xB0 to 0xBF */
6, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0xC0 to 0xCF */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0xD0 to 0xDF */
5, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0xE0 to 0xEF */
4, 0, 1, 0, 2, 0, 1, 0, 3, 0, 1, 0, 2, 0, 1, 0,	/* 0xF0 to 0xFF */

};

0x12 PriorityReadyGroup (00010010)

0x50 ?
0x88 ?

Get the highest priority



Check the PriorityDecisionTable[]

PriorityReadyGroup

PriorityReadyTable[4]

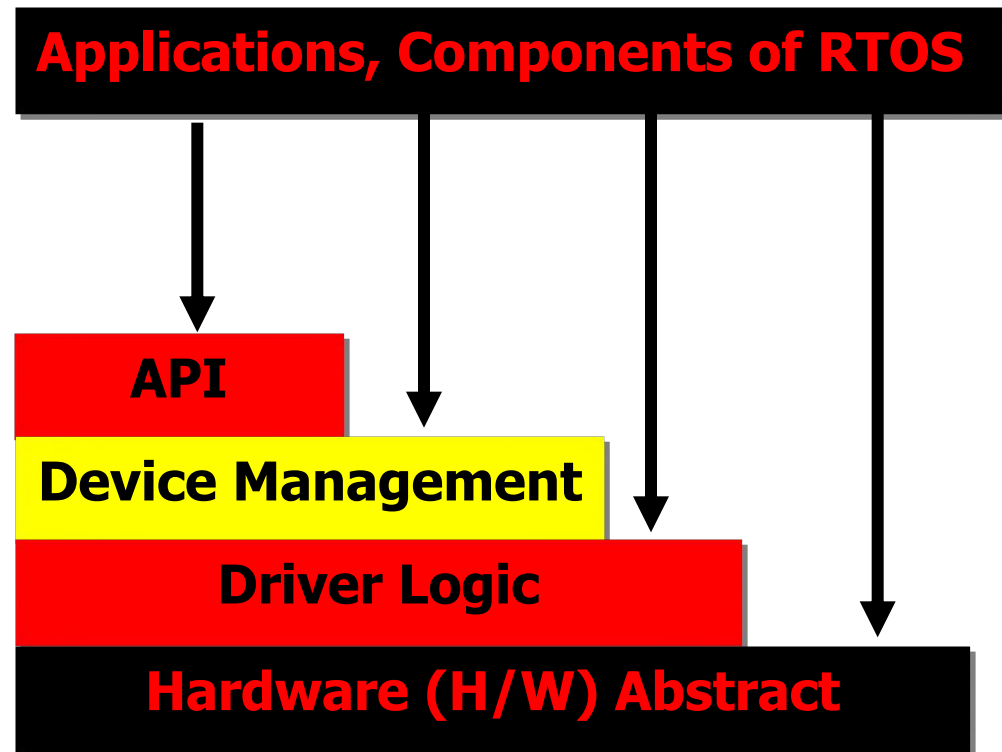
① 0X50 → high3Bit=4

② 0X88 → low3Bit=3

③ 35: 00100 011

0	1	0	1	0	0	0	0									
↑	↑	↑	↑	↑	↑	↑	↑	7	6	5	4	3	2	1	0	[0]
↑	↑	↑	↑	↑	↑	↑	↑	15	14	13	12	11	10	9	8	[1]
↑	↑	↑	↑	↑	↑	↑	↑	23	22	21	20	19	18	17	16	[2]
↑	↑	↑	↑	↑	↑	↑	↑	31	30	29	28	27	26	25	24	[3]
↑	↑	↑	↑	↑	↑	↑	↑	1	0	0	0	1	0	0	0	[4]
↑	↑	↑	↑	↑	↑	↑	↑	47	46	45	54	43	42	41	40	[5]
↑	↑	↑	↑	↑	↑	↑	↑	55	54	53	52	51	50	49	48	[6]
↑	↑	↑	↑	↑	↑	↑	↑	63	62	61	60	59	58	57	56	[7]

Get a pointer to the corresponding task by indexing the OSTCBPriorityTable[]





Initialize	Open	Close	Read	Write	Control
.....					
.....					
xxx_initialize	xxx_open	xxx_close	xxx_read	xxx_write	xxx_control

Driver Address Table



Address on the Driver Address Table

Name	Major	Minor
name1	major1	minor1
name2	major1	minor2
name3	major1	minor3

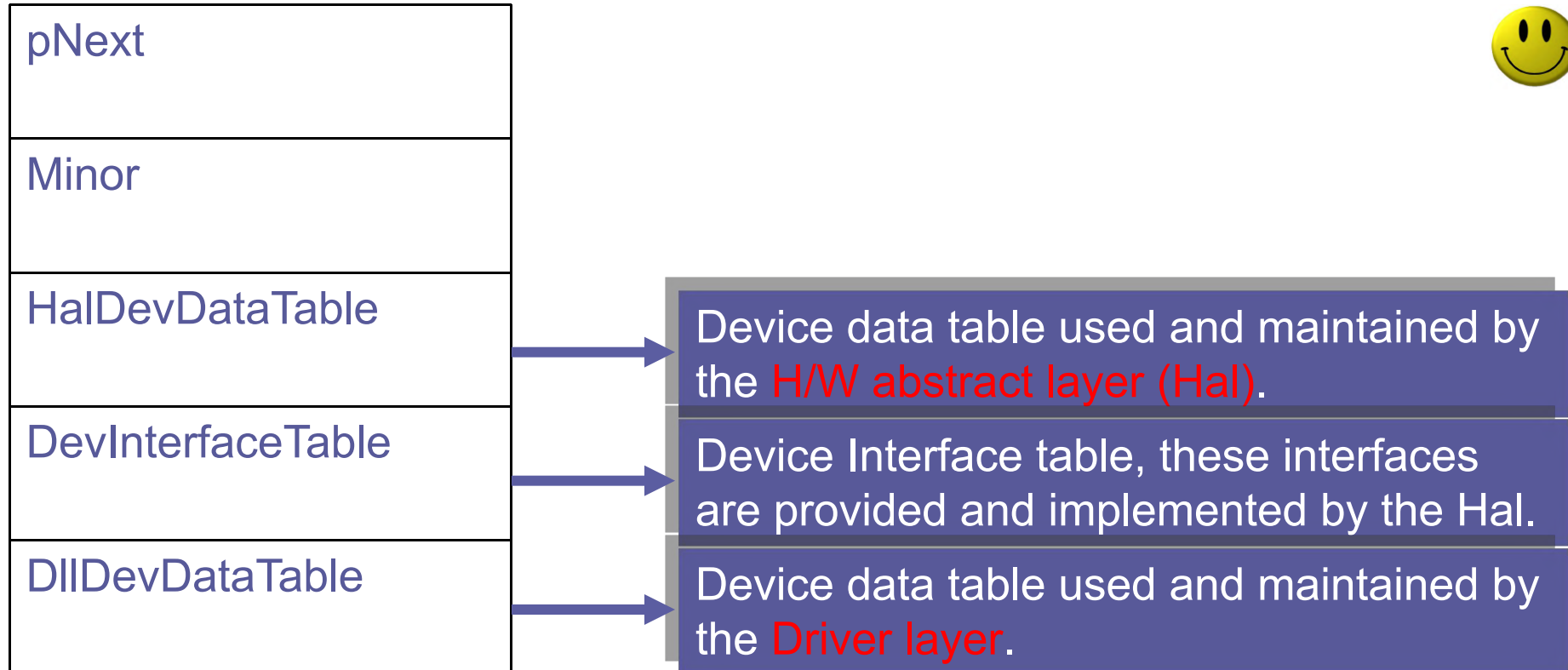
Device Number on the Driver logic

Device Name Table



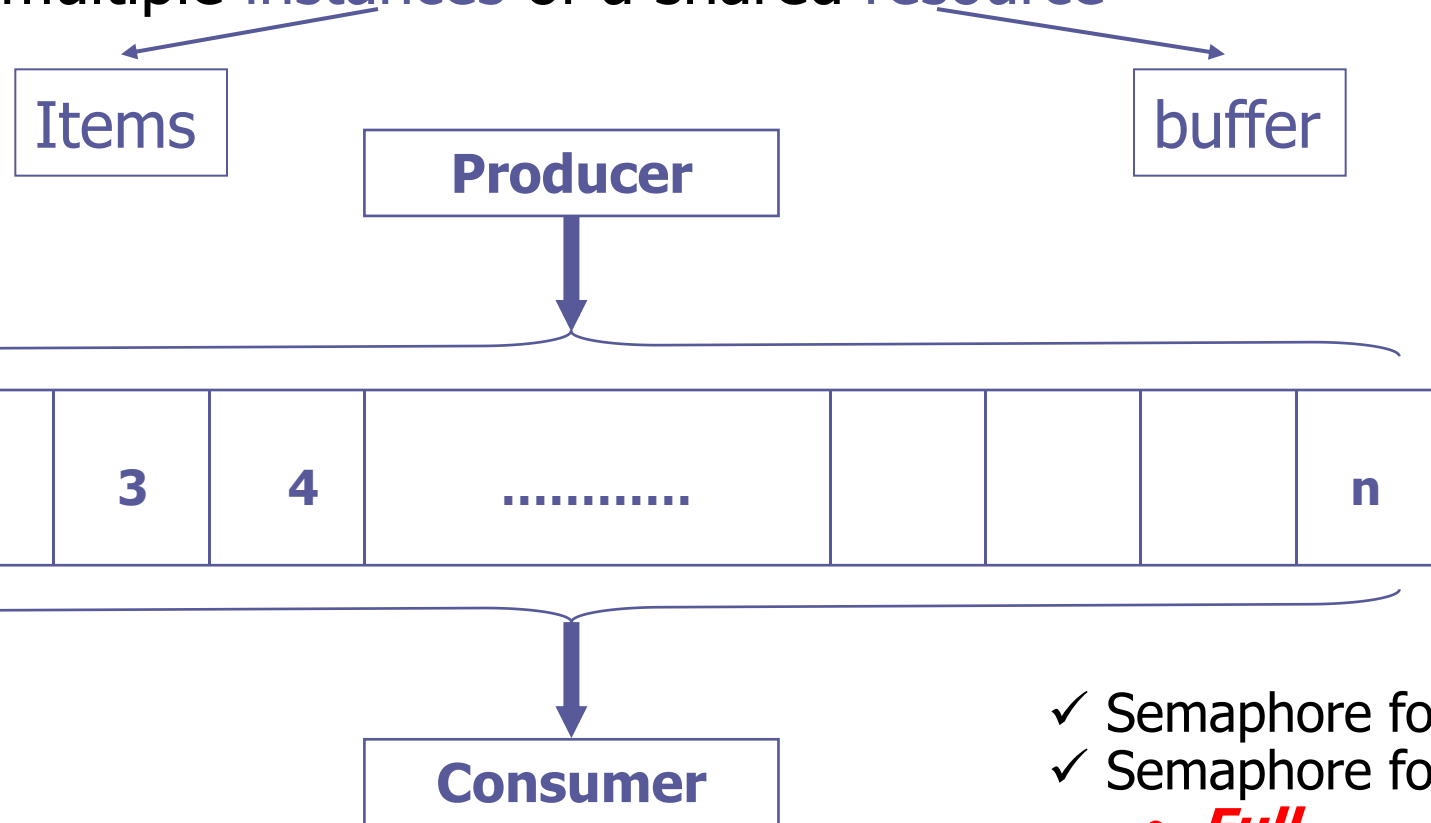
Fd	Driver	Size	Offset	Flags	Sem	Data
1						
2						
3						
4						

File Table (Record the information of the current device)



Device Information Table

Control multiple instances of a shared resource



- ✓ Semaphore for **Mutex**
- ✓ Semaphore for counting
 - **Full**
 - **Empty**

Producer

```

do
{
  ...
  Produce a new item
  ...
  Apply empty → Empty = 0 ?
  Apply mutex
  ...
  Add the new item to the queue
  ...
  Free mutex
  Free full → Empty = Empty-1
  Full = Full +1
} while (1);

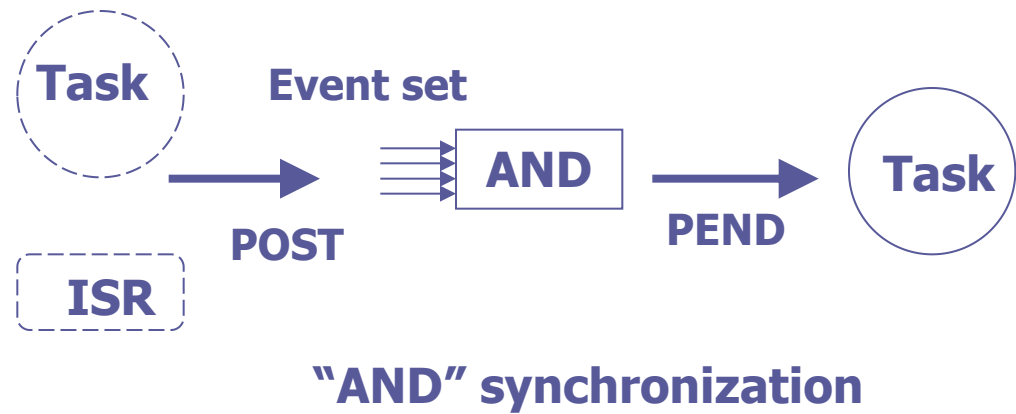
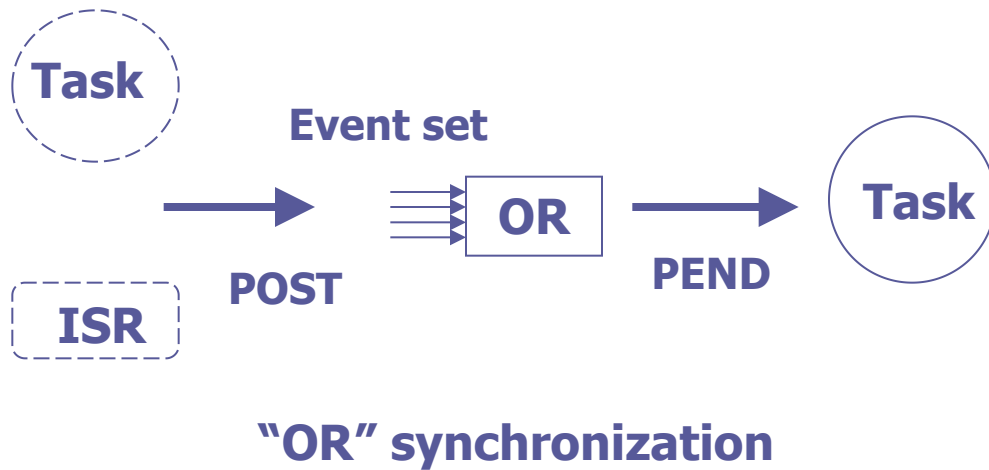
```

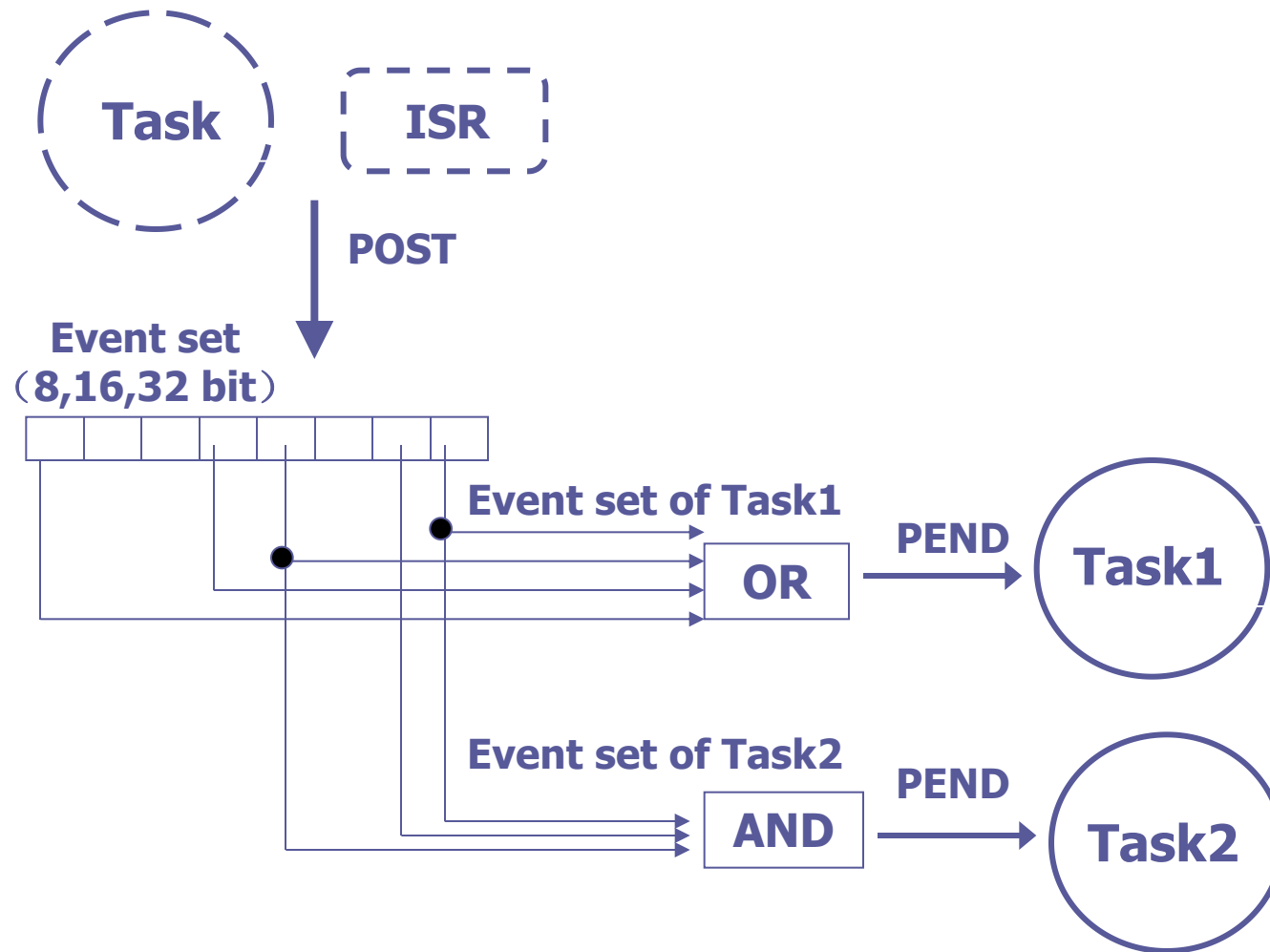
Consumer

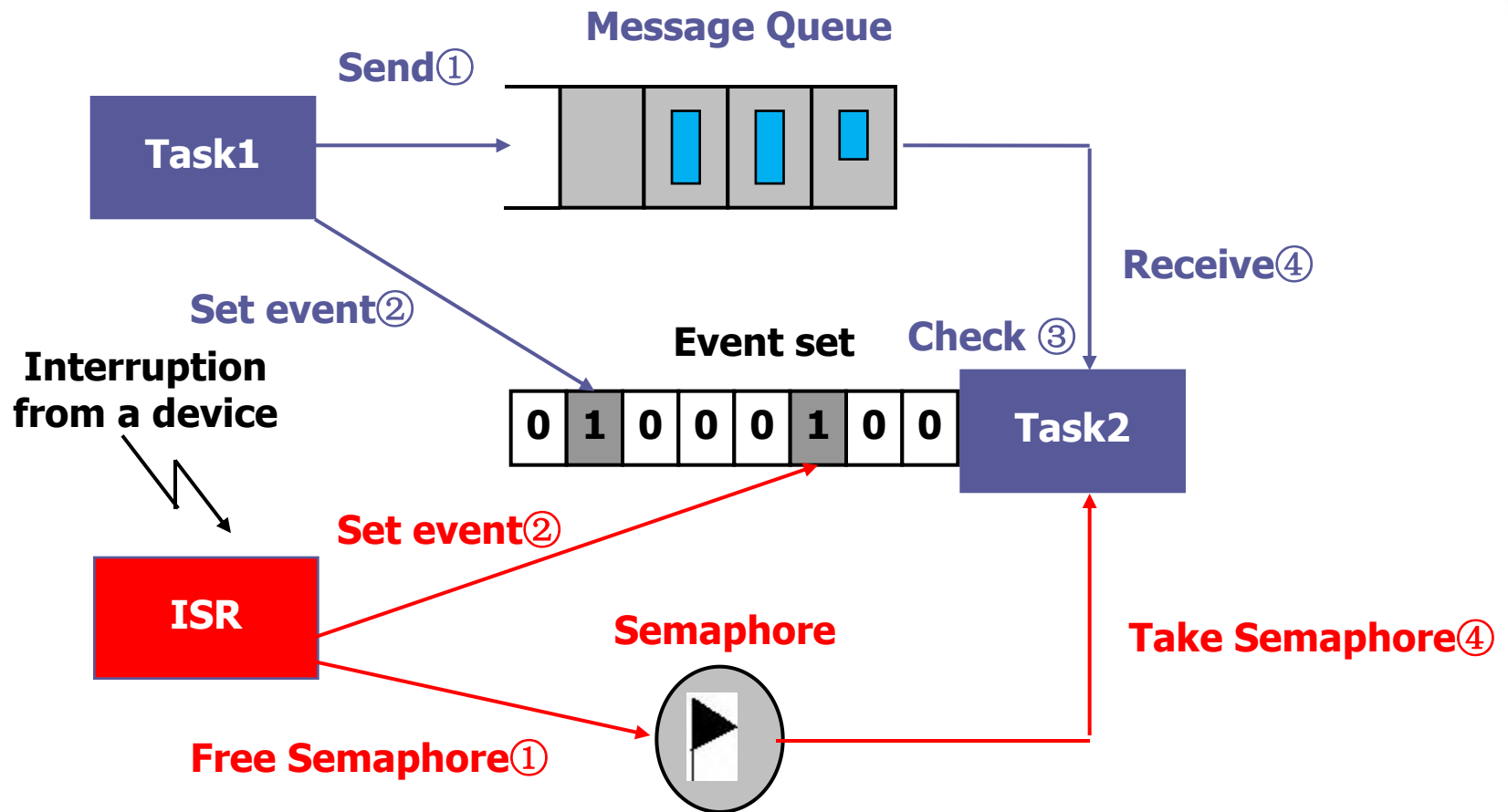
```

do
{
  Apply full → Full = 0 ?
  Apply mutex
  ...
  Move a item from the queue
  ...
  Free mutex
  Free empty → Full = Full -1
  Empty = Empty +1
  ...
  Consume the newly-moved item
  ...
} while (1);

```



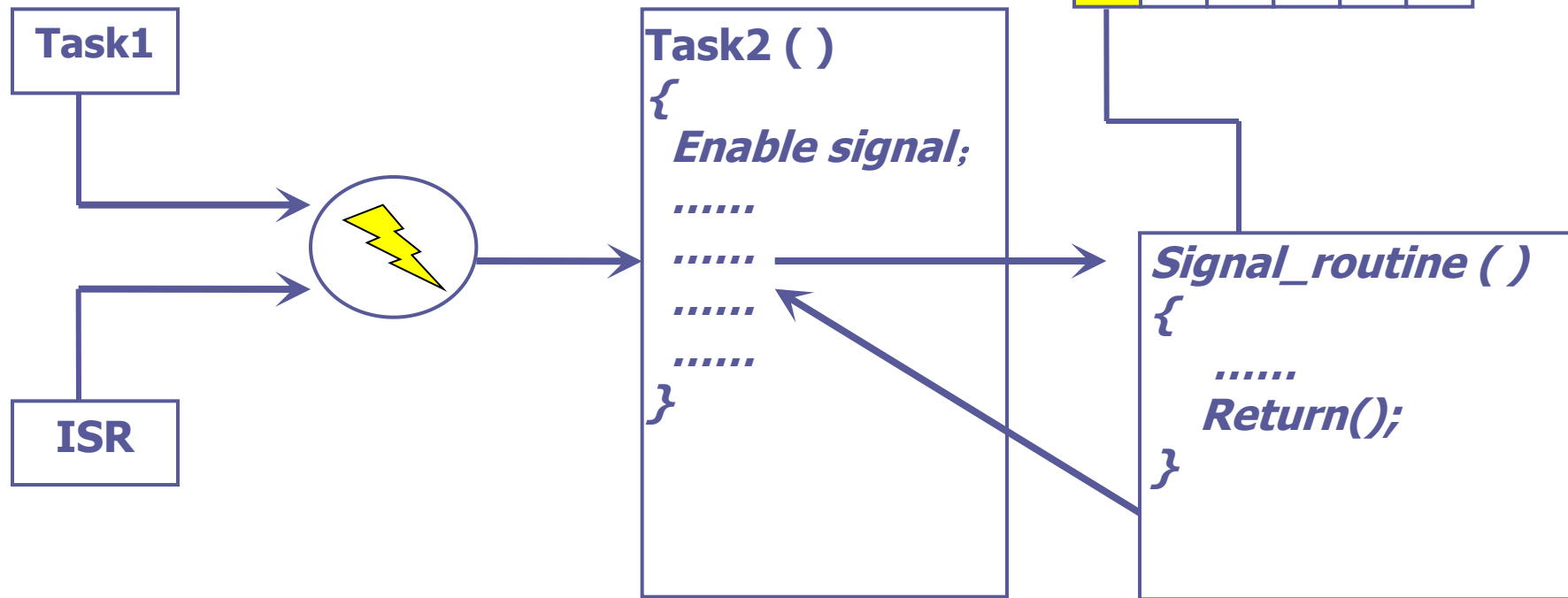




Flag of signals



1	0	0	0	0	0
---	---	---	---	---	---



Asynchronization, Synchronization, Interruption ?

```
void OSTimeDly (INT16U ticks)
```

```
{
```

```
    #if OS_CRITICAL_METHOD == 3
```

```
        OS_CPU_SR cpu_sr;
```

```
    #endif
```

```
    if (ticks > 0)
```

```
    { OS_ENTER_CRITICAL();
```

```
        if ((OSRdyTbl[OSTCBCur->OSTCBY] &= ~OSTCBCur->OSTCBBitX)  
        == 0)
```

```
            { OSRdyGrp &= ~OSTCBCur->OSTCBBitY;  }
```

```
            OSTCBCur->OSTCBDly = ticks;
```

```
            OS_EXIT_CRITICAL();
```

```
            OS_Sched();
```

```
        }
```

```
    }
```





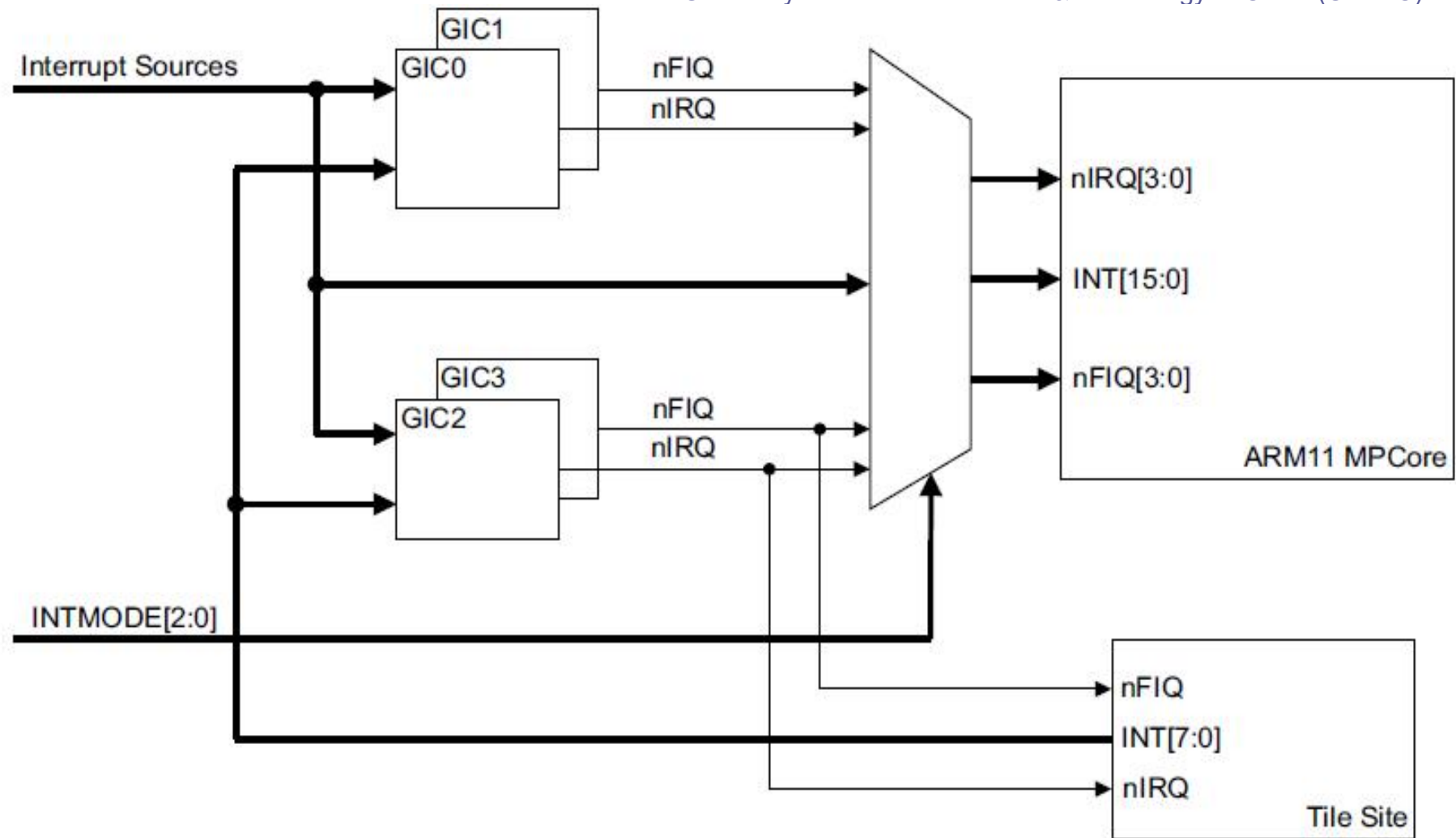
```

OS_EVENT *OSSemCreate (INT16U cnt)
{
    OS_EVENT *pevent;
    pevent = OSEventFreeList;                                //Get next free event control block(ECB)
    if (OSEventFreeList != (OS_EVENT *)0) {                //See if pool of free ECB pool was empty
        OSEventFreeList = (OS_EVENT *)OSEventFreeList->OSEventPtr;
    }
    if (pevent != (OS_EVENT *)0) {                            //Get an ECB and initialize it
        pevent->OSEventType = OS_EVENT_TYPE_SEM; //Event type is Semaphore
        pevent->OSEventCnt = cnt;                        //Set semaphore value
        pevent->OSEventPtr = (void *)0;                  // Unlink from ECB free list
        OS_EventWaitListInit(pevent);                    //Initialize to 'nobody waiting' on Sem
    }
    return (pevent);
}

```



University of Electronic Science & Technology of China (UESTC)



	⋮
0x1C	FIQ
0x18	IRQ
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

copy_vectors:

LDR *r0*,
=**HAL_VECTR_START**

CMP *r0*, #0

BEQ *real_code*

LDR *r1*, =**HAL_VECTR_END**

MOV *r2*, #0

1:

LDR *r3*, [*r0*], #4

STR *r3*, [*r2*], #4

CMP *r0*, *r1*

BLO *1b*

University of Electronic Science & Technology of China (UESTC)

	⋮
0x1C	FIQ
0x18	IRQ
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

```

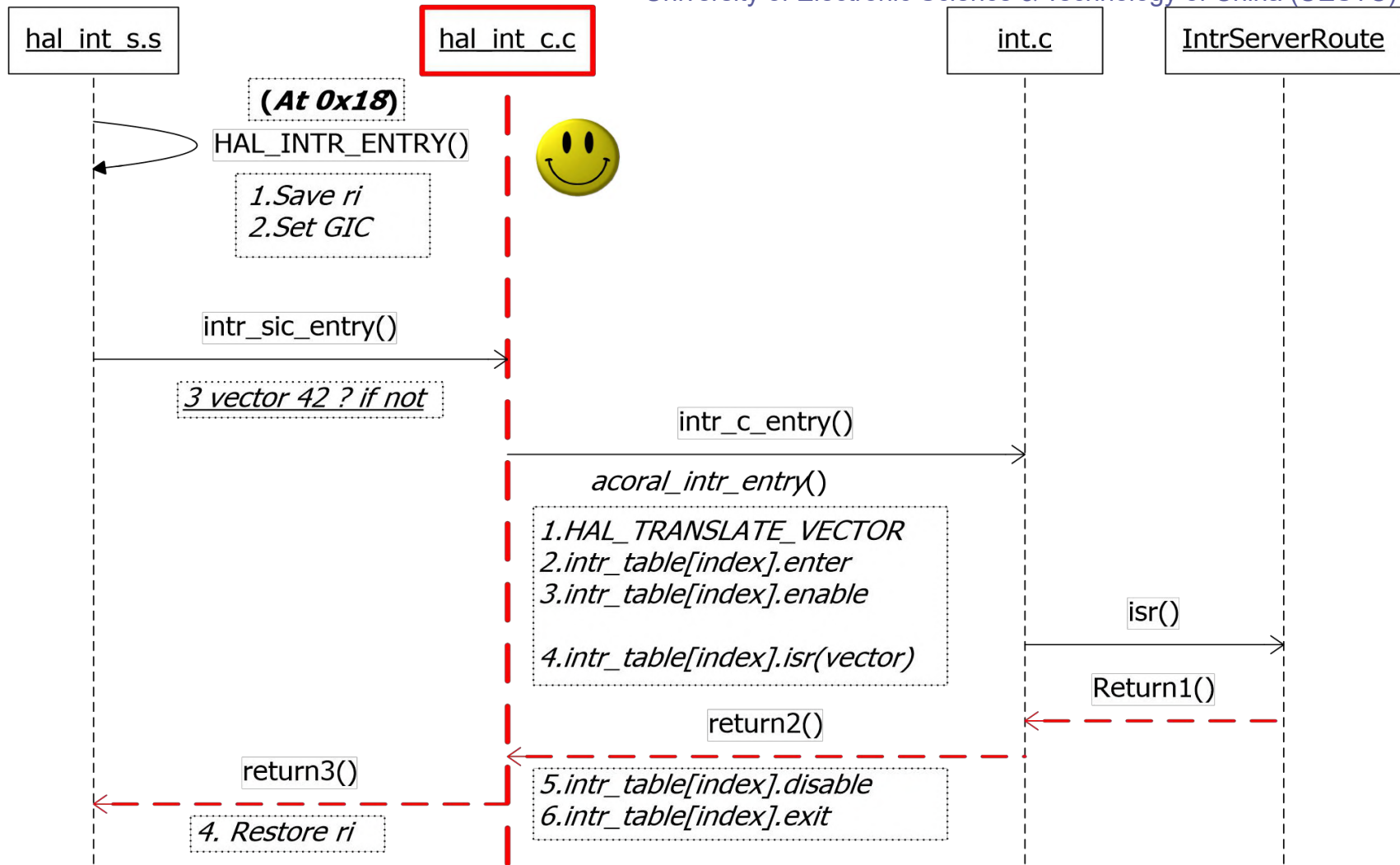
LDR    pc, VECTOR_TABLE+0
LDR    pc, VECTOR_TABLE+4
LDR    pc, VECTOR_TABLE+8
LDR    pc, VECTOR_TABLE+0xc
LDR    pc, VECTOR_TABLE+0x10
LDR    pc, VECTOR_TABLE+0x14
LDR    pc, VECTOR_TABLE+0x18
LDR    pc, VECTOR_TABLE+0x1c
  
```

VECTOR_TABLE:

```

.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long HAL_INTR_ENTRY
.long EXP_HANDLER
  
```





HAL_INTR_ENTRY:

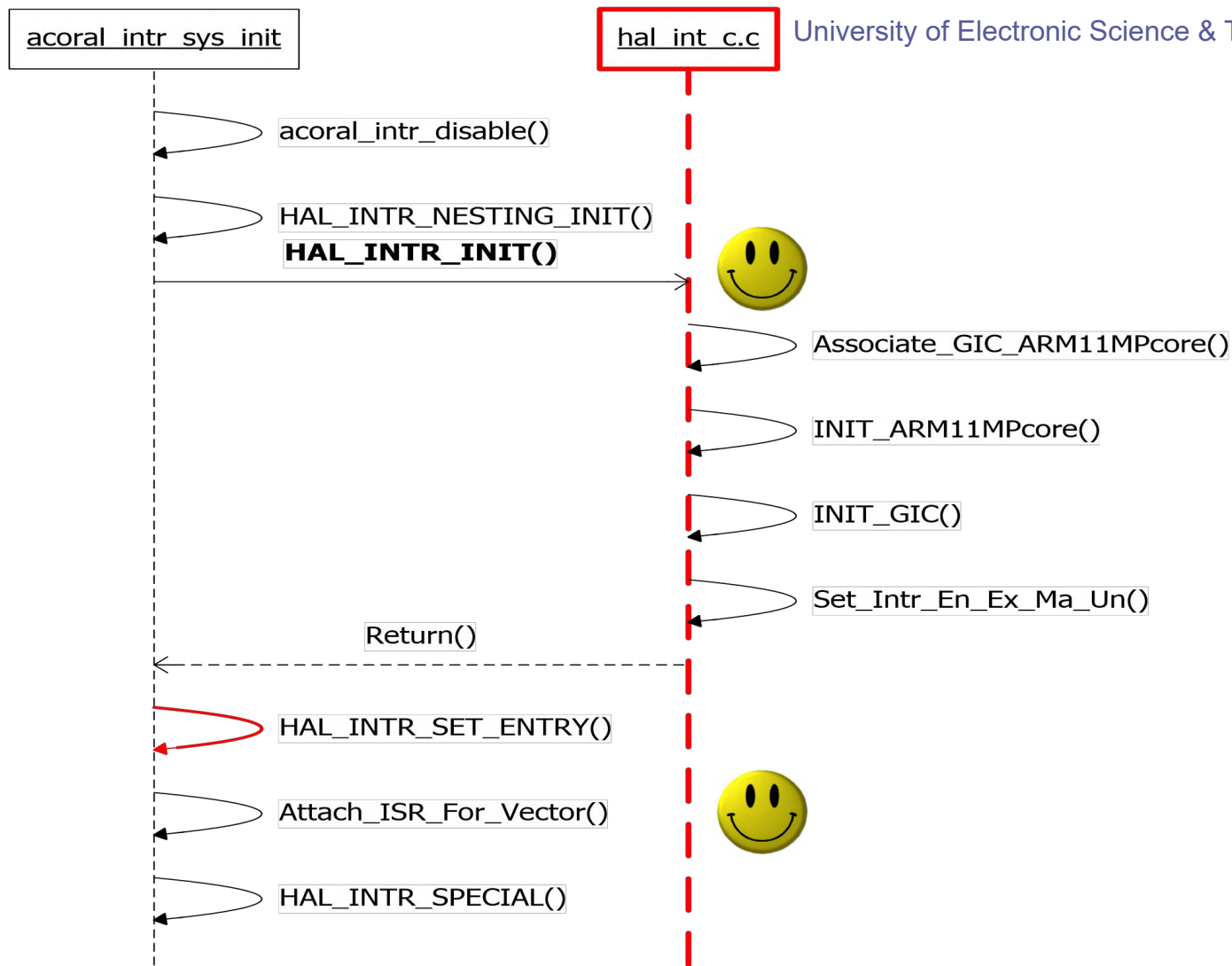
```
stmfd    sp!,{r0-r12,lr}  
mrs      r1,spsr  
stmfd    sp!,{r1}  
msr      cpsr_c,#SVCMODE/NOIRQ  
stmfd    sp!,{r0-r3,lr}
```

```
ldr      r0, =REALVIEW_TC11MP_GIC_CPU_BASE  
ldr      r0, [r0,#GIC_CPU_INTACK]  
bic      r0, r0,#0x1c00
```

```
cmp      r0, #42  
bleq     intr_sic_entry  
ldr      r2,=intr_c_entry  
mov      lr,pc  
ldr      pc,[r2]
```

```
ldmfd    sp!,{r0-r3,lr}  
bl       acoral_intr_exit  
msr      cpsr_c,#IRQMODE/NOINT  
ldmfd    sp!,{r0}  
msr      spsr_cxsf,r0  
ldmfd    sp!,{r0-r12,lr}  
subs     pc,lr,#4
```





```
void hal_intr_init(){  
    acoral_u32 i;  
    acoral_u32 pldctrl;  
  
    HAL_REG(REALVIEW_SYS_LOCK,0)=0x0000a05f;  
    pldctrl = HAL_REG(REALVIEW_SYS_BASE, REALVIEW_PB11MP_SYS_PLD_CTRL1);  
    pldctrl |= 2 << 22;  
    HAL_REG(REALVIEW_SYS_BASE, REALVIEW_PB11MP_SYS_PLD_CTRL1)=pldctrl;  
    HAL_REG(REALVIEW_SYS_LOCK,0)=0;  
    gic_dist_init(0,REALVIEW_TC11MP_GIC_DIST_BASE,29);  
    gic_cpu_init(0,REALVIEW_TC11MP_GIC_CPU_BASE);  
    gic_dist_init(1,REALVIEW_PB11MP_GIC_DIST_BASE,IRQ_PB11MP_GIC_START);  
    gic_cpu_init(1,REALVIEW_PB11MP_GIC_CPU_BASE);  
    for(i=HAL_INTR_MIN;i<=HAL_INTR_MAX;i++){  
        acoral_set_intr_enter(i,NULL);  
        acoral_set_intr_exit(i,hal_intr_ack);  
        acoral_set_intr_mask(i,hal_intr_mask);  
        acoral_set_intr_unmask(i,hal_intr_unmask);  
    }  
}
```



```
for(i=HAL_INTR_MIN;i<=HAL_INTR_MAX;i++){
    HAL_TRANSLATE_VECTOR(i,index);
    intr_table[index].isr=acoral_default_isr;
    acoral_intr_mask(i);
}
```

@ Attach_ISR_For_Vector

When and how to attach ISR to the corresponding interrupt ?

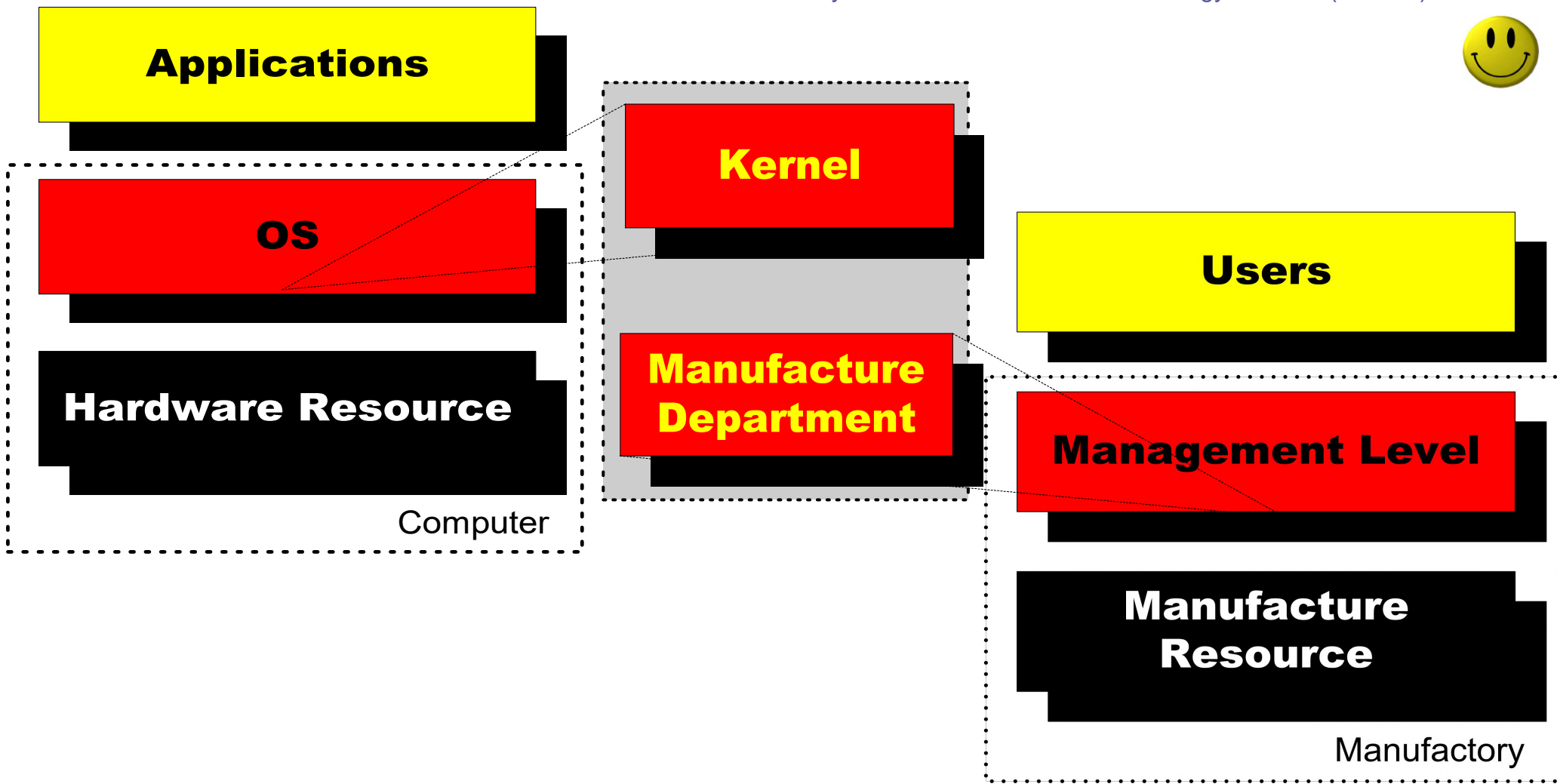
```
void acoral_ticks_init(){
    ticks=0;
    acoral_intr_attach(HAL_TICKS_INTR,acoral_ticks_entry);
    H A L _ T I C K S _ I N I T ( ) ;
    acoral_intr_unmask(HAL_TICKS_INTR);
    return;
}
```





```
typedef struct {  
    acoral_u32 index;  
    acoral_u8 state;  
    void (*isr)(acoral_vector);  
    void (*enter)(acoral_vector);  
    void (*exit)(acoral_vector);  
    void (*mask)(acoral_vector);  
    void (*unmask)(acoral_vector);  
}acoral_intr_ctr_t;
```

```
void hal_intr_unmask(acoral_vector irq){  
    acoral_u32 mask;  
    mask = 1<<(irq%32);  
    HAL_REG(hal_gic_dist_base(irq),GIC_DIST_ENABLE_SET+gic_irq(irq)/32*4)=mask;  
}
```




```
void YourTask (void *pdata)
{
    for (;;)
    {
        /* USER CODE */

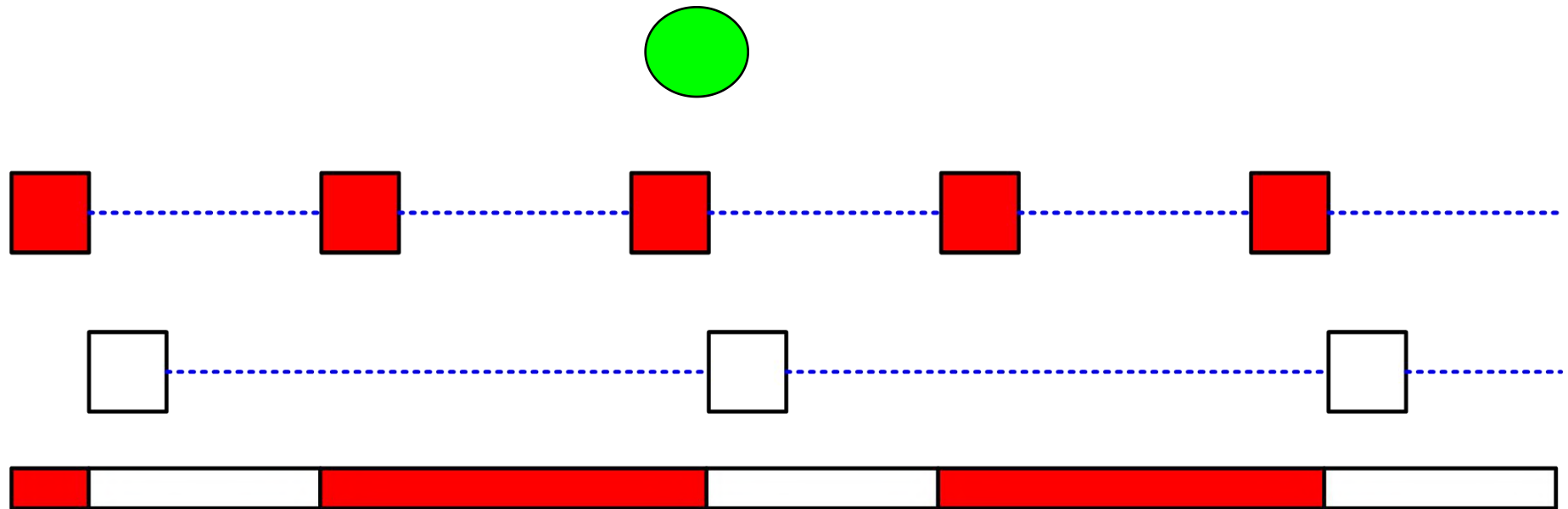
        .....
        OSMboxPend();
        OSQPend();
        OSSemPend();

        .....
        OSTaskSuspend(OS_PRIO_SELF);
        OSTimeDly();
        OSTimeDlyHMSM();
        /* USER CODE */
    }
}
```

Why infinite loop ?
Task & function ?

```
void Task1(void *pdata)  
{  
    WDTCTL = WDT_MDLY_32; /*设置时钟节拍间隔为32ms*/  
    IE1 |= BIT0; /* 开看门狗定时器中断*/  
    P2SEL &= BIT1; //引脚功能选择器  
    P2DIR |= BIT1; //方向寄存器, 1为输出模式  
  
    while(1)  
    {  
        P2OUT |= BIT1; //LED亮  
        OSTimeDly(10); //延时10个时钟节拍  
    }  
}
```

```
void Task2(void *pdata)  
{  
    WDTCTL = WDT_MDLY_32; /*设置时钟节拍间隔为32ms*/  
    IE1 |= BIT0; /*开看门狗定时器中断*/  
    P2SEL &= BIT1; //引脚功能选择器  
    P2DIR |= BIT1; //方向寄存器, 1为输出模式  
  
    while(1)  
    {  
        P2OUT &= ~BIT1; //LED灭  
        OSTimeDly(20); //延时20个时钟节拍  
    }  
}
```



Where and when to create the two tasks?

How to start the two tasks ?

What's the difference between task and process?

```
int i=1;  
  
test(){  
    sleep(10S);    /*waiting for the execution of "i++"*/  
    printf("%d",i);  
}  
  
int main(){  
    create_process(test,.....);  
    i++;  
}
```

```
int i=1;  
  
test(){  
    sleep(10S);    /* waiting for the execution of "i++"*/  
    printf("%d",i);  
}  
  
int main(){  
    create_task(test,.....);  
    i++;  
}
```





```
typedef struct{
    acoral_res_t    res;
    acoral_u8       state;
    acoral_u8       prio;
    acoral_8        cpu;
    acoral_u32      cpu_mask;
    acoral_u8       policy;
    acoral_list_t   ready;
    acoral_list_t   timeout;
    acoral_list_t   waiting;
    acoral_list_t   global_list;
    acoral_evt_t    *evt;
    acoral_u32      *stack;
```



```
    acoral_u32      *stack_bottom;
    acoral_u32      stack_size;
    acoral_32       delay;
    acoral_u32      slice;
    acoral_char     *name;
    acoral_id       console_id;
    void*           private_data;
    void*           data;
}acoral_thread_t;
```

```
typedef union {
```

```
    acoral_id id;
```

```
|* unsigned int*|
```

```
    acoral_u16 next_id;
```

```
|* unsigned short*|
```

```
}acoral_res_t;
```

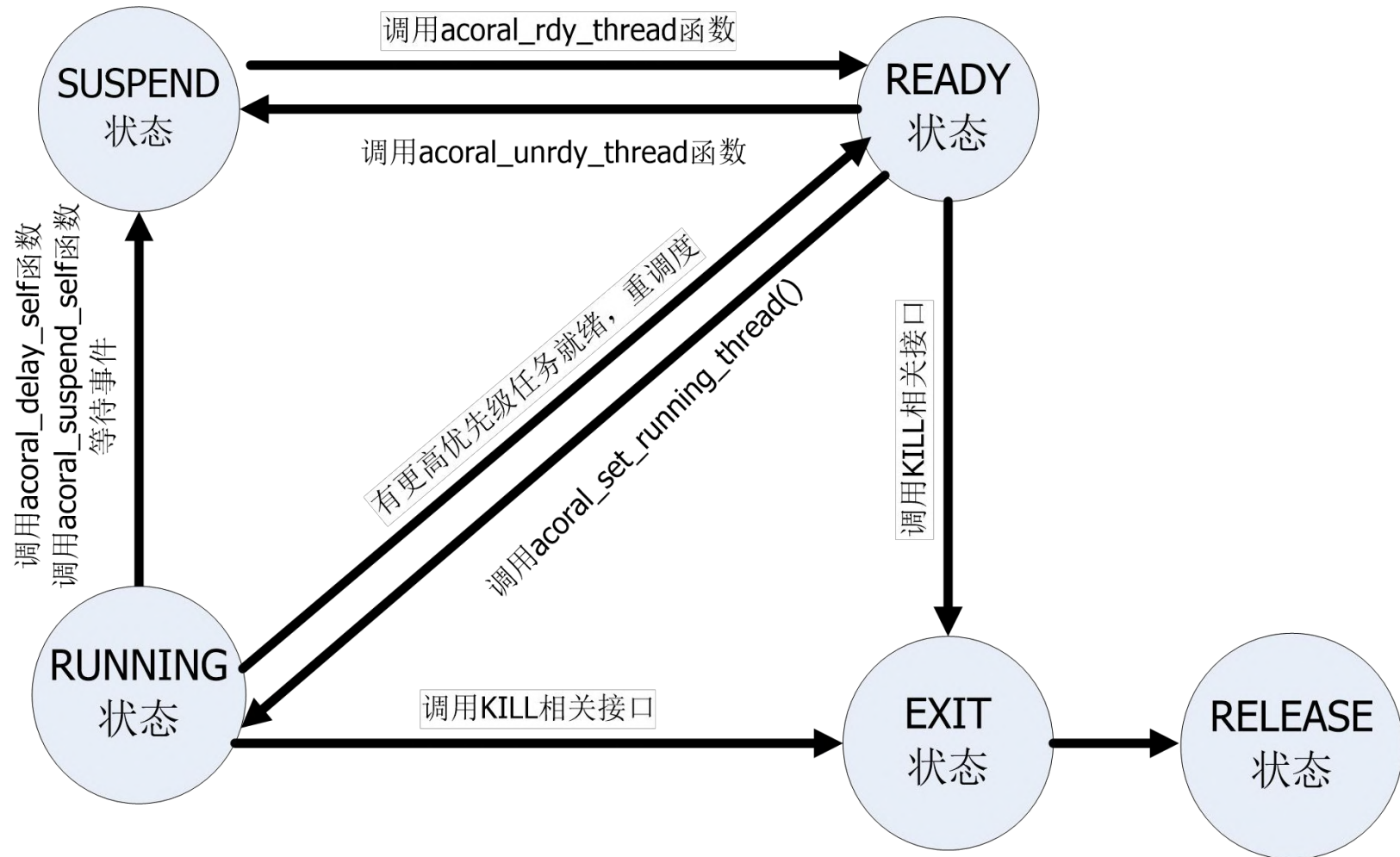


Type

空闲内存块ID（10 bit位）

```
#define ACORAL_RES_THREAD 1  
#define ACORAL_RES_EVENT 2  
#define ACORAL_RES_TIMER 3  
#define ACORAL_RES_DRIVER 4  
#define ACORAL_RES_GUI 5  
#define ACORAL_RES_USER 6
```





```
#ifdef CFG_THRD_POSIX
    #define ACORAL_MAX_PRIO_NUM ((CFG_MAX_THREAD+CFG_POSIX_STAIR_NUM+1)&0xff)
#else
    #define ACORAL_MAX_PRIO_NUM ((CFG_MAX_THREAD+1)&0xff)
#endif
#define ACORAL_MINI_PRIO ACORAL_MAX_PRIO_NUM-1
#define ACORAL_INIT_PRIO 0
#define ACORAL_MAX_PRIO 1

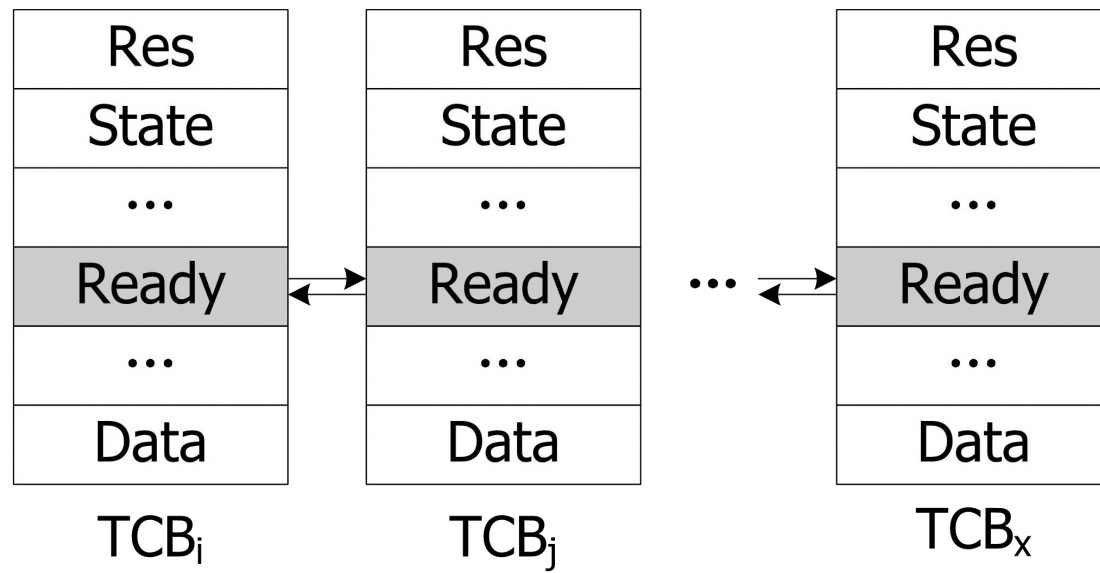
#define CFG_MAX_THREAD (100)
```



```

struct acoral_list {
    struct acoral_list *next, *prev;
#ifdef CFG_CMP
    acoral_spinlock_t lock
#endif
};

```



```

acoral_id create_comm_thread(void (*route)(void *args),acoral_u32
stack_size,void *args,acoral_char *name,acoral_u8 prio,acoral_8 cpu){
    acoral_comm_policy_data_t policy_ctrl; 
    acoral_thread_t *thread;
    thread=acoral_alloc_thread(); 
    if(NULL==thread){
        acoral_printerr("Alloc thread:%s fail\n",name);
        acoral_printk("No Mem Space or Beyond the max thread\n");
        return -1;
    }
}

```

```

typedef struct{
    acoral_8 cpu;
    acoral_u8 prio;
}acoral_comm_policy_data_t;

```

```

typedef struct{
    acoral_8 cpu;
    acoral_u8 prio;
    acoral_u8 prio_type;
    acoral_u32 slice;
}acoral_slice_policy_data_t;

```

```
thread->name=name;  
stack_size=stack_size&(~3);  
thread->stack_size=stack_size;  
thread->stack_bottom=NULL;  
policy_ctrl.cpu=cpu;  
policy_ctrl.prio=prio;  
thread->policy=ACORAL_SCHED_POLICY_COMM;  
thread->cpu_mask=-1;  
return comm_policy_thread_init(thread,route,args,&policy_ctrl);  
}
```



```
typedef struct{  
    acoral_8 cpu;  
    acoral_u8 prio;  
}acoral_comm_policy_data_t;
```



```
/*=====
func: alloc thread struct data in acoral
*   TCB
*=====*/
acoral_thread_t *acoral_alloc_thread(){
return (acoral_thread_t *)acoral_get_res(&acoral_thread_pool_ctrl);}
```



```
acoral_res_t *acoral_get_res(acoral_pool_ctrl_t *pool_ctrl){  
    acoral_sr cpu_sr;  
    acoral_list_t *first;  
    acoral_res_t *res;  
    acoral_pool_t *pool;  
    HAL_ENTER_CRITICAL();  
    acoral_spin_lock(&pool_ctrl->lock);  
    first=pool_ctrl->free_pools->next;  
    if(acoral_list_empty(first)){  
        if(acoral_create_pool(pool_ctrl)){  
            acoral_spin_unlock(&pool_ctrl->lock);  
            HAL_EXIT_CRITICAL();  
            return NULL;  
        }  
        else{  
            first=pool_ctrl->free_pools->next;  
        }  
    }  
}
```



```
pool=list_entry(first,acoral_pool_t,free_list);  
res=(acoral_res_t *)pool->res_free;  
pool->res_free=(void *)((acoral_u8 *)pool->base_adr+res-  
                                >next_id*pool->size);  
res->id=(res->id>>(ACORAL_RES_INDEX_INIT_BIT-  
                    ACORAL_RES_INDEX_BIT))&ACORAL_RES_INDEX_MASK|pool->id;  
pool->free_num--;  
  
if(!pool->free_num){  
    acoral_list_del(&pool->free_list);  
}  
acoral_spin_unlock(&pool_ctrl->lock);  
HAL_EXIT_CRITICAL();  
return res;  
}
```



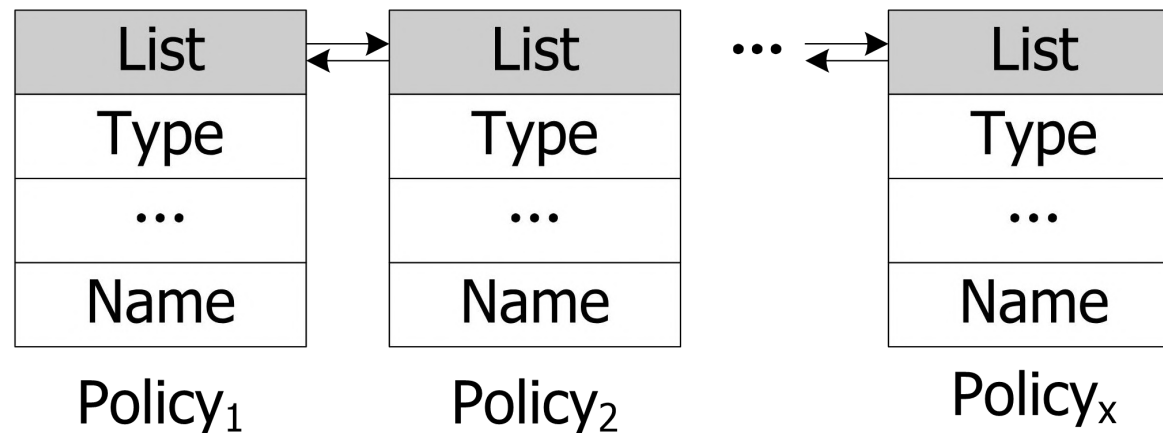

```
acoral_id comm_policy_thread_init(acoral_thread_t *thread,void (*route)(void  
*args),void *args,void *data){  
    acoral_sr cpu_sr;  
    acoral_u32 prio;  
    acoral_comm_policy_data_t *policy_data;  
    policy_data=(acoral_comm_policy_data_t *)data;  
    thread->cpu=policy_data->cpu;  
    prio=policy_data->prio;
```

```
typedef struct{  
    acoral_8 cpu;  
    acoral_u8 prio;  
}acoral_comm_policy_data_t;
```

```
thread->prio=prio;  
if(acoral_thread_init(thread,route,acoral_thread_exit,args)!=0){  
    acoral_printerr("No thread stack:%s\n",thread->name);  
    HAL_ENTER_CRITICAL();  
    acoral_release_res((acoral_res_t *)thread);  
    HAL_EXIT_CRITICAL();  
    return -1;  
}  
acoral_resume_thread(thread);  
return thread->res.id;  
}
```



```
typedef struct{
    acoral_list_t list;
    acoral_u8 type;
    acoral_id (*policy_thread_init)(acoral_thread_t*, void (*route)(void
*args), void *, void *);
    void (*delay_deal)();
    acoral_8 *name;
}acoral_sched_policy_t;
```



```

acoral_err acoral_thread_init(acoral_thread_t *thread,void (*route)(void *args),void
(*exit)(void),void *args){
    acoral_sr cpu_sr;
    acoral_u32 stack_size=thread->stack_size;
    if(thread->stack_bottom==NULL){
        if(stack_size<ACORAL_MIN_STACK_SIZE)
            stack_size=ACORAL_MIN_STACK_SIZE;
        thread->stack_bottom=(acoral_u32 *)acoral_malloc(stack_size);
        if(thread->stack_bottom==NULL)
            return ACORAL_ERR_THREAD_NO_STACK;
        thread->stack_size=stack_size;
    }
    thread->stack=(acoral_u32 *)((acoral_8 *)thread->stack_bottom+stack_size-4);
    HAL_STACK_INIT(&thread->stack,route,exit,args);
    thread->data=NULL;
    thread->state=ACORAL_THREAD_STATE_SUSPEND;

```



```
thread->console_id=acoral_cur_thread->console_id;
acoral_init_list(&thread->waiting);
acoral_init_list(&thread->ready);
acoral_init_list(&thread->timeout);
acoral_init_list(&thread->global_list);
acoral_spin_init(&thread->timeout.lock);
acoral_spin_init(&thread->waiting.lock);
acoral_spin_init(&thread->ready.lock);
acoral_spin_init(&thread->move_lock);
HAL_ENTER_CRITICAL();
acoral_spin_lock(&acoral_threads_queue.head.lock);
acoral_list_add2_tail(&thread->global_list,&acoral_threads_queue.head);
acoral_spin_unlock(&acoral_threads_queue.head.lock);
HAL_EXIT_CRITICAL();
#ifdef CFG_TEST
    acoral_print("%s thread initial well\n",thread->name);
#endif
return 0;
}
```



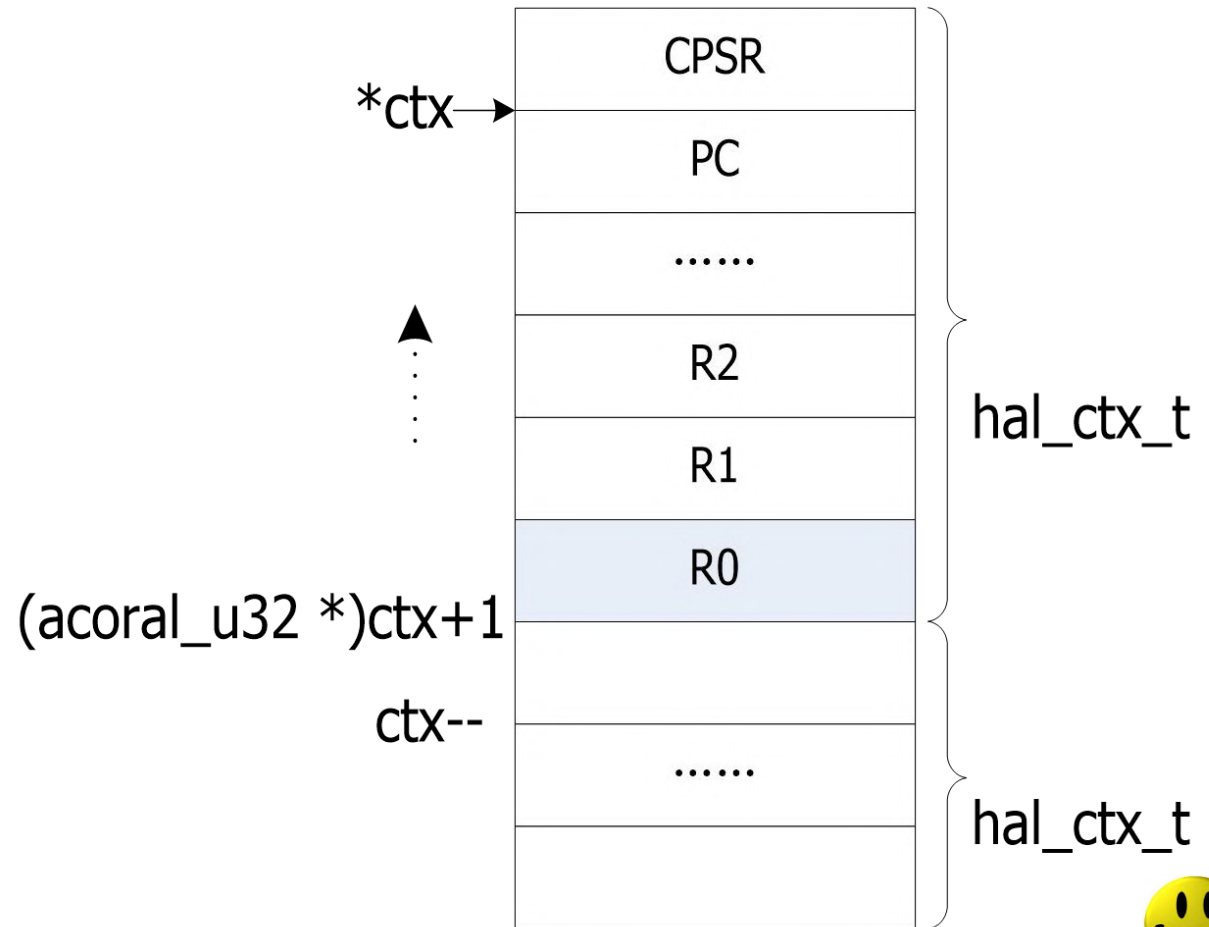
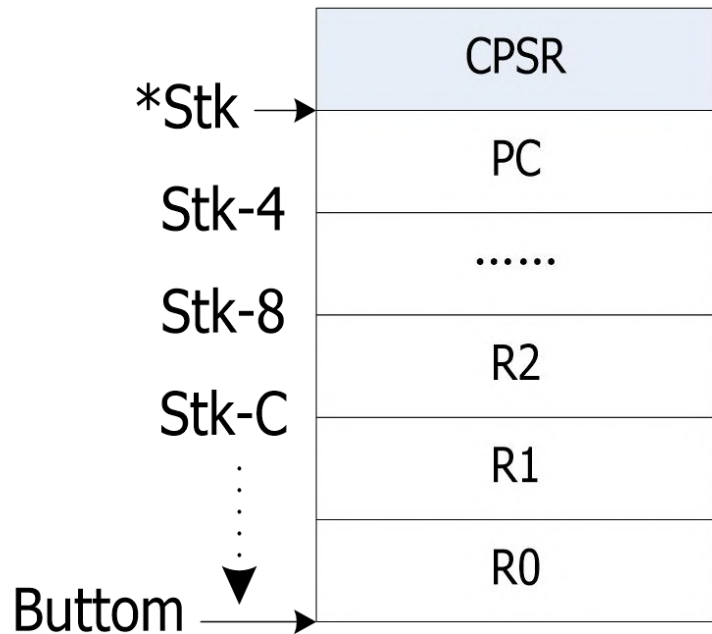
```
void hal_stack_init(acoral_u32 **stk, void  
(*route)(), void (*exit)(), void *args){  
    hal_ctx_t *ctx=*stk;   
    ctx--;  
    ctx=(acoral_u32 *)ctx+1;  
    ctx->r0=(acoral_u32)args;  
    ctx->r1=1;  
    ctx->r2=2;  
    ctx->r3=3;  
    ctx->r4=4;  
    ctx->r5=5;  
    ctx->r6=6;  
    ctx->r7=7;  
    ctx->r8=8;  
    ctx->r9=9;  
    ctx->r10=10;  
    ctx->r11=11;
```

```
    ctx->r12=12;  
    ctx->lr=(acoral_u32)exit;  
    ctx->pc=(acoral_u32)route;  
    ctx->cpsr=0x0000001f;  
    *stk=ctx;
```

```
}
```

```
#define HAL_STACK_INIT(stack,route,exit,args) hal_stack_init(stack,route,exit,args)
typedef struct {
    acoral_u32 r0;
    acoral_u32 r1;
    acoral_u32 r2;
    acoral_u32 r3;
    acoral_u32 r4;
    acoral_u32 r5;
    acoral_u32 r6;
    acoral_u32 r7;
    acoral_u32 r8;
    acoral_u32 r9;
    acoral_u32 r10;
    acoral_u32 r11;
    acoral_u32 r12;
    acoral_u32 lr;
    acoral_u32 pc;
    acoral_u32 cpsr;
} hal_ctx_t;
```

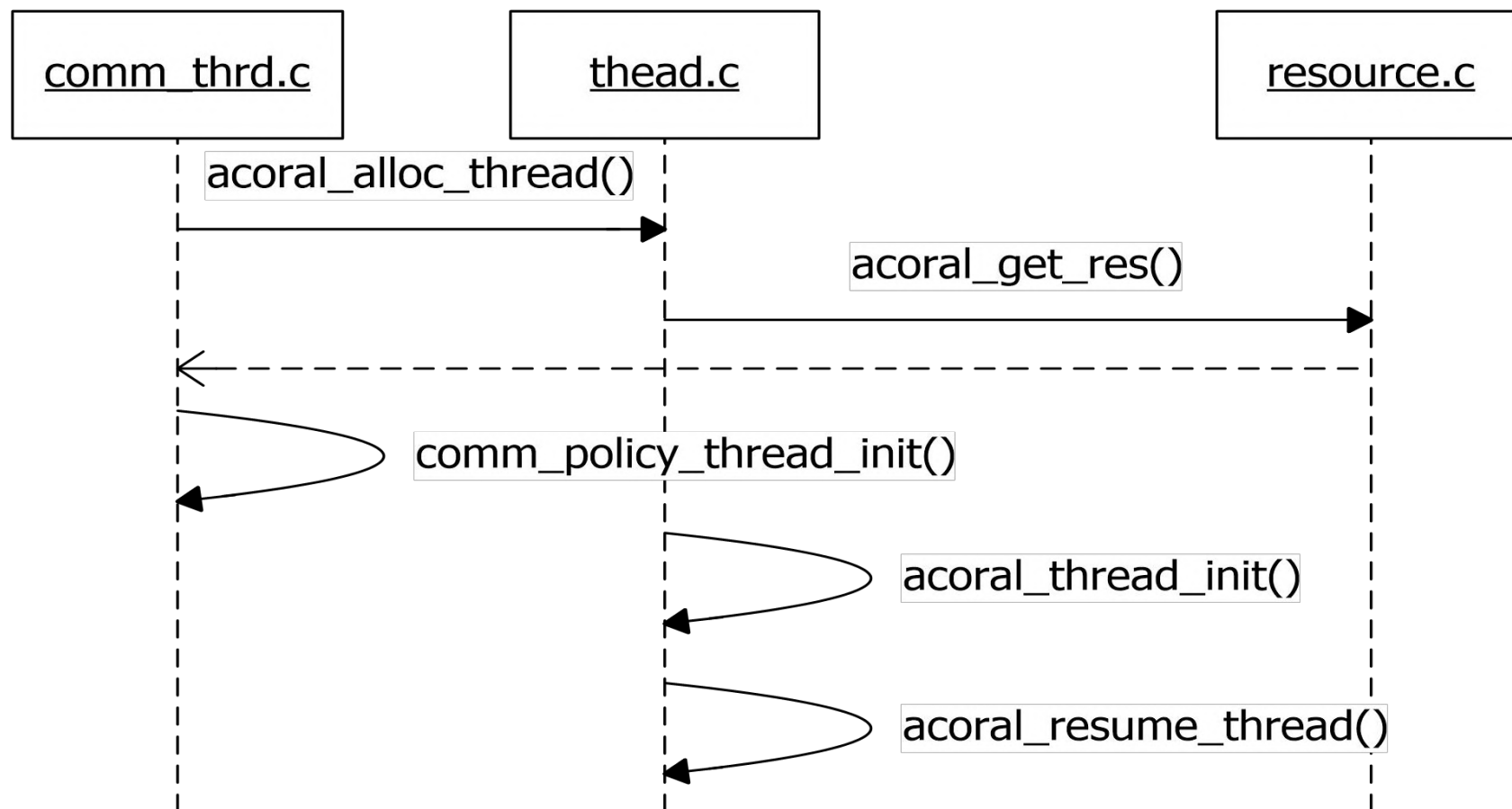


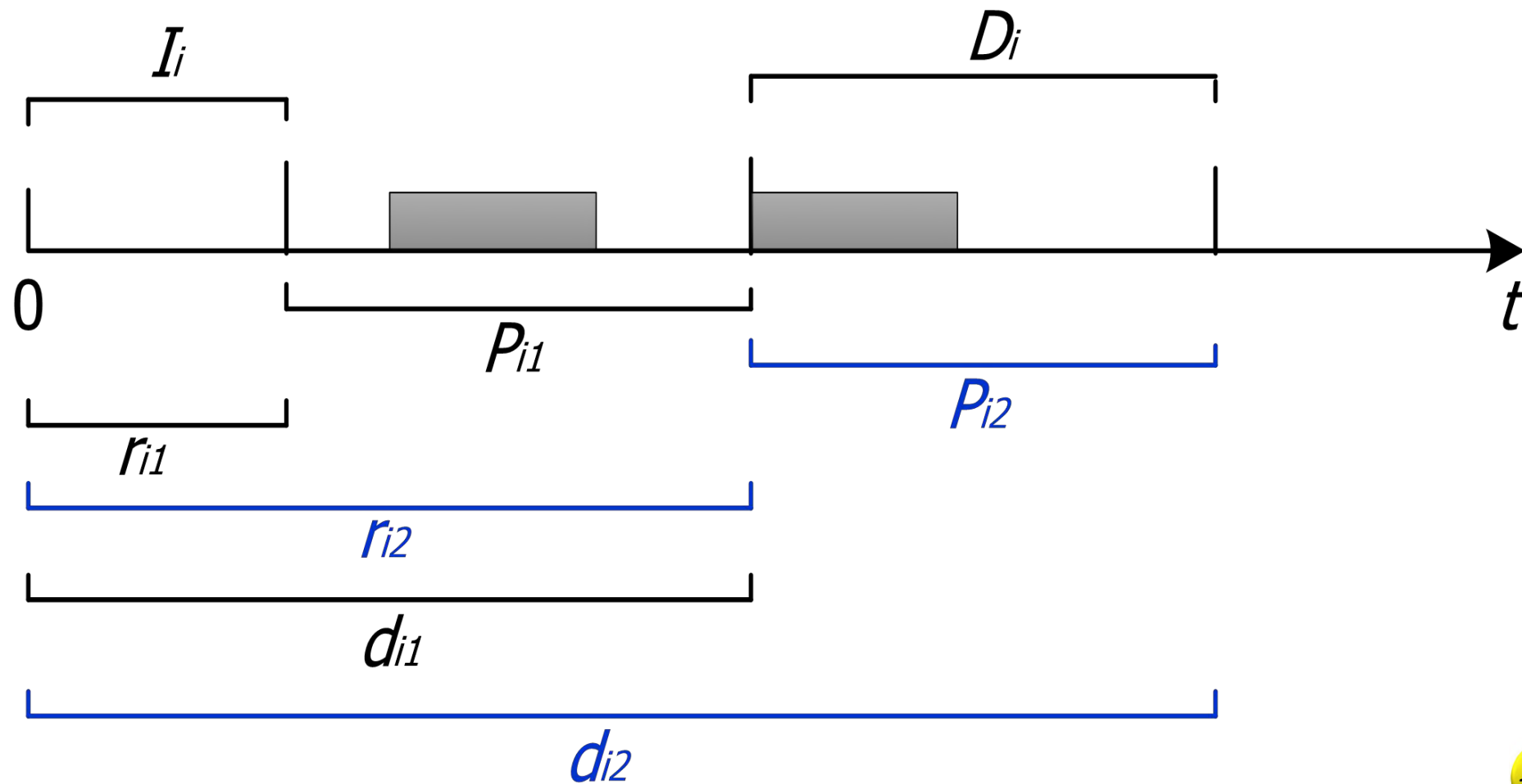



```
void acoral_resume_thread(acoral_thread_t *thread){  
    acoral_sr cpu_sr;  
    acoral_8 cpu;  
    if(!(thread->state&ACORAL_THREAD_STATE_SUSPEND))  
        return;  
    HAL_ENTER_CRITICAL();  
        acoral_rdyqueue_add(thread);  
    HAL_EXIT_CRITICAL();  
    acoral_sched();  
}
```

```
void acoral_rdyqueue_add(acoral_thread_t *thread){  
    acoral_rdy_queue_t *rdy_queue;  
    acoral_u8 cpu;  
    acoral_prio_queue_add(&rdy_queue->array,thread->prio,&thread->ready);  
    thread->state=ACORAL_THREAD_STATE_READY;  
    thread->res.id=thread->res.id/cpu<<ACORAL_RES_CPU_BIT;  
    acoral_set_need_sched(cpu,true);  
}
```







```
void acoral_ticks_init() {  
    ticks=0;  
    acoral_intr_attach(HAL_TICKS_INTR,acoral_ticks_entry);  
    HAL_TICKS_INIT();  
    acoral_intr_unmask(HAL_TICKS_INTR);  
    return;  
}
```

```
void acoral_ticks_entry(acoral_vector vector){  
    #ifdef CFG_HOOK_TICKS  
        acoral_ticks_hook();  
    #endif  
    ticks++;  
    acoral_printdbg("In ticks isr\n");  
    if(acoral_start_sched==true){  
        thread_delay_deal();  
        acoral_policy_delay_deal();  
        timeout_delay_deal();  
    }  
}
```



*HAL_CONTEXT_SWITCH**stmfd sp!,{lr}**stmfd sp!,{r0-r12,lr}**mrs r4,CPSR**stmfd sp!,{r4}**str sp,[r0]**ldr sp,[r1]**ldmfd sp!, {r0}**msr cpsr_cxsf,r0**ldmfd sp!, {r0-r12,lr,pc}*

there were notable and timely successes in the application of this theory to national high technology projects including a Global Positioning Satellite software upgrade (Doyle and Elzey, 1994) and the Space Station. In 1991 the U.S. Department of Defense (DoD) Software Technology Strategy report called the development of generalized Rate Monotonic Scheduling theory a major payoff of DoD-sponsored research. The report stated that “system designers can use this theory to predict whether task deadlines will be met long before the costly implementation phase of a project begins. It also eases the process of making modifications to application software.” In his October 1992 Speech, “Charting The Future: Challenges and Promises Ahead of Space Exploration”, Aaron Cohen, Deputy Administrator of the U.S. National Aeronautics and Space Administration (NASA), said, “through the development of Rate Monotonic Scheduling, we now have a system that will allow [International Space Station] Freedom’s computers to budget their time, to choose between a variety of tasks, and decide not only which one to do first but how much time to spend in the process.”




```
void OS_EventWaitListInit (OS_EVENT *pevent)
{
    #if OS_LOWEST_PRIO <= 63
        INT8U *ptbl;
    #else
        INT16U *ptbl;
    #endif
    INT8U i;

    pevent->OSEventGrp = 0;
    ptbl = &pevent->OSEventTbl[0];

    for (i = 0; i < OS_EVENT_TBL_SIZE; i++) {
        *ptbl++ = 0;
    }
}
```



OSCtxSw:

```

STMFD    SP!, {LR}           ;PC
STMFD    SP!, {R0-R12, LR}   ;R0-R12 LR
MRS      R0, CPSR            ;Push CPSR
STMFD    SP!, {R0}

```



```

;-----
;          OSTCBCur->OSTCBStkPtr = SP
;-----
LDR      R0, =OSTCBCur
LDR      R0, [R0]
STR      SP, [R0]

```



```

BL      OSTaskSwHook

```

*OS_EXT OS_TCB *OSTCBCur;*

```
;-  
; OSTCBCur = OSTCBHighRdy;  
;-  
LDR      R0, =OSTCBHighRdy  
LDR      R1, =OSTCBCur  
LDR      R0, [R0]  
STR      R0, [R1]  
  
;-  
; OSPrioCur = OSPrioHighRdy;  
;-  
LDR      R0, =OSPrioHighRdy  
LDR      R1, =OSPrioCur  
LDRB     R0, [R0]  
STRB     R0, [R1]
```



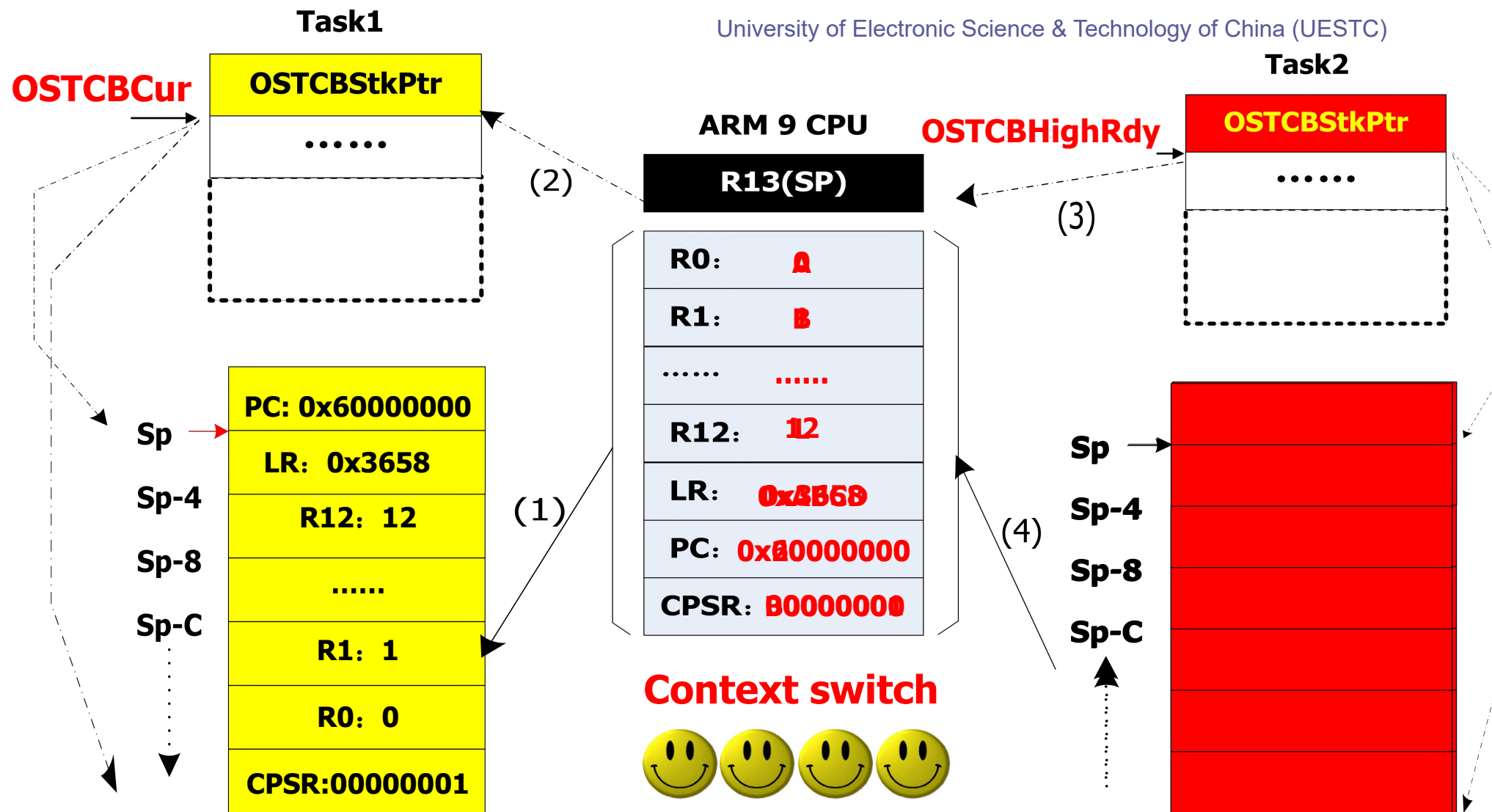
```
OS_EXT OS_TCB *OSTCBHighRdy;
```

```
;-  
; OSTCBHighRdy->OSTCBStkPtr;  
;-  
LDR      R0, =OSTCBHighRdy  
LDR      R0, [R0]  
LDR      SP, [R0]  
;-  
;Restore New task context  
;-  
LDMFD    SP!, {R0}      ;POP CPSR  
MSR      SPSR_cxsf, R0  
LDMFD    SP!, {R0-R12, LR, PC}^
```



```
OS_EXT OS_TCB *OSTCBHighRdy;
```





.....

```
ptcb->OSTCBY = (INT8U)(prio >> 3);  
ptcb->OSTCBBitY = OSMaPTbl[ptcb->OSTCBY];  
ptcb->OSTCBX = (INT8U)(prio & 0x07);  
ptcb->OSTCBBitX = OSMaPTbl[ptcb->OSTCBX];
```

.....

```
OSRdyGrp /= ptcb->OSTCBBitY;  
OSRdyTbl[ptcb->OSTCBY] /= ptcb->OSTCBBitX;
```

.....



	⋮
0x1C	FIQ
0x18	IRQ
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

HAL_VECTR_START:

```

LDR    pc, VECTOR_TABLE+0    @ Reset
LDR    pc, VECTOR_TABLE+4    @ Undefined
LDR    pc, VECTOR_TABLE+8    @ SWI
LDR    pc, VECTOR_TABLE+0xc   @ Prefetch Abort
LDR    pc, VECTOR_TABLE+0x10  @ Data Abort
LDR    pc, VECTOR_TABLE+0x14  @ RESERVED
LDR    pc, VECTOR_TABLE+0x18  @ IRQ
LDR    pc, VECTOR_TABLE+0x1c  @ FIQ

```

VECTOR_TABLE:

```

.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long IRQ
.long EXP_HANDLER

```

HAL_VECTR_END:

	⋮
0x1C	FIQ
0x18	0X30000000
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

0X30000000**IRQ**

```

mov    r0, #4
mov    r1, #5
add    r0, r0, r1
b      .
.....
sub    pc, lr, #4

```

**P1: Where to return ?****P2: How to distinguish different interrupts?****P6: Is it an address in each vector?**

0X30000000**IRQ**


```

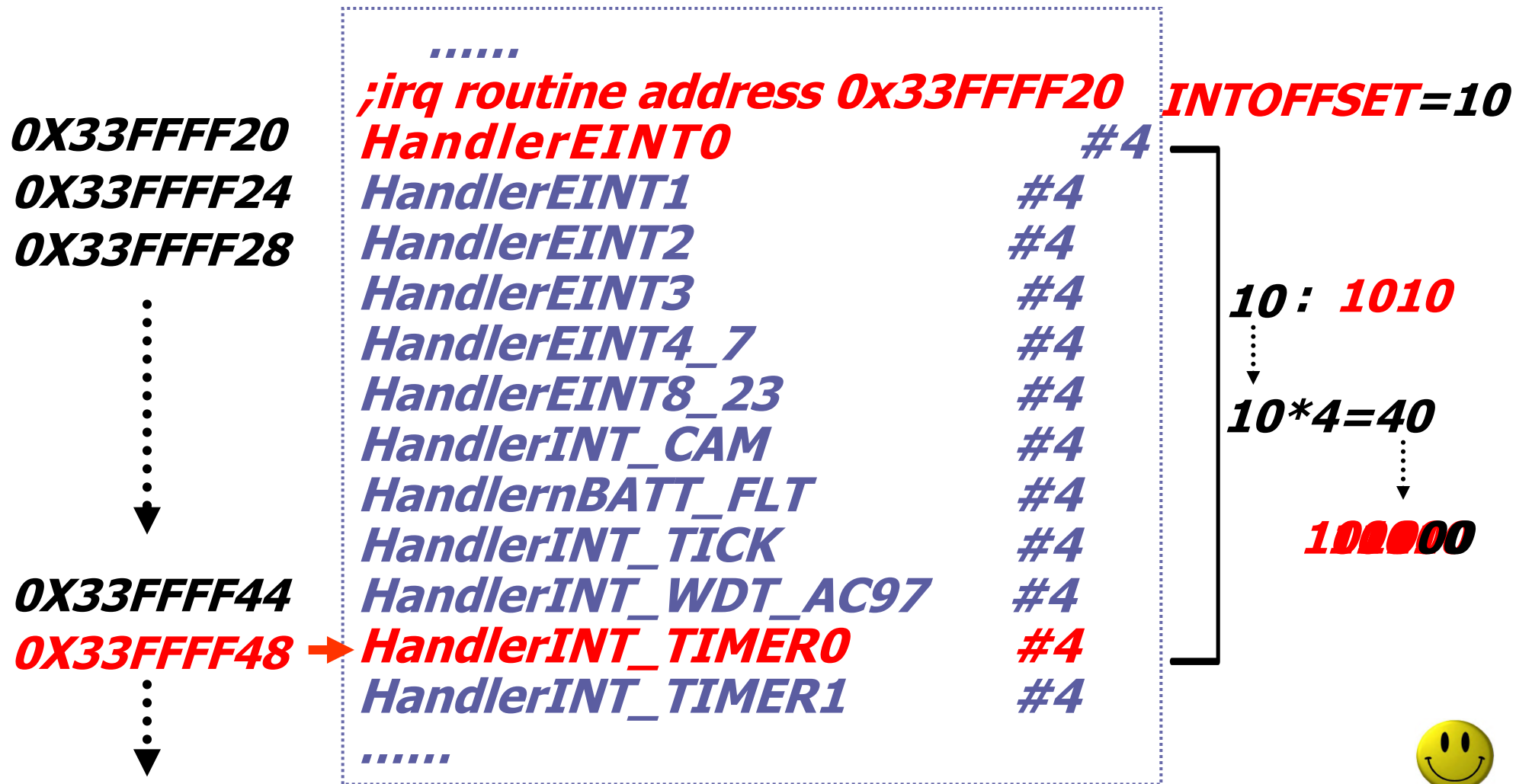
LDR    r0, =INTOFFSET
LDR    r0, [r0]
LDR    r1, =HandleEINT0
LDR    pc, [r1, r0 lsl #2]
.....

```

1010**0x4a000014****0X33FFFF20****101000: 0X28**

P3: How to save the information of the interrupted program?





0X30000000
0X30000024
0X30000028
0X3000002C
IRQ

```

sub      lr,lr,#4
stmfd    sp!,{r0.r12,lr}
mrs      r0,spsr
stmfd    sp!,{r0}
ldr      r0,=INTOFFSET           ;10: 0X28
ldr      r0,[r0]
ldr      r1,=HandlerEINT0       ;0x33FFFF20
add      r1,r1,r0,lsr #2
ldr      r1,[r1]                ;Add of timer ISR
mov      lr,pc                  ;Why not (pc-4)
mov      pc,r1
ldmfd    sp!,{r0}
msr      spsr_cxsf,r0
ldmfd    sp!,{r0.r12,lr}
movs     pc,lr
  
```



0X40000000**OSTickISR**

```

MOV R5,LR
MOV R1, #1
MOV R1, R1, LSL #10
LDR R0, =SRCPND
LDR R2, [R0]
ORR R1, R1, R2
STR R1, [R0]
LDR R0, =INTPND
LDR R1, [R0]
STR R1, [R0]
BL OSTimeTick (.....)
.....

```



P4: How to process the IRQ if uc/OS II is running? 

P5: How to process context switch triggered by the IRQ?  

```
typedef struct {  
    void *base_adr;  
    void *res_free;  
    acoral_id id;  
    acoral_u32 size;  
    acoral_u32 num;  
    acoral_u32 free_num;  
    acoral_pool_ctrl_t *ctrl;  
    acoral_list_t ctrl_list;  
    acoral_list_t free_list;  
#ifdef CFG_CMP  
    acoral_spinlock_t lock;  
#endif  
}acoral_pool_t;
```



- A task is a **executable software entity** specialized to perform a special action (Enemy Detect, Danger Evaluation, Missile Launching.....), which includes a **sequential code**, and **relative data** and **computer resource**.
- A task is an **infinite loop function**. it looks just like any other C function containing a return type and an argument, but it must **never return**. The return type of a task must always be declared to be ***void***.



```
typedef struct os_tcb {  
    OS_STK      *OSTCBStkPtr;  
  
    #if OS_TASK_CREATE_EXT_EN > 0  
        void      *OSTCBExtPtr;  
        OS_STK      *OSTCBStkBottom;  
        INT32U      OSTCBStkSize;  
        INT16U      OSTCBOpt;  
        INT16U      OSTCBIId;  
    #endif  
  
    struct os_tcb *OSTCBNext;  
    struct os_tcb *OSTCBPrev;
```



```
#if ((OS_Q_EN > 0) && (OS_MAX_QS > 0)) || (OS_MBOX_EN > 0)  
|| (OS_SEM_EN > 0) || (OS_MUTEX_EN > 0)  
    OS_EVENT      *OSTCBEventPtr;  
#endif  
  
#if ((OS_Q_EN > 0) && (OS_MAX_QS > 0)) || (OS_MBOX_EN > 0)  
    void          *OSTCBMsg;  
#endif  
.....
```

```
INT16U    OSTCBDly;  
INT8U     OSTCBStat;  
INT8U     OSTCBPrio;
```



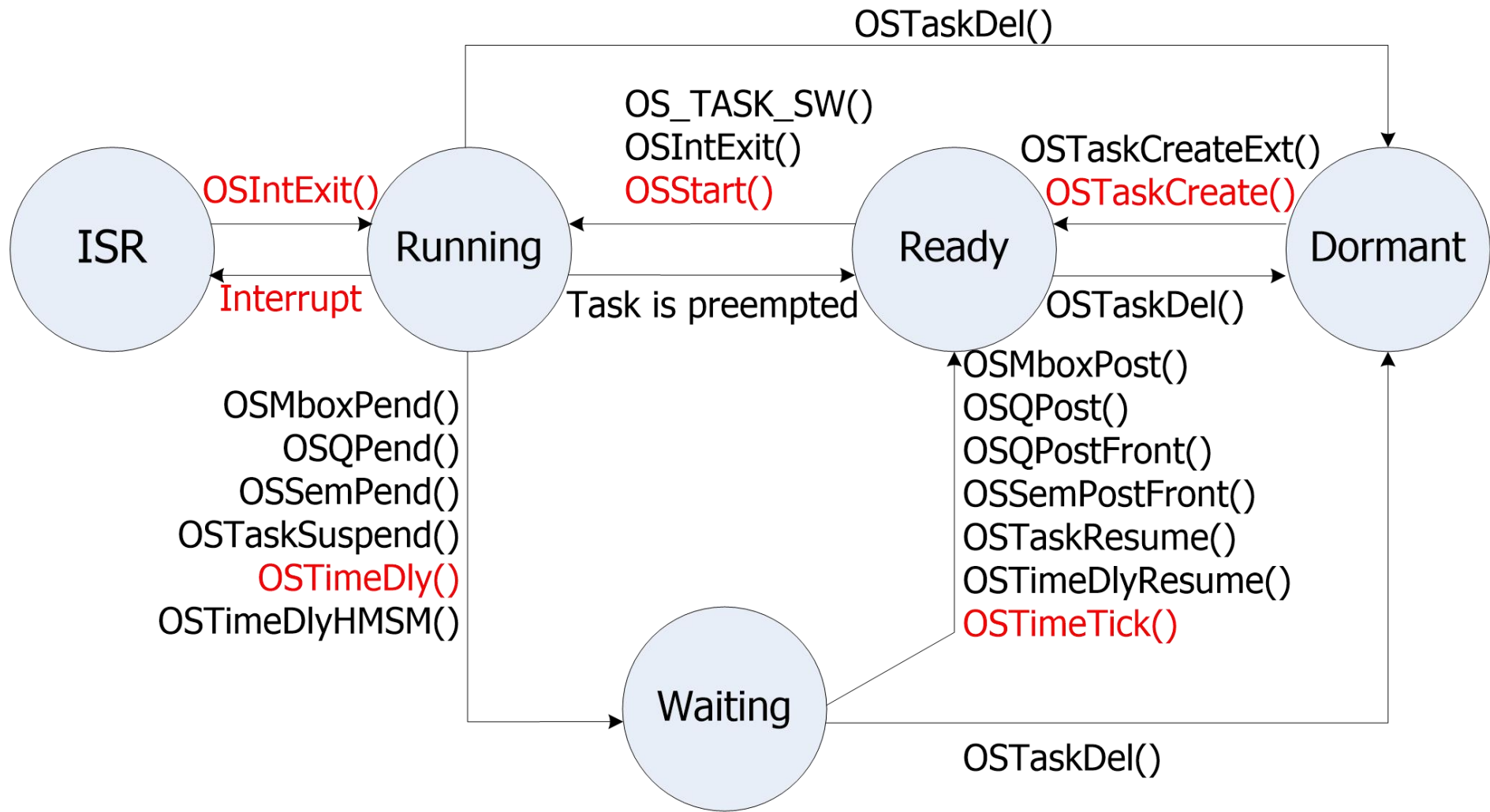
```
INT8U     OSTCBX;  
INT8U     OSTCBY;  
INT8U     OSTCBBitX;  
INT8U     OSTCBBitY;
```



```
#if OS_TASK_DEL_EN > 0  
    BOOLEAN    OSTCBDelReq;  
#endif  
} OS_TCB;
```



University of Electronic Science & Technology of China (UESTC)



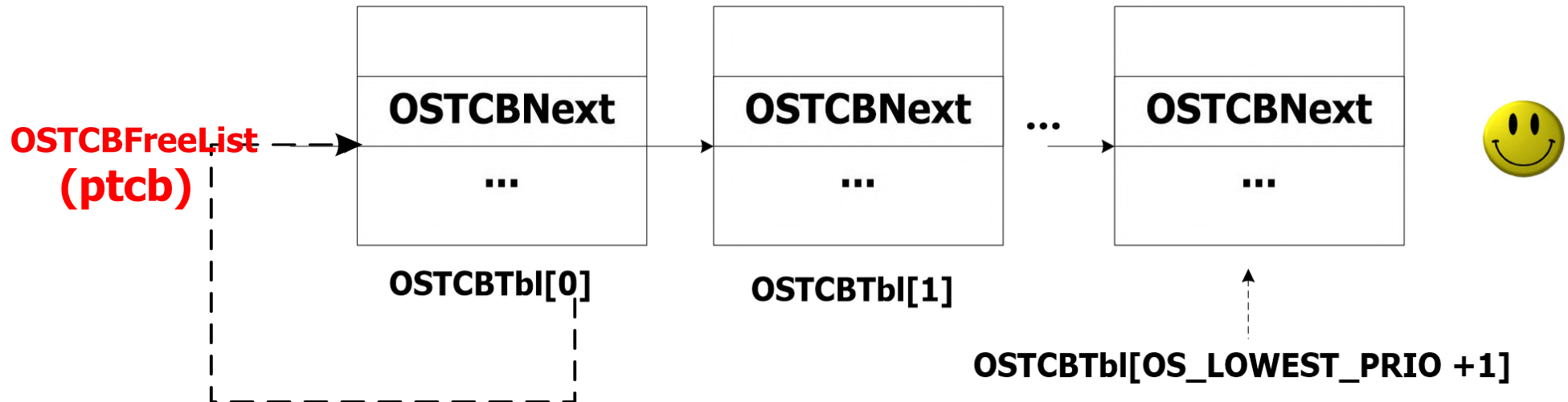
OS_TCB *OSTCBPrioTbl[OS_LOWEST_PRIO + 1];

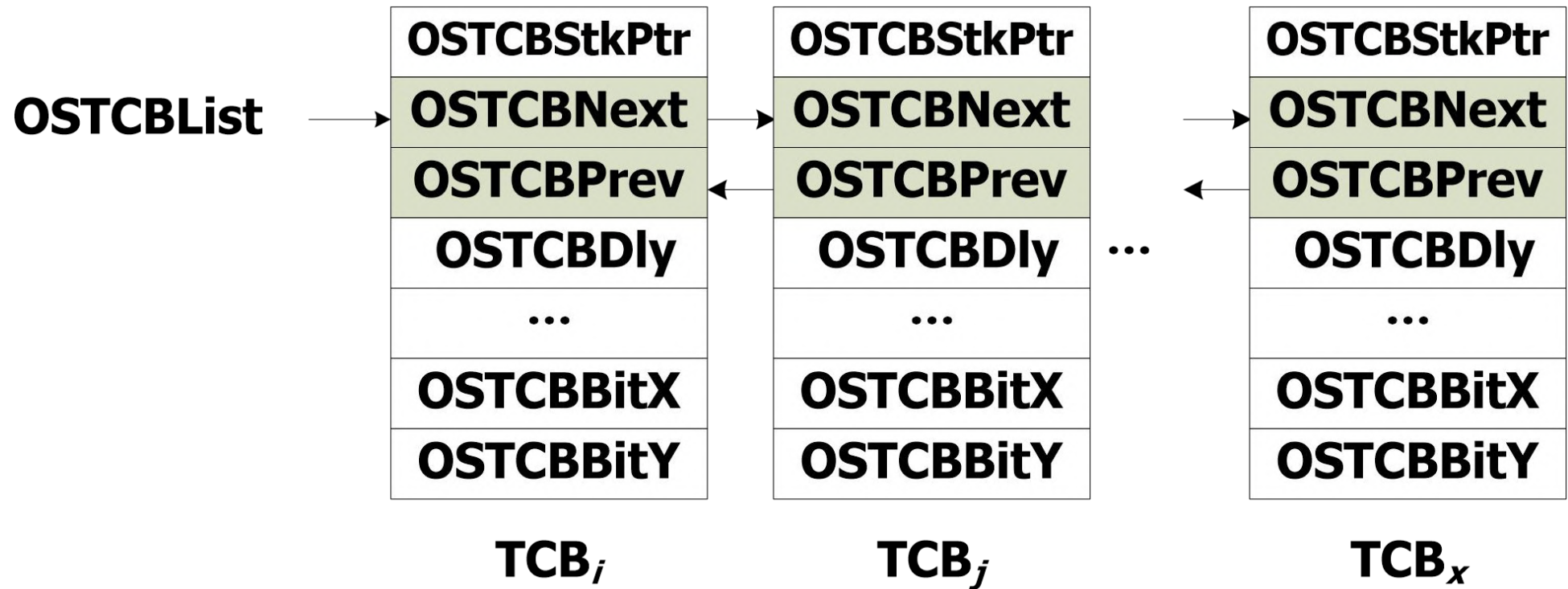


OS_TCB *OSTCBFreeList;
OS_TCB OSTCBTbl[OS_LOWEST_PRIO + 1];

OS_InitTCBList()

OSTCBFreeList = &OSTCBTbl[0]





```
INT8U OSTaskCreate (void (*task)(void *p_arg), void *p_arg,  
OS_STK *ptos,INT8U prio)  
{  
    OS_STK *psp;  
    #if OS_CRITICAL_METHOD == 3  
        OS_CPU_SR cpu_sr = 0;  
    #endif  
    OS_ENTER_CRITICAL();  
    if (OSIntNesting > 0) {  
        OS_EXIT_CRITICAL();  
        return (OS_ERR_TASK_CREATE_ISR);  
    }  
    if (OSTCBPrioTbl[prio] == (OS_TCB *)0) {  
        OSTCBPrioTbl[prio] = (OS_TCB *)1;  
        OS_EXIT_CRITICAL();  
    }  
}
```



```
psp = (OS_STK *)OSTaskStkInit(task, pdata, ptos, 0);  
err = OS_TCBInit(prio, psp, (OS_STK *)0, 0, 0, (void *)0, 0);  
if (err == OS_NO_ERR) {  
    OS_ENTER_CRITICAL();  
    OSTaskCtr++;  
    OS_EXIT_CRITICAL();  
    if (OSRunning == TRUE) {  
        OS_Sched();  
    }  
}
```



```
}else {  
    OS_ENTER_CRITICAL();  
    OSTCBPrioTbl[prio] = (OS_TCB *)0;  
    OS_EXIT_CRITICAL();  
}  
return (err);  
}  
OS_EXIT_CRITICAL();  
return (OS_PRIO_EXIST);  
}
```

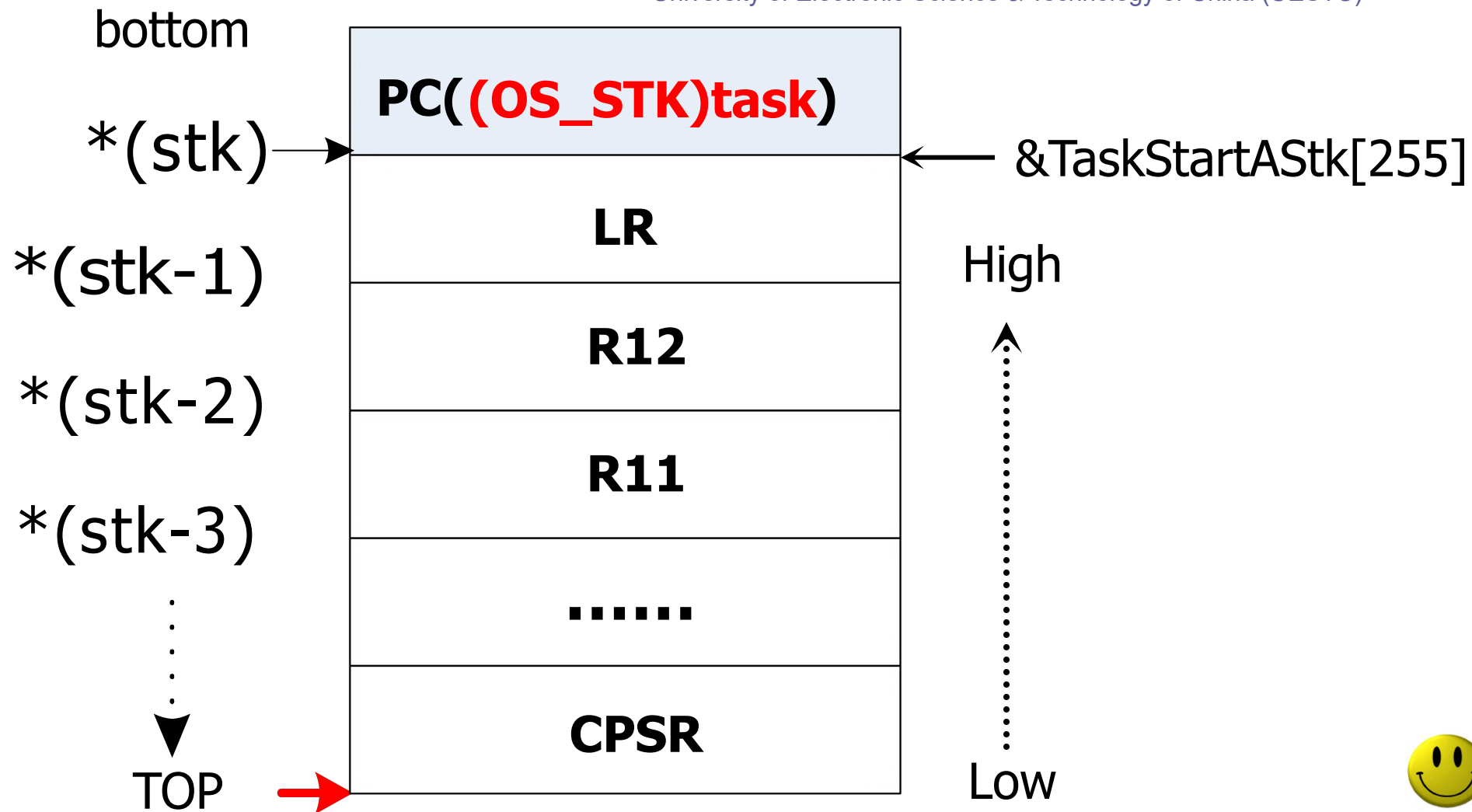


```

OS_STK *OSTaskStkInit (void (*task)(void *pd), void *p_arg,
OS_STK *ptos, INT16U opt)
{
    OS_STK *stk;
    opt    = opt;
    stk    = ptos;
    *(stk) = (OS_STK)task;          /* Entry Point*/
    *(--stk) = (INT32U)0;             /*LR */
    *(--stk) = (INT32U)0;             /*R12*/
    *(--stk) = (INT32U)0;             /*R11*/
    *(--stk) = (INT32U)0;             /*R10*/
    *(--stk) = (INT32U)0;             /*R9 */
    *(--stk) = (INT32U)0;             /*R8 */
}

```

```
*(--stk) = (INT32U)0;          /*R7*/  
*(--stk) = (INT32U)0;          /*R6*/  
*(--stk) = (INT32U)0;          /*R5*/  
*(--stk) = (INT32U)0;          /*R4*/  
*(--stk) = (INT32U)0;          /*R3*/  
*(--stk) = (INT32U)0;          /*R2*/  
*(--stk) = (INT32U)0;          /*R1*/  
*(--stk) = (INT32U)p_arg;  
*(--stk) = (INT32U)0x 0x00000013L;  
return (stk);  
}
```

```
INT8U OS_TCBInit (INT8U prio, OS_STK *ptos, OS_STK  
*pbos, INT16U id, INT32U stk_size, void *pext, INT16U opt)  
{  
    #if OS_CRITICAL_METHOD == 3  
        OS_CPU_SR cpu_sr;  
    #endif  
    OS_TCB *ptcb;  
    OS_ENTER_CRITICAL();  
    ptcb = OSTCBFreeList;   
  
    if (ptcb != (OS_TCB *)0) {  
        OSTCBFreeList = ptcb->OSTCBNext;    
        OS_EXIT_CRITICAL();
```

```
ptcb->OSTCBStkPtr= ptos;  
ptcb->OSTCBPrio = (INT8U)prio;  
ptcb->OSTCBStat = OS_STAT_RDY;  
ptcb->OSTCBDly = 0;  
pext = pext;  
stk_size = stk_size;  
pbos = pbos;  
opt = opt;  
id = id;  
  
ptcb->OSTCBY = prio >> 3;  
ptcb->OSTCBBitY= OSMaTbl[ptcb->OSTCBY];  
ptcb->OSTCBX = prio & 0x07;  
ptcb->OSTCBBitX= OSMaTbl[ptcb->OSTCBX];
```



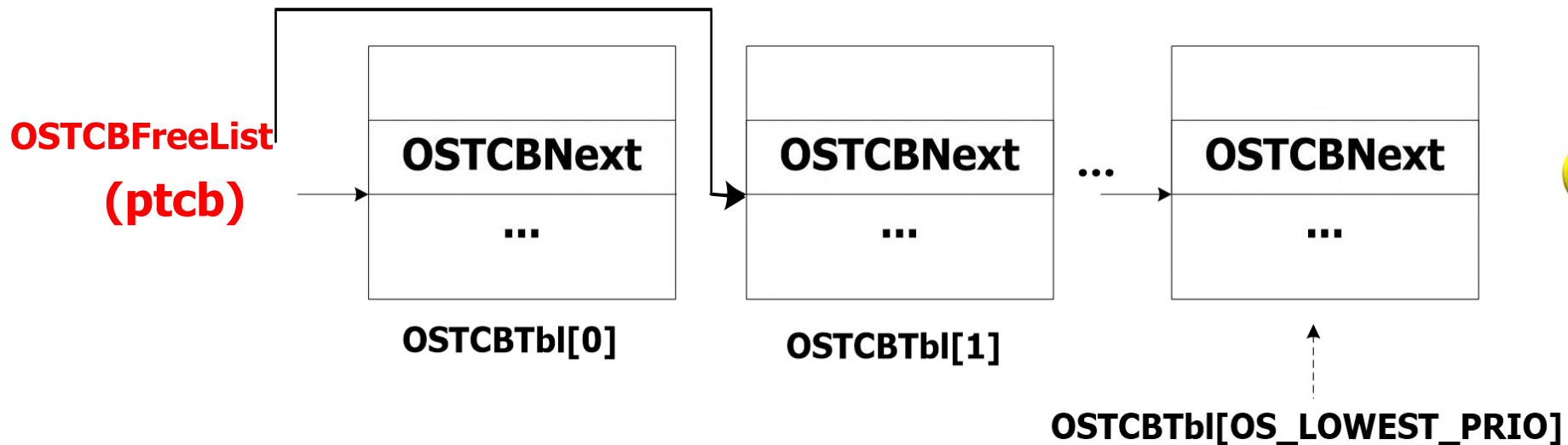
```
OS_ENTER_CRITICAL();  
OSTCBPrioTbl[prio] = ptcb;   
ptcb->OSTCBNext = OSTCBList;   
ptcb->OSTCBPrev = (OS_TCB *)0;  
if (OSTCBList != (OS_TCB *)0) {  
    OSTCBList->OSTCBPrev = ptcb;  
}  
OSTCBList = ptcb;    
OSRdyGrp |= ptcb->OSTCBBitY;  
OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;  
OS_EXIT_CRITICAL();  
return (OS_NO_ERR);  
}  
OS_EXIT_CRITICAL();  
return (OS_NO_MORE_TCB);}
```



```
OS_TCB *OSTCBFreeList;  
OS_TCB OSTCBTbl[OS_LOWEST_PRIO + 1];
```

OS_InitTCBList()

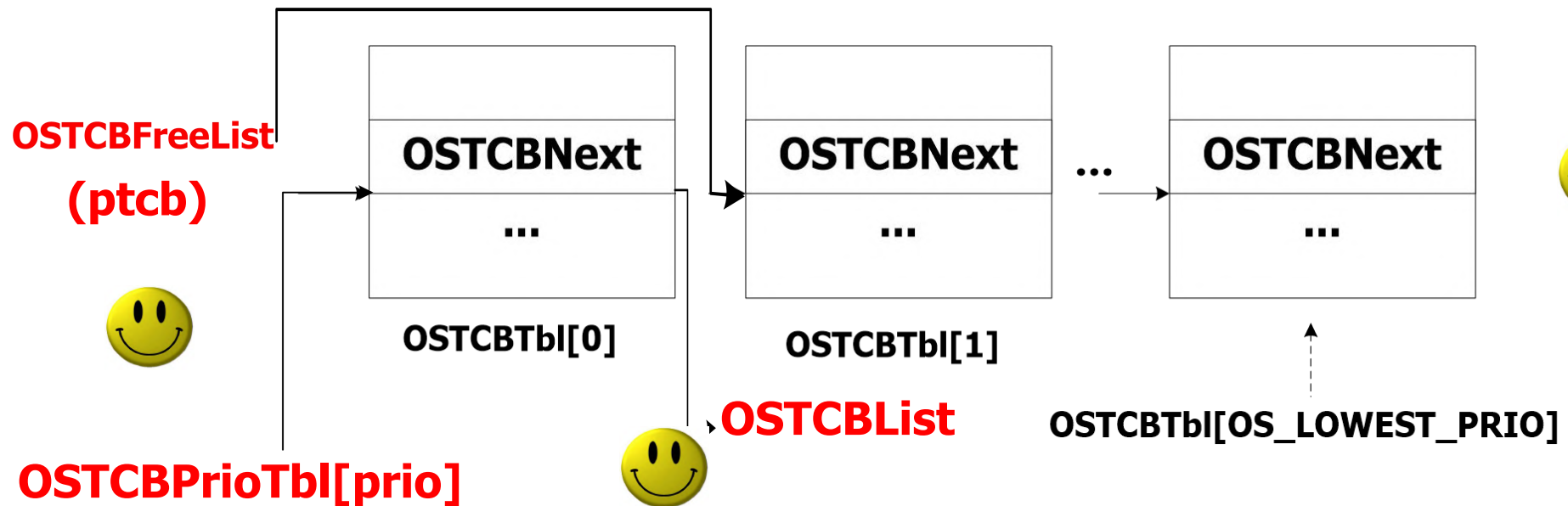
OSTCBFreeList = &OSTCBTbl[0]



```
OS_TCB *OSTCBFreeList;  
OS_TCB OSTCBTbl[OS_LOWEST_PRIO + 1];
```

OS_InitTCBList()

OSTCBFreeList = &OSTCBTbl[0]



```
void OS_Sched(void)
{
    #if OS_CRITICAL_METHOD == 3
        OS_CPU_SR cpu_sr = 0;
    #endif

    OS_ENTER_CRITICAL();
    if (OSIntNesting == 0) {
        if (OSLockNesting == 0) {
            OS_SchedNew();
            if (OSPrioHighRdy != OSPrioCur) {
                OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
            }
        }
    }
}
```



```
#if OS_TASK_PROFILE_EN > 0
    OSTCBHighRdy->OSTCBCtxSwCtr++;
#endif

    OSCtxSwCtr++;
    OS_TASK_SW();
}
}
}
OS_EXIT_CRITICAL();
}
```

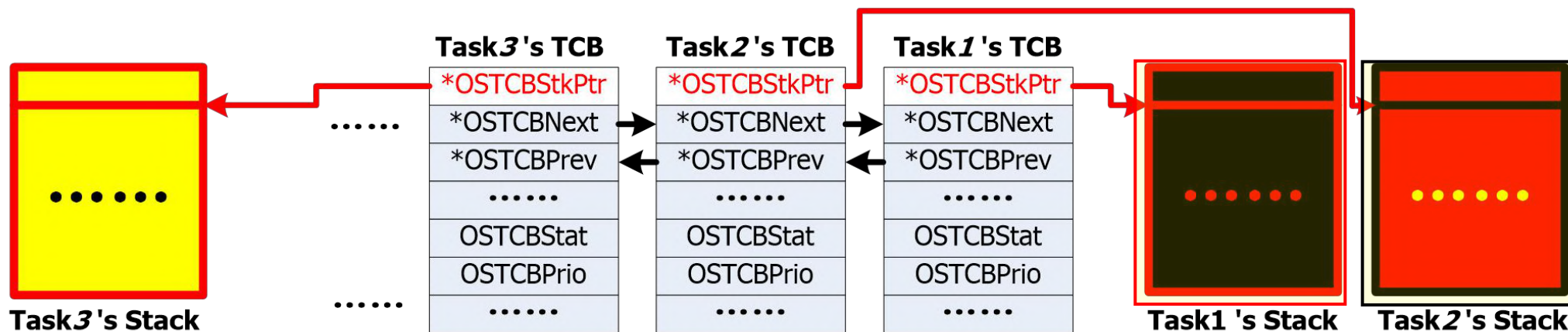



```
static void OS_SchedNew(void)
{
    #if OS_LOWEST_PRIO <= 63
        INT8U y;
        y = OSUnMapTbl[OSRdyGrp];
        OSPrioHighRdy = (INT8U)((y << 3) +
                               OSUnMapTbl[OSRdyTbl[y]]);
    #else
        INT8U y;
        INT16U *ptbl;
        if ((OSRdyGrp & 0xFF) != 0) {
            y = OSUnMapTbl[OSRdyGrp & 0xFF];
        } else {
            y = OSUnMapTbl[(OSRdyGrp >> 8) & 0xFF] + 8;
        }
    }
```

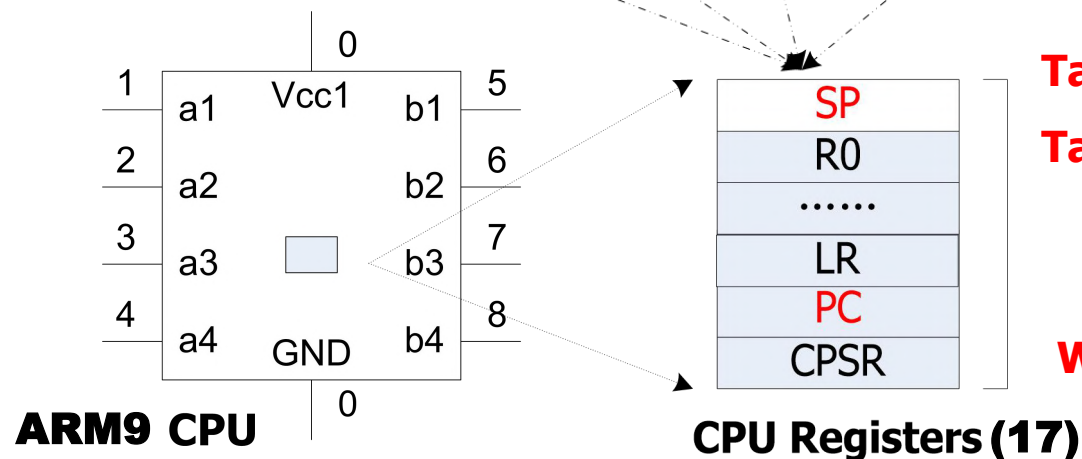


```
ptbl = &OSRdyTbl[y];  
if ((*ptbl & 0xFF) != 0) {  
    OSPrioHighRdy = (INT8U)((y << 4) +  
                             OSUnMapTbl[( *ptbl & 0xFF)]);  
} else {  
    OSPrioHighRdy = (INT8U)((y << 4) +  
                             OSUnMapTbl[( *ptbl >> 8) & 0xFF] + 8);  
}  
#endif  
}
```





MEMORY MAP



Task 1 finishes one iteration
Task2 is the new current task

Context of task1
(Current task is 1)
Where will the registers be saved ?
Task1's stack

- When a kernel decides to run a different task, it simply saves the current task's **context (CPU registers)** in the current task's context storage area (**Current task's stack**). Once this operation is performed, the new task's context is restored from its storage area (**New task's stack**) and then resumes execution of the new task's code. This process is called a **context switch**. (p47)



```
INT8U OSTaskCreate (void (*task)(void *p_arg), void *p_arg,  
OS_STK *ptos,INT8U prio)  
{  
    OS_STK *psp;  
    #if OS_CRITICAL_METHOD == 3   
        OS_CPU_SR cpu_sr = 0;  
    #endif  
    OS_ENTER_CRITICAL();  
    if (OSIntNesting > 0) {  
        OS_EXIT_CRITICAL();  
        return (OS_ERR_TASK_CREATE_ISR);  
    }  
    if (OSTCBPrioTbl[prio] == (OS_TCB *)0) {  
        OSTCBPrioTbl[prio] = (OS_TCB *)1;  
        OS_EXIT_CRITICAL();  
    }
```

```
#if OS_CRITICAL_METHOD==1
```

```
#define OS_ENTER_CRITICAL() (Cli())
```

```
#define OS_EXIT_CRITICAL() (Sti())
```



```
#elseif OS_CRITICAL_METHOD == 2
```

```
#define OS_ENTER_CRITICAL() (PushAndCli())
```

```
#define OS_EXIT_CRITICAL() (Pop())
```



```
#elseif OS_CRITICAL_METHOD == 3
```

```
#define OS_ENTER_CRITICAL() (cpu_sr = OSCPU_SaveSR())
```

```
#define OS_EXIT_CRITICAL() (OSCPURestoreSR(cpu_sr))
```

```
#endif
```



Cli

```

MRS    R0,CPSR
ORR     R1,R0,#0x80
MSR     CPSR_c,R1
MOV     PC,LR

```

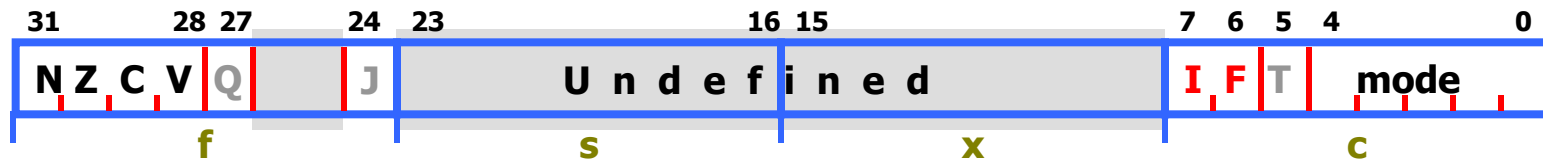
Sti

```

MRS     R0,CPSR
AND     R1,R0,#0xffffffff7f
MSR     CPSR_c,R1
MOV     PC,LR

```

01111111



PushAndCli

```
MRS      R0,CPSR_c  
STMFD    sp!,R0  
ORR      R1,R0,#0x80  
MSR      CPSR_c,R1  
MOV      PC,LR
```

Pop

```
LDMFD    sp!,R0  
MSR      CPSR_c,R0
```



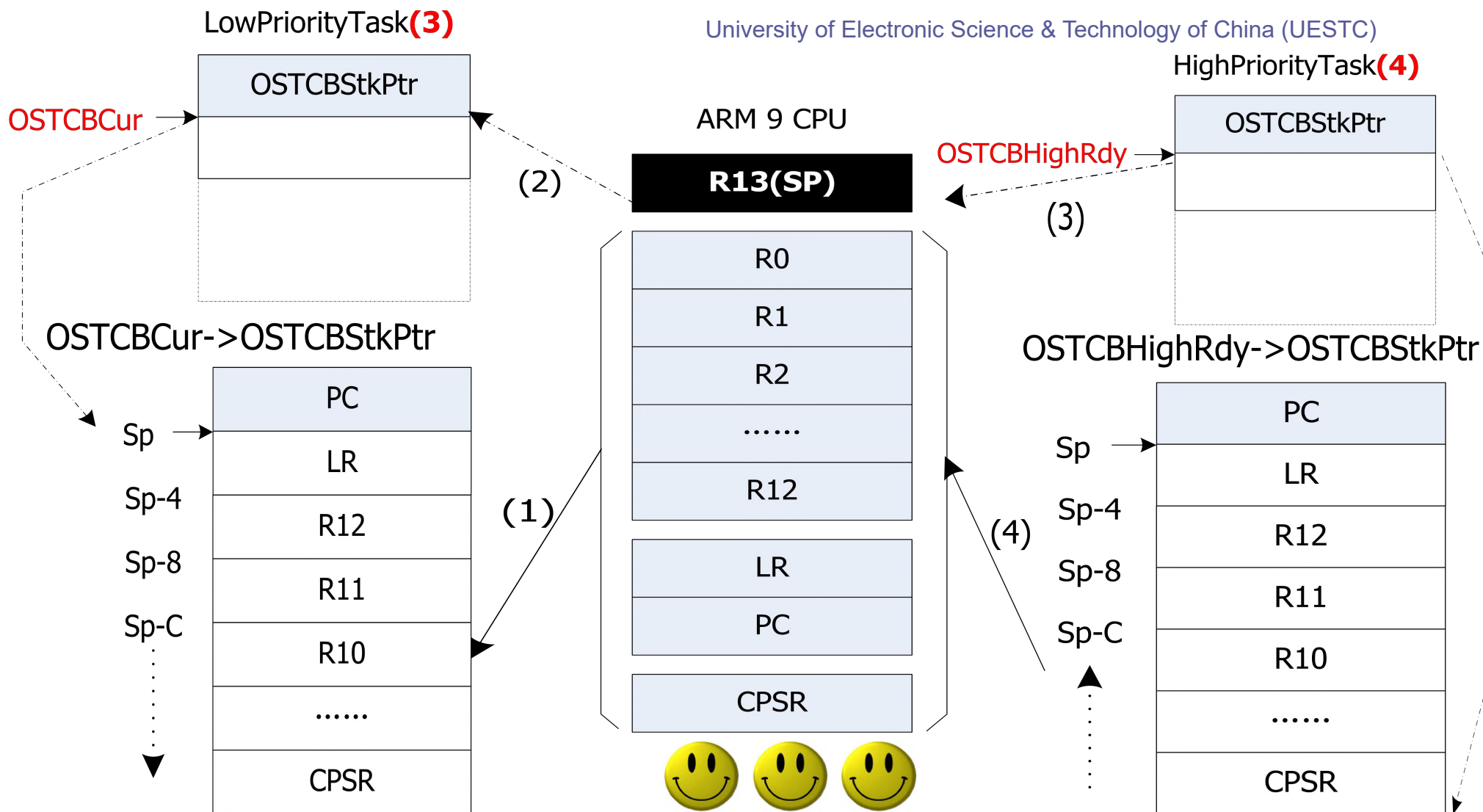
OSCPUSaveSR

```
MRS    R0, CPSR
ORR     R1, R0, #0x80
MSR     CPSR_c, R1
MOV     PC, LR
```

OSCPURestoreSR

```
MSR     CPSR_c, R0
MOV     PC, LR
```





OSTCBCur:

```

STMFD    SP!, {LR}           ;PC
STMFD    SP!, {R0-R12, LR}   ;R0-R12 LR
MRS      R0, CPSR            ;Push CPSR
STMFD    SP!, {R0}

```



```

;-----
;          OSTCBCur->OSTCBStkPtr = SP
;-----
LDR      R0, =OSTCBCur
LDR      R0, [R0]
STR      SP, [R0]

```

```
OS_EXT OS_TCB *OSTCBCur;
```

```
;/-----  
; OSTCBCur = OSTCBHighRdy;  
;/-----  
LDR      R0, =OSTCBHighRdy  
LDR      R1, =OSTCBCur  
LDR      R0, [R0]  
STR      R0, [R1]
```



```
;/-----  
; OSPrioCur = OSPrioHighRdy;  
;/-----  
LDR      R0, =OSPrioHighRdy  
LDR      R1, =OSPrioCur  
LDRB     R0, [R0]  
STRB     R0, [R1]
```



```
OS_EXT OS_TCB *OSTCBHighRdy;
```

```

;-----
; OSTCBHighRdy->OSTCBStkPtr;
;-----
LDR      R0, =OSTCBHighRdy
LDR      R0, [R0]
LDR      SP, [R0]
;-----
;Restore New task context
;-----
LDMFD    SP!, {R0}      ;POP CPSR
MSR      SPSR_cxsf, R0
LDMFD    SP!, {R0-R12, LR, PC}^

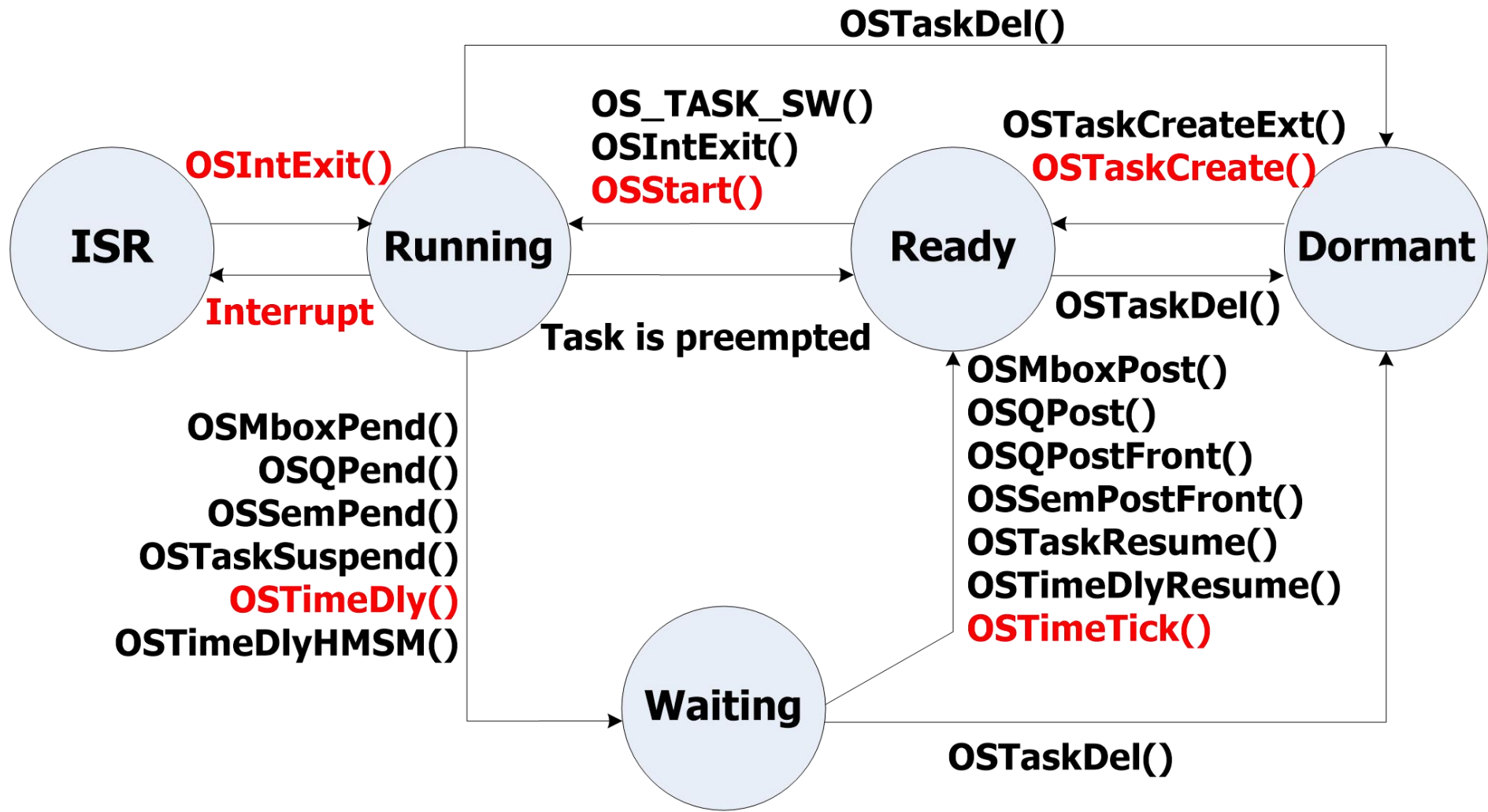
```



```
OS_EXT OS_TCB *OSTCBHighRdy;
```



University of Electronic Science & Technology of China (UESTC)



0X30000000 IRQ

stmdb	sp!,{r0-r2}	;sp_irq
mov	r0,sp	
add	sp,sp,#12	
sub	r1,lr,#4	;lr_irq
mrs	r2,spsr	
msr	cpsr_cxsf,#SVCMODE/NOINT	
stmdb	sp!,{r1}	;sp_svc
stmdb	sp!,{r3-r12,lr}	
ldmia	r0!,{r3-r5}	
stmdb	sp!,{r2-r5}	

```
ldr      r0,=OSIntNesting  
ldr      r1,[r0]  
add      r1,r1,#1  
strb     r1,[r0]  
teq      r1,#1  
bne      %F1
```

```
ldr      r0,=OSTCBCur  
ldr      r0,[r0]  
str      sp,[r0]
```



```

1  msr          psr_c, #IRQMODE / NOINT
   ldr          r0, = INTOFFSET           ;10: 0x28
   ldr          r0, [r0]
   ldr          r1, = HandleEINT0        ;0x33FFFF20 😊

   mov          lr, pc
   ldr          pc, [r1, r0, lsl #2]      ;OSTickISR 😊😊
   msr          cpsr_c, #SVCMODE / NOINT
   bl           OSIntExit 😊

   ldr          r0, [sp], #4
   msr          spsr_cxsf, r0
   ldmia        sp!, {r0-r12, lr, pc}^

```



```
void OSIntExit (void)  
{  
#if OS_CRITICAL_METHOD == 3  
    OS_CPU_SR cpu_sr;  
#endif  
  
    if (OSRunning == TRUE) {  
        OS_ENTER_CRITICAL();  
        if (OSIntNesting > 0) {  
            OSIntNesting--;  
        }  
    }
```

```

If ((OSIntNesting == 0) && (OSLockNesting == 0)) {
OSIntExitY= OSUnMapTbl[OSRdyGrp];
OSPrioHighRdy=(INT8U)((OSIntExitY<<3)+ 
OSUnMapTbl[OSRdyTbl[OSIntExitY]]);
if (OSPrioHighRdy != OSPrioCur) {
OSTCBHighRdy= OSTCBPrioTbl[OSPrioHighRdy];
OSCtxSwCtr++;
OSIntCtxSw(); 
}
}
OS_EXIT_CRITICAL();
}
}

```

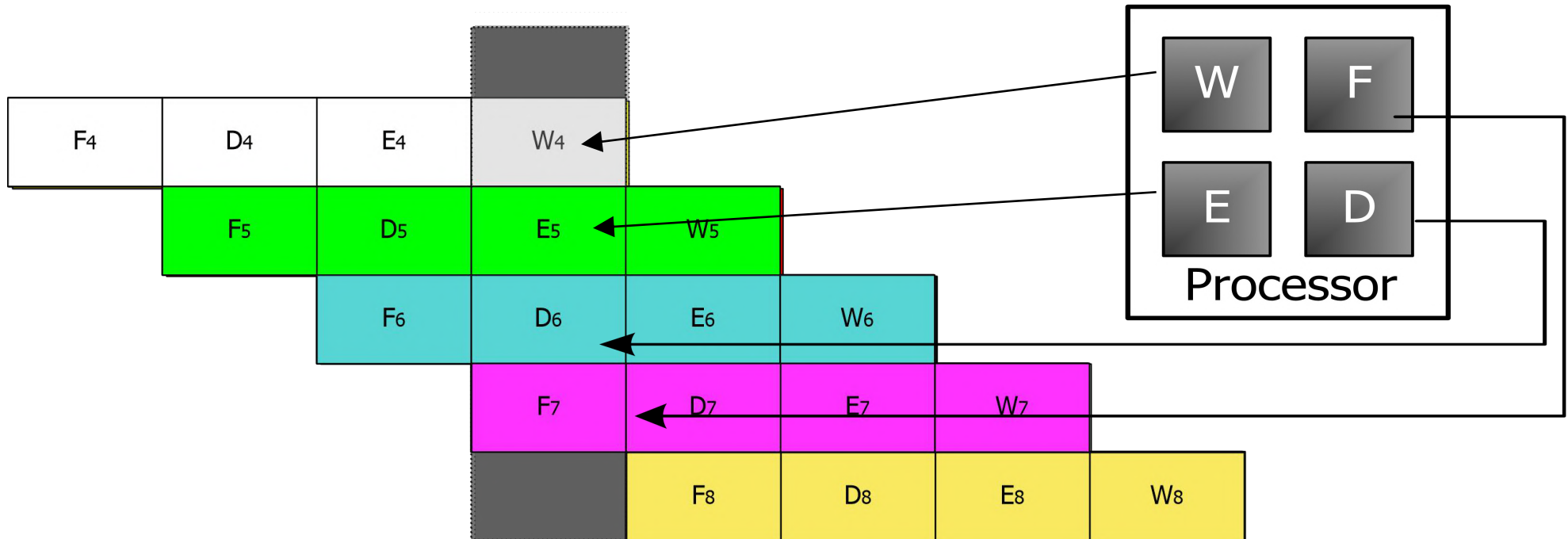


```
void OSTimeTick (void)
{
#if OS_CRITICAL_METHOD == 3
    OS_CPU_SR cpu_sr;
#endif
    OS_TCB *ptcb;
    if (OSRunning == TRUE) {
        ptcb = OSTCBLIST;
        while (ptcb->OSTCBPrio != OS_IDLE_PRIO) {
            OS_ENTER_CRITICAL();
            if (ptcb->OSTCBDly != 0) {
                if (--ptcb->OSTCBDly == 0) {
                    if ((ptcb->OSTCBStat & OS_STAT_SUSPEND)
== OS_STAT_RDY)

```

```
    OSRdyGrp |= ptcb->OSTCBBitY;  
    OSRdyTbl[ptcb->OSTCBBY] |= ptcb->OSTCBBitX;  
} else {  
    ptcb->OSTCBDly = 1;  
}  
}  
}  
ptcb = ptcb->OSTCBNext;  
OS_EXIT_CRITICAL();  
}  
}  
}
```



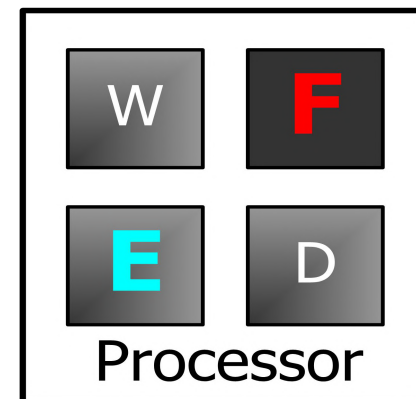


- A instruction is divided into **several segments**. each segment does a certain job. In the CPU, there is **corresponding unit** to do the certain job;
- Traditional design
- The CPU **staggeres** the executions of different segments.

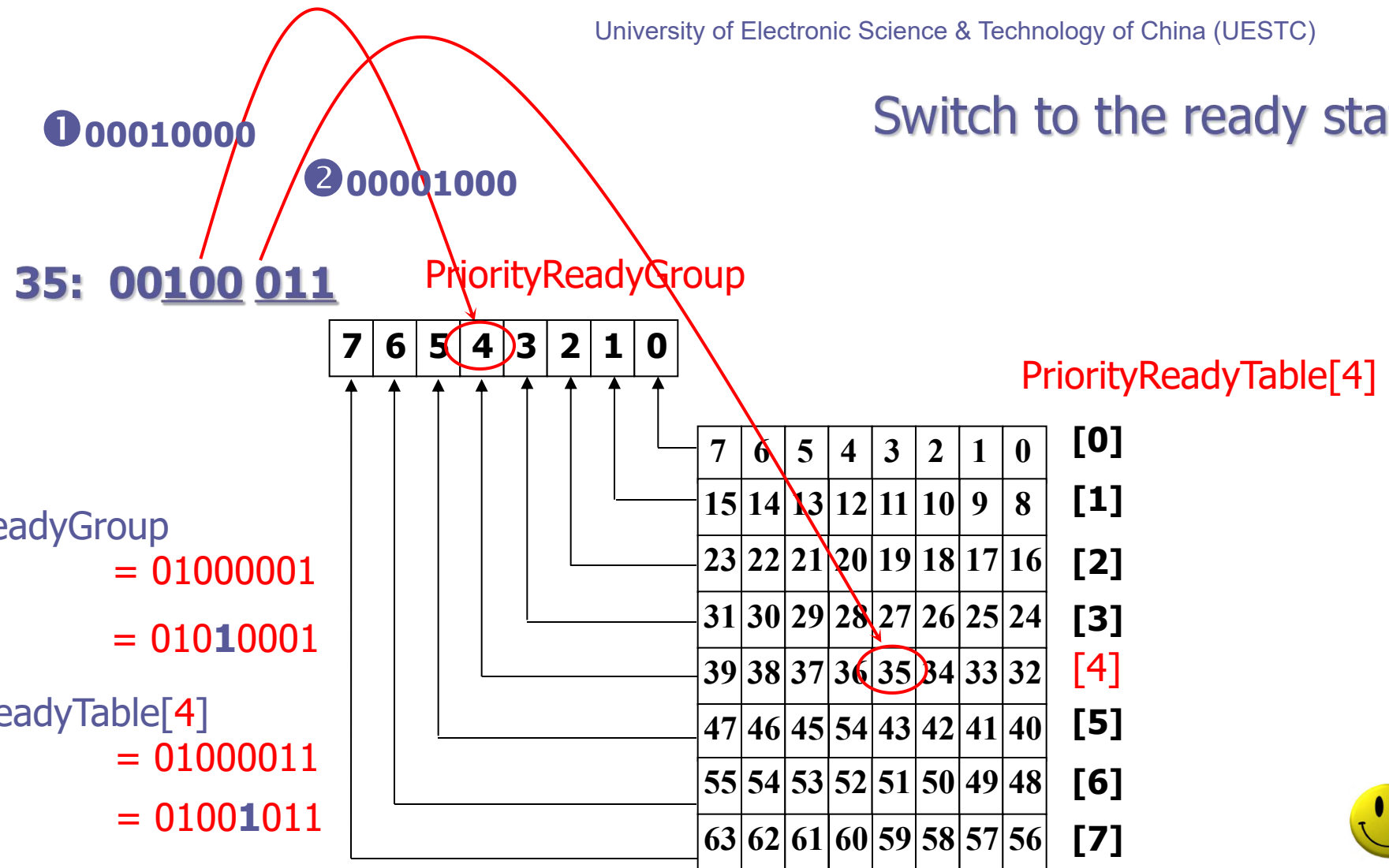
Interrupt !Current... F4 ... D4 ... **E4** ... W4 ... 0x80**(LR-4) Return** → F5 D5 E5 W5 0x84LR ← **PC** → F6 D6 E6 W6 0x88

F7 D7 E7 W7 0x8C

F8 D8 E8 W8 ...



Switch to the ready state



Get the highest priority



Check the PriorityDecisionTable[]

PriorityReadyGroup

PriorityReadyTable[4]

① 0X50 → high3Bit=4

② 0X88 → low3Bit=3

③ 35: 00100 011

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

7	6	5	4	3	2	1	0	[0]
15	14	13	12	11	10	9	8	[1]
23	22	21	20	19	18	17	16	[2]
31	30	29	28	27	26	25	24	[3]
1	0	0	0	1	0	0	0	[4]
47	46	45	54	43	42	41	40	[5]
55	54	53	52	51	50	49	48	[6]
63	62	61	60	59	58	57	56	[7]

Get a pointer to the corresponding task by indexing the OSTCBPriorityTable[]

Quit the ready state

① ~~00001000~~ → 11110111

② 00010000 → 11101111

35: 00100 011

PriorityReadyGroup

PriorityReadyTable[4]

7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---

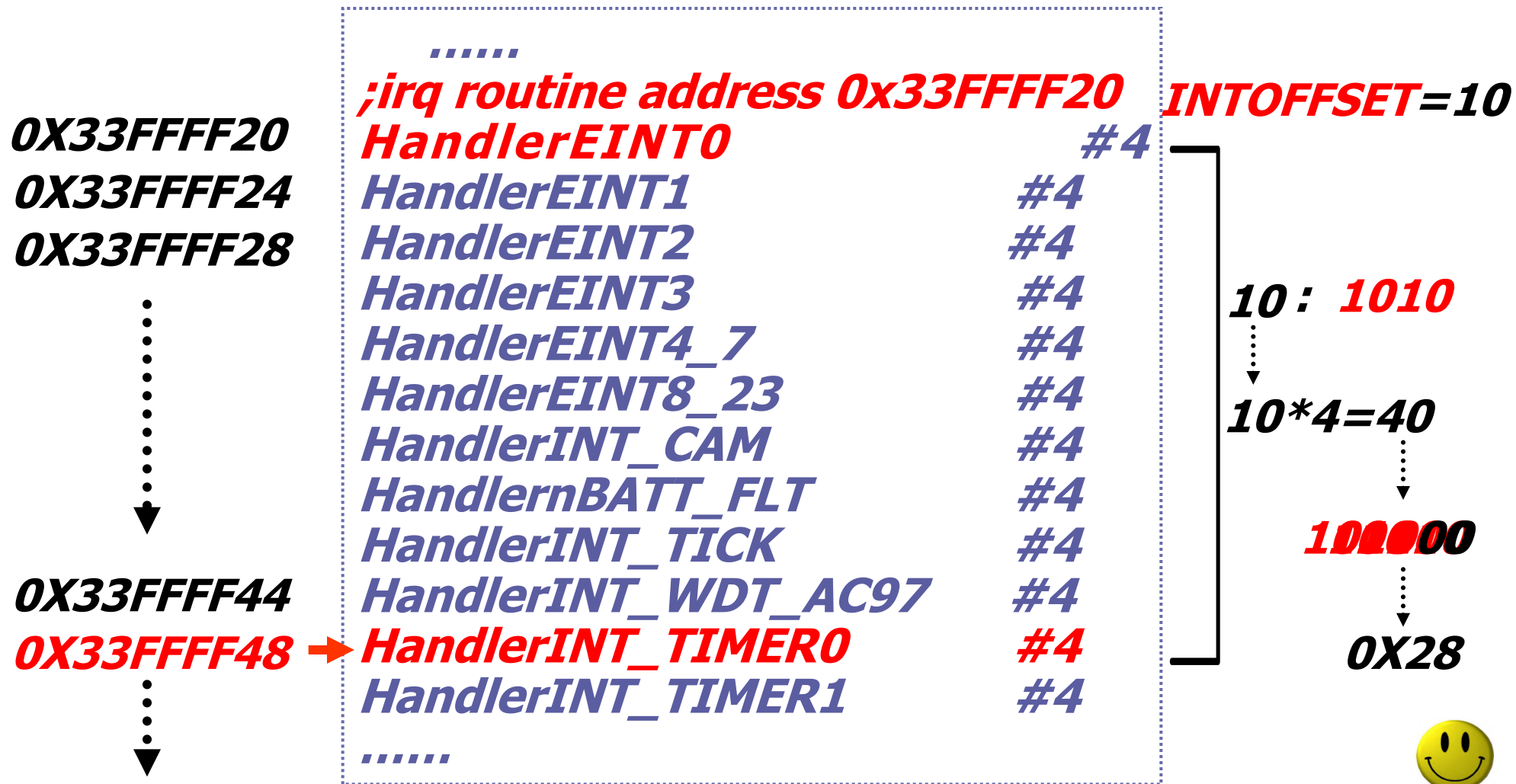
7	6	5	4	3	2	1	0	
15	14	13	12	11	10	9	8	[0]
23	22	21	20	19	18	17	16	[2]
31	30	29	28	27	26	25	24	[3]
39	38	37	36	35	34	33	32	[4]
47	46	45	54	43	42	41	40	[5]
55	54	53	52	51	50	49	48	[6]
63	62	61	60	59	58	57	56	[7]

PriorityReadyGroup

= 010**1**0001= 010**0**0001

PriorityReadyTable[4]

= 0100**1**011= 0100**0**001



University of Electronic Science & Technology of China (UESTC)

	⋮
0x1C	FIQ
0x18	IRQ
0x14	(Reserved)
0x10	Data Abort
0x0C	Prefetch Abort
0x08	Software Interrupt
0x04	Undefined Instruction
0x00	Reset

```

LDR    pc, VECTOR_TABLE+0
LDR    pc, VECTOR_TABLE+4
LDR    pc, VECTOR_TABLE+8
LDR    pc, VECTOR_TABLE+0xc
LDR    pc, VECTOR_TABLE+0x10
LDR    pc, VECTOR_TABLE+0x14
LDR    pc, VECTOR_TABLE+0x18
LDR    pc, VECTOR_TABLE+0x1c
  
```

VECTOR_TABLE:

```

.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long EXP_HANDLER
.long HAL_INTR_ENTRY
.long EXP_HANDLER
  
```



- **Exclusiveness (mutual exclusion):** refers to the requirement of ensuring that no two concurrent tasks are in their **critical section** at the same time.



- **Binary Semaphore:** a semaphore is a protected variable which constitutes the classic method for restricting(exclusive) access to shared resources such as shared memory in a parallel programming environment.



■ APIs provided by the uC/OSII

➤ **Semaphore:** OSSemCreate ()
 OSSemPend ()
 OSSemPost ()

.....

.....

➤ **Mutex:** OSMutexCreate ()
 OSMutexPend ()
 OSMutexPost ()

.....

.....

Sem=OSSemCreate(1)

.....

OSSemPend(Sem,0,&err);

.....

OSSemPost(Sem)



■ APIs provided by the uC/OSII

➤ **Semaphore:** OSSemCreate ()

OSSemPend ()



OSSemPost ()



➤ **Mutex:** OSMutexCreate ()

OSMutexPend ()

OSMutexPost ()

.....

.....

Sem=OSSemCreate(1)

.....

OSSemPend(Sem,0,&err);

.....

OSSemPost(Sem)

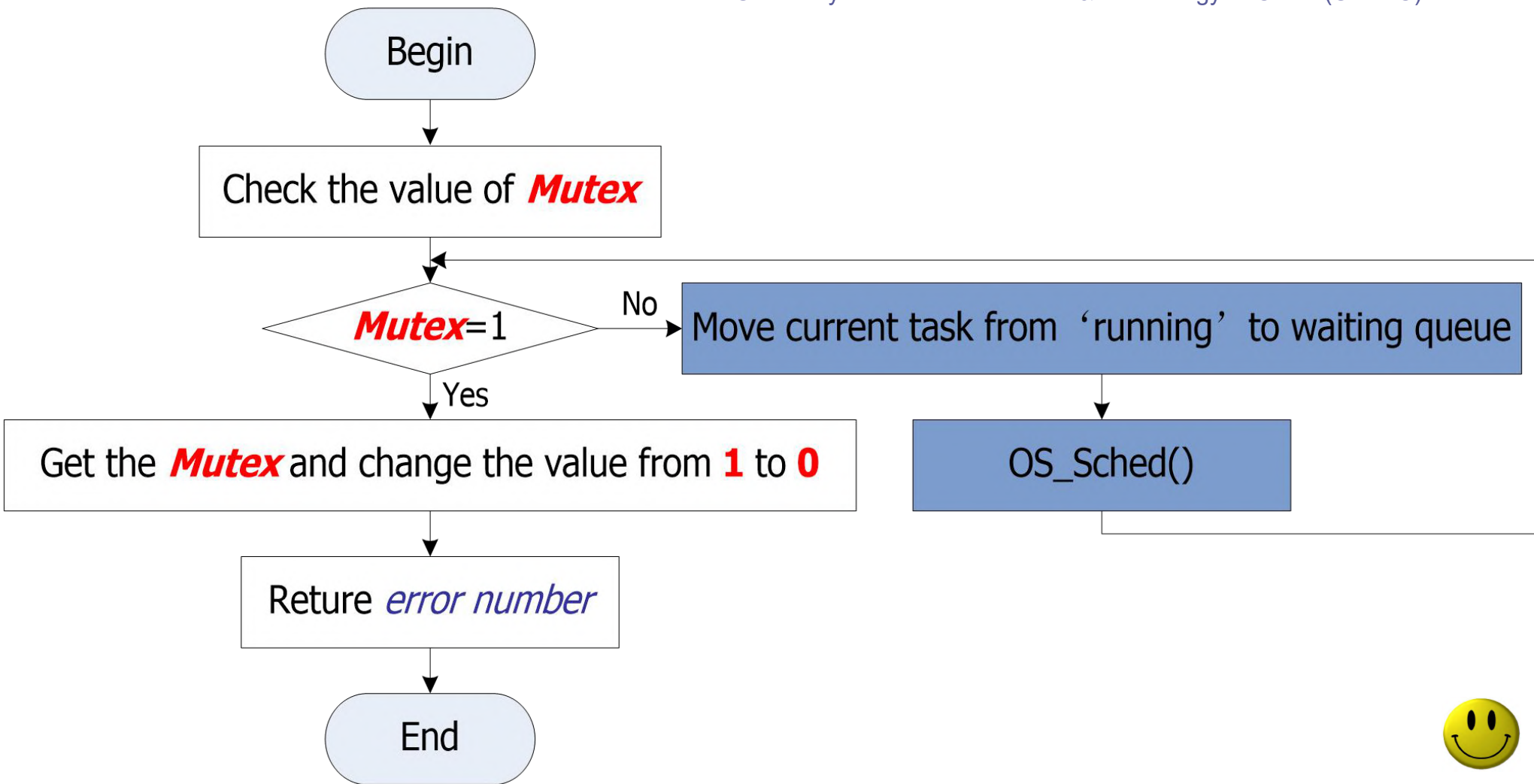


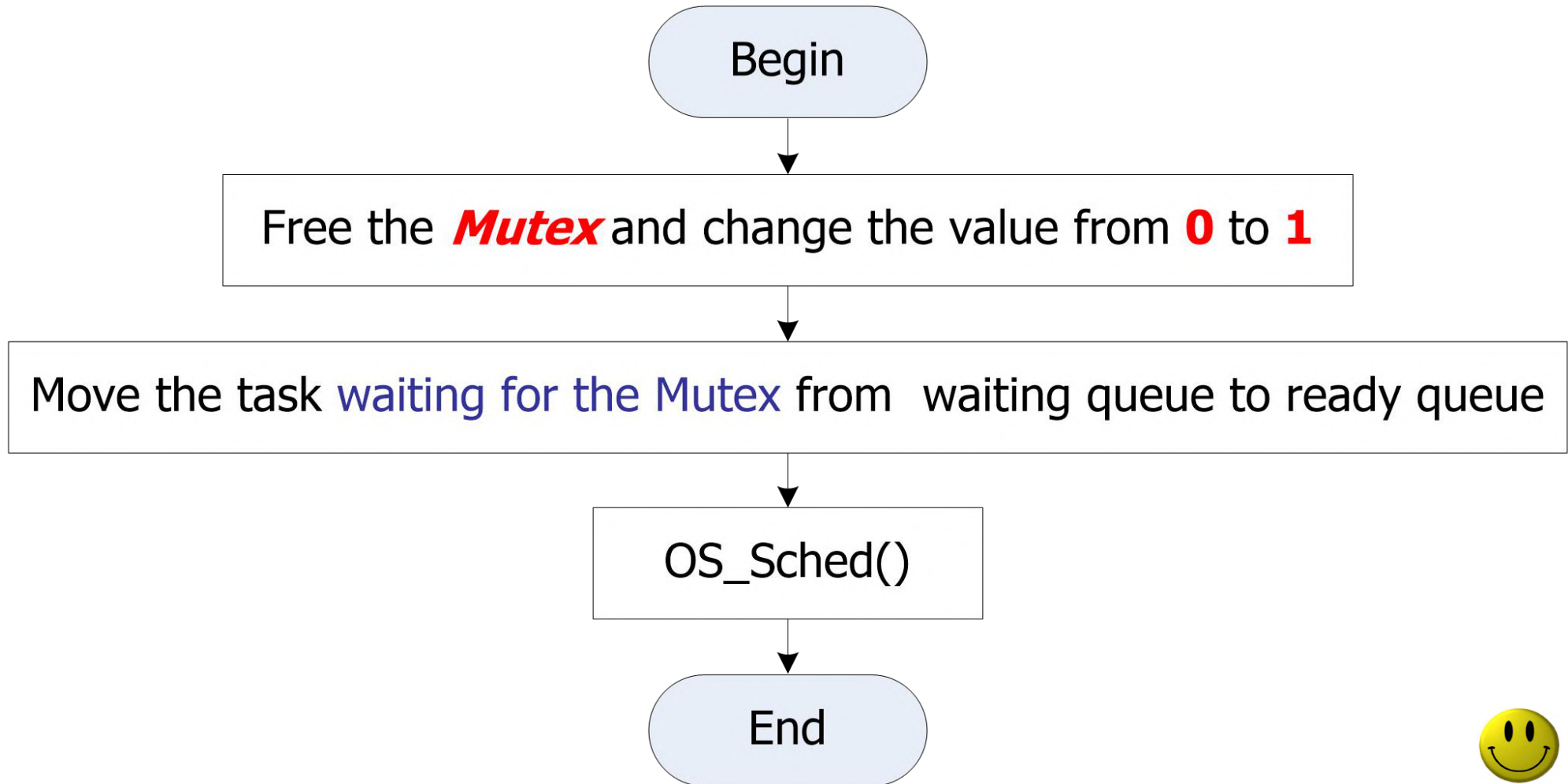


```

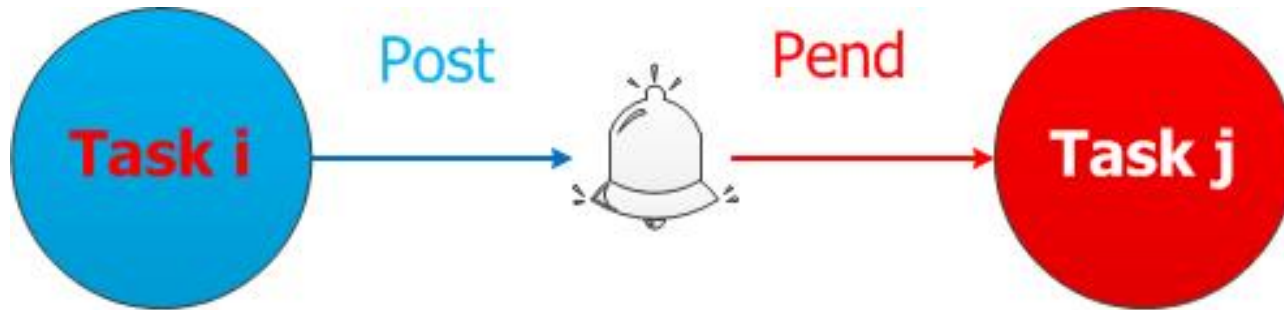
OS_EVENT *OSSemCreate (INT16U cnt)
{
    OS_EVENT *pevent;
    pevent = OSEventFreeList;           //Get next free event control block(ECB)
    if (OSEventFreeList != (OS_EVENT *)0) { //See if pool of free ECB pool was empty
        OSEventFreeList = (OS_EVENT *)OSEventFreeList->OSEventPtr;
    }
    if (pevent != (OS_EVENT *)0) {      //Get an ECB and initialize it
        pevent->OSEventType = OS_EVENT_TYPE_SEM; //Event type is Semaphore
        pevent->OSEventCnt = cnt;           //Set semaphore value
        pevent->OSEventPtr = (void *)0;     // Unlink from ECB free list
        OS_EventWaitListInit(pevent);       //Initialize to 'nobody waiting' on Sem
    }
    return (pevent);
}

```

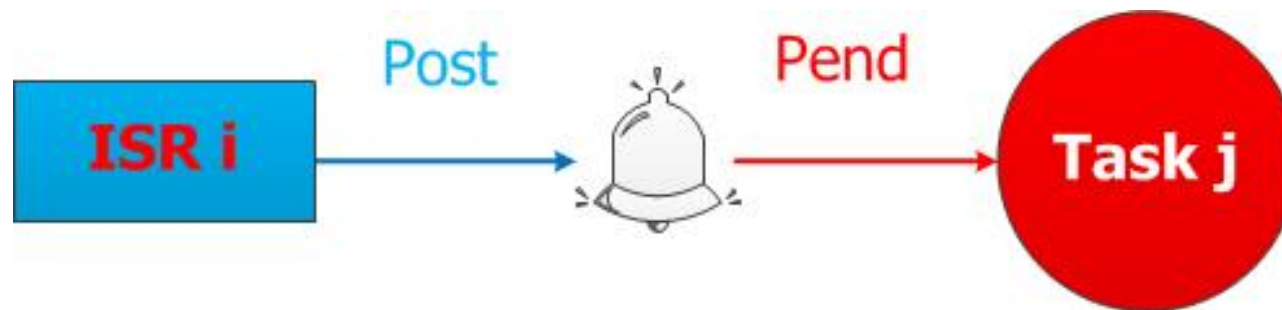




- **Synchronization:** A task can be synchronized with an ISR or another task, when no data is being exchanged, by using a semaphore as shown in Figure 3. Note that, in this case, the semaphore is drawn as a small bell, to indicate that it is used to signal the occurrence of an event (rather than to ensure mutual exclusion, in which case it would be drawn as a key). When used as a synchronization mechanism, the semaphore is initialized to 0.



Synchronization between tasks



Synchronization between ISR and task



■ APIs provided by the uC/OSII



➤ **Semaphore:** OSSemCreate ()
 OSSemPend ()
 OSSemPost ()

.....

.....



Sem=OSSemCreate(0)

.....

OSSemPend(Sem,0,&err);

.....

OSSemPost(Sem)

➤ **Message queue:** OSQCreate ()
OSQPend ()
OSQPost ()

.....

➤ **Message queue:** OSMboxCreate ()
OSMboxPend ()
OSMboxPost ()

.....

